

# NỀN TẢNG CÓ GIỚI HẠN

---



## TÓM TẮT

Phần A dựng đúng lượng nền cần thiết cho bốn phần sau: ngôn ngữ toán tối thiểu, và chuỗi vật lý biến ánh sáng thành tensor mà mô hình nhìn thấy. Đọc xong bạn tự derive lại được đạo hàm khi cần, và quan trọng hơn, bạn nhận ra được khi một giả định về ánh đầu vào đã bị vi phạm. Nếu chỉ nhớ một điều: phần lớn khoảng cách giữa lúc huấn luyện và lúc triển khai sinh ra ở khâu tạo ảnh, không ở kiến trúc mạng.

Phần này tồn tại để phục vụ bốn phần sau, và nguyên tắc chi phối nó là *học theo nhu cầu*: mỗi mục chỉ đi tới mức dùng được và giải thích được, không chứng minh định lý. Đây là một lựa chọn có chủ ý chứ không phải sự lười biếng. Một người dừng lại học trọn vẹn giải tích ma trận trước khi chạm vào mô hình đầu tiên thường không bao giờ tới được mô hình đầu tiên; ngược lại, một người bỏ hẵn phần nền sẽ đọc paper như đọc bùa chú và không bao giờ đoán được một phương pháp sẽ hỏng ở đâu. Ranh giới đúng nằm ở chỗ: học đủ để *tự derive lại được khi cần* và để *nhận ra khi một giả định bị vi phạm*.

Hai chương của phần có vai trò rất khác nhau. Chương 1 là ngôn ngữ: đại số tuyến tính, vi phân, xác suất, lý thuyết thông tin, hình học biến đổi và xử lý tín hiệu — không phải toàn bộ các môn đó, mà đúng những mảnh quay lại nhiều lần nhất trong phần B và C. Chương 2 là thứ bị bỏ sót nhiều nhất và cũng là thứ phân biệt người làm computer vision với người chỉ dùng CNN: ảnh không rơi từ trên trời xuống thành một tensor, nó là kết quả của một chuỗi vật lý và xử lý dài, và phần lớn khoảng cách giữa lúc huấn luyện và lúc triển khai sinh ra ở chuỗi đó chứ không ở kiến trúc mạng.

Vì vậy thứ tự đọc đề xuất không phải là đọc hết phần này rồi mới sang phần B. Hãy đọc chương 2 kỹ ngay từ đầu, còn chương 1 thì đọc lướt một lượt để biết trong đó có gì, rồi quay lại từng mục khi phần sau thực sự cần đến — mỗi mục trong chương 1 đều ghi rõ nó quay lại ở chương nào.

**Bảng 1.** Bản đồ Phần A: các chương, nội dung, và mức đầu tư đề xuất.

| Ch. | Thư mục           | Nội dung                                                                                                                                         | Mức                    |
|-----|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| 1   | 01-toan-toi-thieu | Ngôn ngữ toán tối thiểu: đại số tuyến tính, vi phân và tối ưu, xác suất và lý thuyết thông tin, hình học biến đổi và xử lý tín hiệu              | tham khảo, tra khi cần |
| 2   | 02-vat-ly-tao-anh | Ảnh sinh ra thế nào: ánh sáng và nhiễu, ISP và không gian màu, mô hình camera và hình học nhiều view, cảm biến ngoài RGB và nguồn gốc domain gap | <b>cốt lõi</b>         |

Chương 1 đọc lướt rồi quay lại từng mục khi phần B và C cần đến; chương 2 đọc kỹ ngay, vì phần lớn khoảng cách giữa huấn luyện và triển khai sinh ra ở đó.

# Toán tối thiểu dùng lại suốt đời (The Minimal Mathematics You Reuse Forever)

**PHỤ THUỘC** không — đây là chương mở đầu của cây. Chương này chỉ nên đọc *theo nhu cầu*: quay lại đúng mục cần khi một chương ở Phần B, C hay D đòi hỏi, chứ không đọc một mạch từ đầu tới cuối.

## TÓM TẮT

Chương này giữ lại đúng phần toán quay lại nhiều lần về sau, chia theo *vai trò* chứ không theo tên môn học. Mục 1 lo không gian và phổ. Mục 2 dựng bộ máy đưa mô hình từ khởi tạo tới nghiệm. Mục 3 nói nguồn gốc của hàm mất mát và độ tin cậy của một con số đo được. Mục 4 nói cấu trúc có sẵn trong dữ liệu ảnh. Đọc xong bạn chẩn đoán được nguyên nhân của một đường cong huấn luyện xấu, suy được loss từ giả định nhiễu thay vì tra bảng, và bắt được các lỗi im lặng ở khâu tiền xử lý. Nếu chỉ nhớ một điều: đây là chương [T1], nên dừng học cho đủ — hãy quay lại đúng mục mà một chương sau đang cần, vì kiến thức học không có nhu cầu đi kèm sẽ bay hết trong hai tuần.

## KIẾN THỨC CẦN CÓ TRƯỚC

- **deep learning** — cách tiếp cận trong đó ta xếp chồng nhiều tầng biến đổi có tham số, rồi để dữ liệu tự quyết định các tham số đó thông qua tối ưu hoá, thay vì tự tay thiết kế đặc trưng.
- **mô hình, tham số, huấn luyện** — mô hình là một hàm có tham số nhận đầu vào và trả về dự đoán; tham số là các con số bên trong nó; huấn luyện là quá trình tìm bộ tham số làm dự đoán sát nhãn nhất.
- **hàm mất mát (loss function)** — một con số đo mức sai của dự đoán so với nhãn — đại lượng mà quá trình huấn luyện thực sự cực tiểu.
- **tensor** — mảng số nhiều chiều; một ảnh màu là tensor ba chiều (cao  $\times$  rộng  $\times$  kênh), một batch ảnh là tensor bốn chiều.
- **nhãn tầng bán rã [T1]–[T4]** — quy ước của cây này về tuổi thọ của tri thức: [T1] là toán và vật lý gần như không đổi; [T2] là nguyên lý thiết kế hệ thống; [T3] là thuật toán và mô hình; [T4] là thư viện và công cụ, cũ nhanh nhất.
- **SLAM và hiệu chỉnh chùm tia (bundle adjustment)** — SLAM là bài toán vừa dựng bản đồ vừa định vị trong chính bản đồ đó; hiệu chỉnh chùm tia là bước tinh chỉnh đồng thời mọi tư thế camera và điểm 3D bằng bình phương tối thiểu. Chương này dùng chúng làm điểm neo quen thuộc, không đòi hỏi bạn phải cài được chúng.

## CÂU HỎI MỞ ĐẦU

- Q1. Nếu quy tắc chuỗi là thứ ai cũng học từ năm nhất, vì sao tính *gradient* cho một mạng  $10^9$  tham số lại từng là chuyện bất khả thi?
- Q2. Vì sao đổi đơn vị từ mét sang milimét trong một bài hiệu chỉnh chùm tia làm số điều kiện đổi ba bậc, trong khi bài toán không hề dễ hay khó đi?
- Q3. Hai mô hình chênh nhau một điểm phần trăm trên tập test 500 mẫu, mà khoảng tin cậy của mỗi mô hình rộng hơn năm điểm — vậy vì sao kết luận “A tốt hơn B” vẫn có thể đúng?
- Q4. Vì sao một mạng dự đoán phép quay bằng bốn số lại khó huấn luyện hơn bằng sáu số, dù bốn số đã thừa để mô tả ba bậc tự do?

Mọi lộ trình học deep learning đều bắt đầu bằng một chương toán, và phần lớn người học đều bỏ dở nó. Lý do không phải toán khó mà vì chương đó thường được viết như một giáo trình rút gọn: đầy đủ, tuyến tính, và không nói cho người đọc biết thứ nào sẽ quay lại vào lúc nào. Kết quả là người ta học đại số tuyến tính suốt sáu tuần rồi vẫn không biết vì sao huấn luyện của mình không hội tụ. Chương này cố ý làm ngược lại: chỉ giữ những gì *thật sự* quay lại nhiều lần, và mỗi mục phải trả lời được câu “thứ này xuất hiện lại ở đâu”.

Cách chia bốn mục theo vai trò, và giữa chúng có phụ thuộc thật chứ không phải thứ tự tùy tiện:

- **Mục 1 — đại số tuyến tính:** bộ từ vựng về không gian và phổ. Trả lời “bài toán này suy biến ở hướng nào, tôi mất bao nhiêu chữ số”.
- **Mục 2 — vi phân và tối ưu hoá:** bộ máy đưa mô hình từ khởi tạo tới nghiệm. *Phụ thuộc mục 1*, vì “hình dạng landscape” chính là phổ trị riêng của Hessian nói bằng lời khác.
- **Mục 3 — xác suất và lý thuyết thông tin:** trả lời hai câu mà hai mục trước không đụng tới — hàm mất mát từ đâu ra, và con số đo được đáng tin đến đâu. *Tiếp nhận câu hỏi mà mục 2 để ngỏ*.
- **Mục 4 — hình học và tín hiệu:** khác hẳn ba mục kia ở chỗ nó không nói về *cách học* mà về *cấu trúc có sẵn trong dữ liệu ảnh*. Đọc được đọc lập, và là cầu nối trực tiếp sang chương [A/02](#).

Thứ tự đọc đề nghị là  $1 \rightarrow 2 \rightarrow 3$ , còn mục 4 đọc được bất cứ lúc nào. Với người đã làm SLAM bằng C++, phần lớn mục 1 và nửa đầu mục 4 là ôn tập; bốn chỗ đáng dừng lại vì kinh nghiệm ước lượng trạng thái *không* chuyển sang miễn phí là: số điều kiện đọc theo nghĩa tốc độ huấn luyện, điều kiện dừng theo validation, toàn bộ mục 3, và phần lấy mẫu của mục 4.

## Bản đồ chương — Toán tối thiểu dùng lại suốt đời

### TÓM TẮT

Bản đồ này nói chương 1 gồm bốn mục nào, và vì sao chia theo vai trò chứ không theo tên môn học. Nó cũng nói mục nào phụ thuộc mục nào, và phần nào là cốt lõi so với phần nào chỉ cần biết là có. Đọc xong bạn quyết định được trong một phút nên đọc mục nào trước cho nhu cầu hiện tại thay vì đọc tuần tự cả chương. Nếu chỉ nhớ một điều: đây là chương [\[T1\]](#), nên nguyên tắc là học theo nhu cầu. Chuỗi phụ thuộc bắt buộc chỉ là  $1 \rightarrow 2 \rightarrow 3$ . Mục 4 vào được bất cứ lúc nào, và nên đọc ngay trước chương [A/02](#).

### A. Chương này có gì

- **01-dai-so-tuyen-tinh** — không gian con, phép chiếu, phân rã trị riêng và SVD, số điều kiện, xấp xỉ hạng thấp.
- **02-vi-phan-va-toi-uu** — quy tắc chuỗi dạng ma trận, Jacobian, Hessian, hạ gradient, lỗi so với không lồi, điều kiện dừng, hình dạng landscape.
- **03-xac-suat-va-thong-tin** — sai số lấy mẫu và khoảng tin cậy, MLE và MAP như nguồn gốc chung của mọi hàm mất mát, entropy-cross-entropy-KL và sự bất đối xứng của KL.
- **04-hinh-hoc-va-tin-hieu** — toạ độ đồng nhất,  $SO(3)$  và  $SE(3)$  ở mức ba thao tác dùng được, convolution, Fourier, lấy mẫu và aliasing.

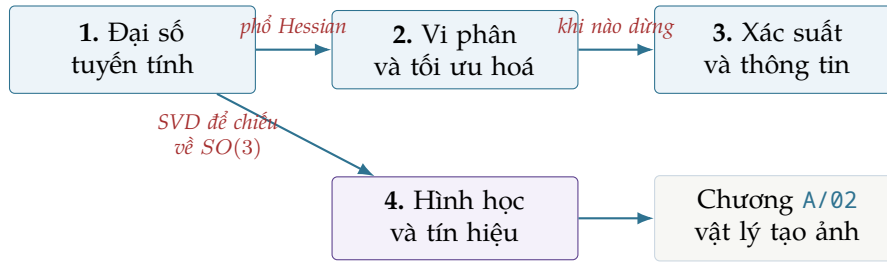
### B. Vì sao chia như vậy

Cách chia thông thường theo tên môn học — “đại số tuyến tính, giải tích, xác suất” — không nói thứ nào quay lại vào lúc nào. Nó vì thế khuyến khích học tuyến tính cho đủ, rồi quên sạch. Bốn

mục ở đây được đặt theo *câu hỏi chúng trả lời*: mục 1 trả lời “bài toán này suy biến ở hướng nào và tôi mất bao nhiêu chữ số”; mục 2 trả lời “làm sao đi từ khởi tạo tới nghiệm, và cái gì làm nó chậm”; mục 3 trả lời “hàm mất mát từ đâu ra, và con số tôi vừa đo đáng tin đến đâu”; mục 4 khác hẳn ba mục kia vì nó không nói về cách học mà về *cấu trúc có sẵn trong dữ liệu ảnh trước khi mô hình nhìn thấy*.

### C. Phụ thuộc và thứ tự đọc

Hình 1.1 vẽ quan hệ phụ thuộc thật giữa bốn mục.



**Hình 1.1.** Phụ thuộc giữa bốn mục của chương 1. Chuỗi ngang là chuỗi bắt buộc; mục 4 (tô khác màu) chỉ mượn một kết quả nhỏ của mục 1 nên vào được bất cứ lúc nào, và nó là cửa dẫn sang chương A/02.

### D. Cốt lõi hay tham khảo

Toàn chương thuộc tầng [T1], tức học theo nhu cầu và không đầu tư sâu. Trong đó:

- **Cốt lõi — phải nắm chắc, sẽ quay lại nhiều lần:** số điều kiện và xấp xỉ hạng thấp (mục 1); quy tắc khớp shape, chi phí bộ nhớ của lan truyền ngược, quan hệ độ cong–learning rate (mục 2); *toàn bộ* mục 3; phần lấy mẫu và aliasing (mục 4).
- **Tham khảo — biết là có, tra khi cần:** chi tiết các phân rã ma trận, phân tích lỗi hình thức, phần Lie algebra vượt quá ba thao tác exp, log và cập nhật trên không gian tiếp tuyến.

### E. Riêng với người đọc đã làm SLAM

Bốn chỗ mà kinh nghiệm ước lượng trạng thái *không* chuyển sang miễn phí, nên dừng lại kỹ: (i) số điều kiện đọc theo nghĩa tốc độ huấn luyện chứ không chỉ theo nghĩa ổn định số học; (ii) điều kiện dừng theo validation thay cho chuẩn gradient; (iii) toàn bộ mục 3, vì ước lượng trạng thái hiếm khi phải trả lời câu “hiệu số này có ý nghĩa thống kê không”; (iv) aliasing trong kiến trúc mạng, vốn không có đối ứng trong pipeline hình học cổ điển. Ngược lại, tọa độ đồng nhất,  $SE(3)$  và bình phương tối thiểu thì đọc lướt được.

## I 1.1 Đại số tuyến tính ứng dụng (Applied Linear Algebra)

**PHỤ THUỘC** không — đây là mục mở đầu của cây. Nếu bạn đã cài hiệu chỉnh chùm tia (*bundle adjustment*) thì phần không gian con là ôn lại; chỗ đáng đọc kỹ là *số điều kiện* và *hạng hiệu dụng*, hai chỗ mà ngôn ngữ SLAM và ngôn ngữ deep learning gọi cùng một hiện tượng bằng hai cái tên khác nhau.

### TÓM TẮT

Mục này rút đại số tuyến tính xuống đúng bốn thứ quay lại suốt cây: không gian con và phép chiếu, hai phân rã eigen và SVD, số điều kiện, xấp xỉ hạng thấp. Đọc xong bạn phân biệt được “huấn luyện

chậm vì bài toán điều hoà tệ” với “chậm vì learning rate sai”. Bạn đọc được phổ trị kỳ dị của một tầng đặc trưng. Và bạn biết vì sao một solver nghiêm túc không bao giờ lập  $A^T A$ . Toàn bộ được neo vào thứ bạn đã làm được: hiệu chỉnh chùm tia, ellipsoid hiệp phương sai, gauge freedom. Nếu chỉ nhớ một điều: bình phương tối thiểu là một phép chiếu vuông góc, còn số điều kiện là hệ số khuếch đại sai số của *bài toán* chứ không phải của thuật toán. Hệ quả: đối optimizer không cứu được một bài toán bị điều hoà tệ.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **Quy ước ký hiệu** — vector luôn là vector *cột*;  $A^T$  là ma trận chuyển vị; ma trận  $m \times n$  có  $m$  hàng và  $n$  cột;  $\|\cdot\|$  không ghi chỉ số nghĩa là chuẩn Euclid (chuẩn 2).
- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán. “Huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhãn nhất.
- **trọng số (weight)** — chính là các tham số của mô hình. Trong một tầng tuyến tính, chúng được xếp thành ma trận  $W$  và phép tính là  $Wx + b$ .
- **gradient** — vector gồm các đạo hàm riêng của một hàm theo từng tham số; nó chỉ hướng làm hàm tăng nhanh nhất, nên thuật toán tối ưu đi *ngược* hướng đó.
- **ma trận Hessian** — ma trận đạo hàm bậc hai; nó đo *độ cong* của hàm — hướng nào dốc gắt, hướng nào phẳng.
- **trị riêng và vector riêng** — vector riêng là hướng mà ma trận chỉ kéo dãn chứ không xoay; trị riêng là hệ số kéo dãn theo hướng đó. Tập hợp các trị riêng gọi là *phổ* của ma trận.
- **hạng (rank)** — số chiều thật sự mà đầu ra của ma trận trải ra. Hạng thấp nghĩa là ma trận nén dữ liệu về một không gian con nhỏ hơn.
- **hiệu chỉnh chùm tia (bundle adjustment)** — trong SLAM và dựng lại 3D, bài toán tinh chỉnh đồng thời mọi tư thế camera và mọi điểm 3D bằng cách cực tiểu tổng bình phương sai số tái chiếu. Nó là ví dụ chuẩn của bình phương tối thiểu phi tuyến quy mô lớn.

Đọc nhanh. Nếu bốn ý trên đã rõ, đọc thẳng tiểu mục D (số điều kiện) và E (giới hạn).

### A. Đặt vấn đề

Một kỹ sư SLAM giải hệ tuyến tính mỗi ngày mà không gọi tên nó là đại số tuyến tính: mỗi vòng *Gauss–Newton* của hiệu chỉnh chùm tia là một bài bình phương tối thiểu. Mỗi ma trận **hiệp phương sai** (*covariance*) của keyframe là một dạng toàn phương đối xứng. Và mỗi lần khôi phục pose từ ma trận cơ bản là một lần gọi **phân rã giá trị kỳ dị** (*singular value decomposition, SVD*). Khi bước sang deep learning, những đối tượng ấy quay lại nguyên vẹn nhưng dưới tên khác: “không gian đặc trưng” thay cho không gian cột, **rank collapse** thay cho ma trận suy biến, “*ill-conditioned loss*” thay cho số điều kiện lớn. Vấn đề không phải thiếu kiến thức mà thiếu ánh xạ giữa hai bộ từ vựng.

**Học xong mục này thì làm được gì mà trước đó không làm được?** Ba việc rất cụ thể: (i) nhìn một đường cong huấn luyện chậm và phân biệt được “bài toán bị điều hoà tệ” với “learning rate sai” với “mô hình quá nhỏ”; (ii) đọc một biểu đồ phổ trị kỳ dị của tầng đặc trưng và nói được mô hình đang thu hẹp biểu diễn hay không; (iii) biết vì sao một thư viện giải bài bình phương tối thiểu nghiêm túc không bao giờ lập ma trận  $A^T A$ .

Mục này không dạy lại đại số tuyến tính. Nó chọn ra bốn ý sẽ quay lại suốt Phần B, C, D và cho mỗi ý một chỗ neo trong thứ bạn đã làm được bằng C++:

- **Bốn ý được giữ lại, và câu hỏi mà mỗi ý trả lời:**
  - **không gian con** (*subspace*) và **phép chiếu** (*projection*) — “nghiệm bình phương tối thiểu là cái gì, chứ không phải tính bằng công thức nào”.

- **phân rã trị riêng** (*eigendecomposition*) và SVD — “ma trận này kéo dãn không gian theo những trục nào, mạnh yếu ra sao”.
- **số điều kiện** (*condition number*) — “tôi sắp mất bao nhiêu chữ số, và hướng nào của bài toán gần như không bị ràng buộc”.
- **xấp xỉ hạng thấp** (*low-rank approximation*) — “giữ lại bao nhiêu là đủ, và bỏ đi cái gì”.

Nguyên tắc của cả Phần A áp dụng nguyên: chỉ tới mức dùng được và giải thích được, không chứng minh định lý. Ai muốn chứng minh thì đọc [1]; bản trình bày định hướng học máy ở [2].

### TRỰC GIÁC

Bạn đã nhìn thấy SVD mỗi lần vẽ ellipsoid bất định của một keyframe mà không biết. Ellipsoid đó là hình ảnh của quả cầu đơn vị sau khi bị ma trận hiệp phương sai kéo dãn: hướng các trục chính là các **vector riêng** (*eigenvector*), độ dài các trục là căn của các **trị riêng** (*eigenvalue*). Mọi ma trận đều làm đúng ba động tác ấy với quả cầu đơn vị — xoay, kéo dãn theo vài trục ưu tiên, rồi xoay lần nữa — và SVD chỉ là biên bản ghi lại ba động tác. Khi ellipsoid dẹt gần thành đĩa, bài toán suy biến theo một hướng, và đó chính là lúc số điều kiện lớn.

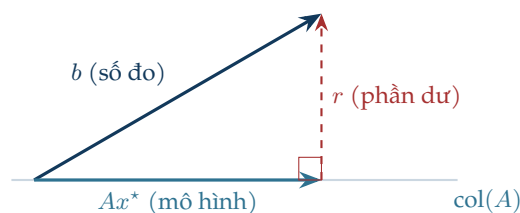
## B. Không gian con và phép chiếu: bình phương tối thiểu không phải một công thức

**Bình phương tối thiểu** (*least squares*) thường được nhớ như công thức  $x^* = (A^T A)^{-1} A^T b$ . Nhớ như vậy thì không giải thích được vì sao nó hoạt động và hỏng khi nào. Cách nhớ dùng được lâu hơn là hình học: cột của  $A$  căng ra một không gian con của  $\mathbb{R}^m$ ; vector đo  $b$  nói chung không nằm trong đó vì có nhiều; nghiệm bình phương tối thiểu là *hình chiếu vuông góc* của  $b$  xuống không gian cột, và phần dư  $r = b - Ax^*$  vuông góc với mọi cột của  $A$ . Điều kiện vuông góc viết ra là

$$A^T r = 0 \iff A^T A x = A^T b.$$

Nói bằng lời: *phương trình chuẩn tắc không phải một mẹo đại số, nó chỉ là cách viết lại câu “phần dư phải vuông góc với mọi thứ mô hình giải thích được”*. Công thức là hệ quả, hình học là nguyên nhân.

**Ví dụ nhỏ nhất tính tay được.** Khớp một hằng số  $c$  vào ba số đo 1, 2, 6. Ở đây  $A = (1, 1, 1)^T$ , không gian cột là đường chéo. Chiều  $b = (1, 2, 6)^T$  xuống đường ấy cho  $c = (1 + 2 + 6)/3 = 3$ , phần dư là  $(-2, -1, 3)^T$ , tổng bằng 0 — đúng là vuông góc với  $(1, 1, 1)^T$ . Trung bình cộng không phải một quy ước thống kê; nó là một phép chiếu. Hình 1.2 vẽ lại quan hệ đó ở dạng hai chiều.



**Hình 1.2.** Nghiệm bình phương tối thiểu là hình chiếu vuông góc của số đo  $b$  xuống không gian cột của  $A$ . Mọi biến thể — có trọng số, có ràng buộc, phi tuyến sau khi tuyến tính hoá — giữ nguyên bức tranh này và chỉ đổi cách đo góc vuông.

Bài toán phi tuyến trong SLAM không nằm ngoài bức tranh đó: Gauss–Newton tuyến tính hoá phần dư quanh điểm hiện tại, giải một bài chiếu, đi một bước, lặp lại. Ma trận  $H = J^T J$  mà bạn quen gọi là “Hessian xấp xỉ” chính là  $A^T A$  của bài chiếu ấy. Trong deep learning nó xuất hiện lại dưới tên **Gauss–Newton matrix**, và là bộ khung của mọi phương pháp bậc hai xấp xỉ. Đây là chỗ nối đầu tiên trong nhiều chỗ nối: cùng một ma trận, hai cộng đồng, hai cái tên.

## I Mạch 1 — từ phép chiếu tới các phân rã

- 1 Vấn đề** giải  $A^\top Ax = A^\top b$  trực tiếp mất chính xác một cách khó hiểu.

**Giải pháp** không lập  $A^\top A$  mà phân rã  $A$  bằng **QR** hoặc SVD rồi giải trên dạng đã phân rã.

**Hệ quả** trị kỳ dị của  $A^\top A$  là bình phương trị kỳ dị của  $A$ , nên số điều kiện bị bình phương và bạn mất khoảng một nửa số chữ số có nghĩa.
- 2 Vấn đề** nhưng hiệu chỉnh chùm tia vẫn lập  $J^\top J$  mỗi vòng.

**Giải pháp** bài toán được điều hoà bằng **giảm chấn** (*damping*) kiểu Levenberg–Marquardt, và cấu trúc thưa làm **Cholesky** rẻ hơn QR nhiều bậc.

**Giá phải trả** bạn đánh đổi độ chính xác số học lấy tốc độ, và trả giá đúng lúc bài toán suy biến — ví dụ chuỗi keyframe gần thẳng hàng.
- 3 Vấn đề** cần biết “hướng nào của bài toán yếu”.

**Giải pháp** phân rã trị riêng của ma trận đối xứng  $H$ , hoặc SVD của  $J$ .

**Hệ quả** trị riêng nhỏ nhất chỉ ra hướng mà dữ liệu gần như không ràng buộc: trong SLAM đó là *gauge freedom*, trong deep learning đó là hướng phẳng của **bề mặt hàm mất mát** (*loss landscape*) mà **hạ gradient** (*gradient descent*) đi rất chậm.

## C. Ba phân rã, chọn cái nào

Ba công cụ hay bị dùng lẫn. Bảng 1.1 đặt chúng cạnh nhau; cột cuối là cột đáng giá nhất.

**Bảng 1.1.** Ba phân rã ma trận dùng nhiều nhất, so trên cùng tiêu chí.  $m \times n$  với  $m \geq n$ .

| Phân rã  | Cơ chế                                             | Chi phí                            | Mạnh khi nào                                                 | Hỏng / không dùng được khi                                           |
|----------|----------------------------------------------------|------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------|
| Cholesky | $H = LL^\top$ trên ma trận đối xứng xác định dương | $\frac{1}{3}n^3$ , rất rẻ khi thưa | Bài toán lớn, thưa, đủ điều hoà — lựa chọn mặc định trong BA | $H$ suy biến hoặc gần suy biến; đã bình phương số điều kiện từ trước |
| QR       | Trực giao hoá cột của $A$                          | $\approx 2mn^2$                    | Bình phương tối thiểu cần độ chính xác, $A$ điều hoà kém     | Ma trận rất lớn; không cho biết phổ trị kỳ dị                        |
| SVD      | $A = U\Sigma V^\top$ , mọi ma trận                 | $\approx O(mn^2)$ , đắt nhất       | Cần biết hạng, cần xấp xỉ hạng thấp, cần chiếu về $SO(3)$    | Kích thước lớn (phải dùng phương pháp lặp hoặc phác thảo ngẫu nhiên) |

Quan hệ giữa phân rã trị riêng và SVD gọn trong một câu: trị kỳ dị của  $A$  là căn bậc hai của trị riêng của  $A^\top A$ , và vector kỳ dị phải là vector riêng của  $A^\top A$ . Phân rã trị riêng chỉ đẹp khi ma trận đối xứng (hiệp phương sai, Hessian, ma trận Gram của đặc trưng); SVD tồn tại cho mọi ma trận nên nó là lựa chọn mặc định khi ma trận là một phép biến đổi giữa hai không gian khác nhau, chẳng hạn ma trận trọng số của một tầng tuyến tính.

Hệ quả thực dụng dùng lại nhiều nhất là **định lý xấp xỉ hạng thấp**: cắt bỏ các trị kỳ dị nhỏ cho ra ma trận hạng  $k$  gần  $A$  nhất. Ba chỗ trong cây này sống trên nó:

- **Nén và phân tích biểu diễn — phân tích thành phần chính** (*principal component analysis, PCA*) chỉ là chiếu dữ liệu đã trừ trung bình lên  $k$  vector kỳ dị phải đầu tiên.

- **Ép về cấu trúc hợp lệ** — ước lượng ma trận cơ bản rồi ép trị kỳ dị thứ ba về không; chiếu một ma trận nhiều về  $SO(3)$  bằng cách đặt cả ba trị kỳ dị bằng một. Đây là xấp xỉ hạng thấp có ràng buộc, và nó quay lại ở A/01/04 khi mạng phải xuất ra một phép quay.
- **Tinh chỉnh rẻ** — *LoRA* [3] giả thiết phần cập nhật trọng số khi tinh chỉnh có hạng thấp, nên chỉ học hai ma trận gầy thay vì cả ma trận đầy.

↔ **trường hợp riêng của C** — LoRA áp dụng định lý xấp xỉ hạng thấp, nhưng cho ma trận *cập nhật trọng số* thay vì cho ma trận dữ liệu

#### D. Số điều kiện: đại lượng dự báo rắc rối

Số điều kiện  $\kappa(A) = \sigma_{\max}/\sigma_{\min}$  trả lời đúng một câu hỏi: nếu đầu vào sai lệch tương đối  $\varepsilon$ , nghiệm có thể sai lệch tương đối tới  $\kappa\varepsilon$ . Nói bằng lời:  $\kappa$  là *hệ số khuếch đại sai số của bài toán, không phải của thuật toán*.

**Ví dụ tính tay.** Với  $A = \text{diag}(1; 0,01)$  thì  $\kappa = 100$ ; một nhiễu 1% đặt đúng vào thành phần thứ hai của  $b$  làm thành phần thứ hai của nghiệm sai 100%. Ma trận không hề suy biến, định thức khác không, mọi phép tính vẫn chạy — và kết quả vẫn vô nghĩa. Đó là lý do định thức gần như vô dụng làm chỉ báo, còn  $\kappa$  thì không.

Trong deep learning cùng đại lượng ấy chi phối tốc độ hội tụ, và gần như mọi “mẹo huấn luyện” quen thuộc là một cách hạ  $\kappa$  gián tiếp:

- **Chuẩn hoá đầu vào và chuẩn hoá theo batch (*batch normalization*)** [4] — kéo các hướng về cùng thang đo, tức trực tiếp giảm tỉ số trị riêng lớn nhất trên nhỏ nhất.
- *Adam* [5] — chia mỗi toạ độ cho một ước lượng độ lớn riêng, tương đương đổi thang đo từng chiều, nên nó “thấy” một bài toán điều hoà tốt hơn bài toán thật.
- **Khởi tạo cẩn thận** — giữ phổ trị kỳ dị của từng tầng không quá lệch ngay từ đầu, để  $\kappa$  không tích lũy qua độ sâu.

↔ **hệ quả của D** — chuẩn hoá, khởi tạo và optimizer thích nghi ở chương huấn luyện đều là cách hạ số điều kiện một cách gián tiếp

Một câu để mang theo: *phần lớn cái gọi là mẹo huấn luyện là kỹ thuật điều hoà số học đội lốt*. Mạch này quay lại đầy đủ ở D khi bàn optimizer.

#### E. Giới hạn và trường hợp hỏng

Bốn công cụ trên có giới hạn rõ, và biết giới hạn quan trọng hơn biết công thức.

- **Số điều kiện phụ thuộc đơn vị đo.** Trong một bài BA trộn tịnh tiến tính bằng mét với góc quay tính bằng radian, đổi mét sang milimét làm  $\kappa$  đổi ba bậc mà bài toán không dễ hay khó đi. So sánh  $\kappa$  giữa hai mô hình tham số hoá khác nhau là so sánh vô nghĩa.
- **Ma trận suy biến không phải lúc nào cũng là bệnh.** Hessian của BA không ràng buộc gauge suy biến đúng bảy chiều — đó là cấu trúc thật của bài toán. Hessian của mạng nơ-ron cũng suy biến vì nhiều đối xứng nội tại: hoán vị neuron trong một tầng, co giãn bù trừ giữa hai tầng liên tiếp qua *ReLU*. Cách xử lý đúng là cố định gauge hoặc chấp nhận, không phải thêm nhiễu cho đủ hạng.
- **PCA giả định cấu trúc tuyến tính và đo bằng phương sai.** Đặc trưng của mạng sâu nằm trên một đa tạp cong. Vì thế “95% phương sai trong  $k$  chiều” không có nghĩa thông tin hữu ích nằm ở đó. Hướng phương sai lớn thường là độ sáng toàn cục hoặc phong cách ảnh; hướng quyết định nhân có thể nằm ở phương sai nhỏ.

↔ **tiền đề E** — *linear probe* là bài bình phương tối thiểu trên không gian đặc trưng, nên chất lượng của nó bị chi phối bởi chính số điều kiện ở đây

Vì thể trực quan hoá đặc trưng hiện đại dùng *t-SNE* [6] hay *UMAP* [7]. Nhưng hai thứ đó đánh đổi tính trung thực về khoảng cách toàn cục — một trade-off mỗ ở E.

- **Chi phí.** SVD đầy đủ của ma trận  $50\,000 \times 2048$  là chuyện của phương pháp lập hoặc phác thảo ngẫu nhiên, không phải một lời gọi thư viện vô tư.

#### CẠM BÃY

Đừng đọc “hạng thấp” như “ít thông tin”. Ma trận đặc trưng của một mô hình tự giám sát bị *rank collapse* là bệnh; ma trận cập nhật LoRA có hạng thấp là giả thiết thiết kế có chủ đích. Cùng một con số hạng, hai ý nghĩa trái ngược, và cái phân biệt chúng là *ma trận đó đang biểu diễn dữ liệu hay biểu diễn một phép hiệu chỉnh*. Trong thực hành nên đo **hạng hiệu dụng**, tức entropy của phổ trị kỳ dị đã chuẩn hoá. Đừng đếm số trị kỳ dị khác không: với số dấu phẩy động thì hầu như không trị kỳ dị nào đúng bằng không.

#### CẦN NHỚ

Bình phương tối thiểu là một phép chiếu vuông góc. Phân rã là danh sách trực kèm hệ số kéo dẫn. Số điều kiện nói bạn sắp mất bao nhiêu chữ số có nghĩa — và trong huấn luyện, nó nói bạn sắp chờ lâu cỡ nào. Ba câu đó phủ gần hết chỗ đại số tuyến tính cần cho cả cây này. [T1]

## F. Kết nối

Mục này là tiền đề trực tiếp của [A/01/02-vi-phan-va-toi-uu](#): hình dạng landscape bàn ở đó chính là phổ trị riêng của Hessian nói ở đây. Nó cũng là tiền đề của [A/01/04-hinh-hoc-va-tin-hieu](#), nơi  $SO(3)$  xuất hiện như một ràng buộc trực giao mà SVD dùng để ép về. Theo chiều ngang, C dùng lại xấp xỉ hạng thấp ở phần tinh chỉnh hiệu quả tham số. Còn E dùng lại phép chiếu khi bàn *linear probe*. Một linear probe *chính là* bài bình phương tối thiểu trên không gian đặc trưng, nên nó thừa hưởng đúng số điều kiện đã bàn ở trên. Mini project cuối chương (PCA và *whitening* trên CIFAR-10) là bài kiểm tra thực hành cho toàn mục.

### ▮ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao giải phương trình chuẩn tắc bằng cách lập  $A^T A$  làm mất khoảng một nửa số chữ số có nghĩa, trong khi QR thì không?
2. Số điều kiện lớn của một ma trận trọng số nói gì và *không* nói gì về độ ổn định huấn luyện?
3. Hessian của hiệu chỉnh chùm tia suy biến bảy chiều. Vì sao đó không phải lỗi, và làm gì thì đúng?
4. Bạn đo được 95% phương sai của đặc trưng nằm trong 10 chiều đầu. Vì sao điều đó không cho phép kết luận có thể nén đặc trưng về 10 chiều mà không mất độ chính xác?
5. Cùng là “hạng thấp”, vì sao rank collapse trong học tự giám sát là bệnh còn LoRA lại là tính năng?
6. Bạn phải chọn giữa Cholesky, QR và SVD cho một bài bình phương tối thiểu 20 nghìn phương trình, 300 ẩn, ma trận thưa và điều hoà tốt. Chọn cái nào và vì sao?

# Nguồn

---

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [29] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [30] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [31] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [32] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [33] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [34] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [37] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [38] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [39] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.

## 1.2 Vi phân nhiều biến và tối ưu hoá (Matrix Calculus and Optimization)

**PHỤ THUỘC** [A/01-toan-toi-thieu/01-dai-so-tuyen-tinh](#) (phổ trị riêng, số điều kiện — toàn bộ phần “hình dạng landscape” dưới đây là cách đọc lại phổ ấy).

### TÓM TẮT

Mục này giải thích cấu trúc của phép lấy đạo hàm trên đồ thị tính toán và hình dạng của bài toán tối ưu mà huấn luyện thực sự giải. Đọc xong bạn tự derive được backward của một tầng bất kỳ mà không nhầm chiều. Bạn biết vì sao **lan truyền ngược** (*backpropagation*) đắt về bộ nhớ chứ không đắt về phép tính. Và bạn chẩn đoán được “huấn luyện chậm” thuộc loại nào trong ba loại: điều hoà tệ, learning rate sai, hay tín hiệu giám sát yếu. Nếu chỉ nhớ một điều: *khớp shape trước, đầu và hệ số sau* — và trước khi đổ lỗi cho tính không lỗi, hãy kiểm tra số điều kiện, vì phần lớn “bài toán khó tối ưu” hoá ra chỉ là bài toán bị điều hoà tệ.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình và tham số** — mô hình là một hàm có tham số nhận đầu vào và trả về dự đoán; tham số là các con số bên trong hàm đó mà quá trình huấn luyện được phép thay đổi.
- **hàm mất mát (loss function)** — một con số đo mức sai của dự đoán so với nhãn. Huấn luyện là quá trình đi tìm bộ tham số làm con số này nhỏ nhất.
- **gradient** — vector các đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm loss tăng nhanh nhất, nên ta cập nhật theo hướng ngược lại.
- **learning rate** — độ dài bước mà thuật toán tối ưu đi theo hướng ngược gradient ở mỗi lần cập nhật. Quá lớn thì nghiệm bật ra ngoài và phân kỳ; quá nhỏ thì hội tụ chậm tới mức vô dụng.
- **mini-batch và epoch** — mini-batch là một nhóm nhỏ mẫu dùng để ước lượng gradient cho một lần cập nhật; một epoch là một lượt đi hết toàn bộ tập huấn luyện.
- **tập validation** — phần dữ liệu tách riêng, không dùng để cập nhật tham số, chỉ dùng để đo xem mô hình đang tốt lên hay xấu đi. Nó tồn tại vì loss trên tập huấn luyện luôn giảm kể cả khi mô hình đang học vẹt.
- **hàm lồi (convex)** — hàm mà đoạn thẳng nối hai điểm bất kỳ trên đồ thị luôn nằm trên đồ thị — hình dạng “lồng chảo”. Với hàm lồi, mọi cực tiểu địa phương đều là cực tiểu toàn cục; mạng nơ-ron thì không lồi.
- **Jacobian và Hessian** — Jacobian là ma trận đạo hàm bậc nhất của một hàm nhiều đầu vào nhiều đầu ra — nó là bản tuyến tính hoá tốt nhất tại một điểm. Hessian là ma trận đạo hàm bậc hai, đo độ cong.

#### A. Đặt vấn đề

Bạn đã viết **ma trận Jacobian** (*Jacobian*) bằng tay cho một hàm chi phí **sai số tái chiếu** (*reprojection error*) và đã từng mất một buổi chiều vì nhầm dấu hoặc nhầm thứ tự nhân. Trong deep learning, **autograd** làm hộ việc đó, nên câu hỏi đổi hình. Không còn là “đạo hàm bằng bao nhiêu”, mà là “vì sao **gradient** chết ở tầng thứ ba”, “vì sao đổi thứ tự hai phép toán làm bộ nhớ tăng gấp đôi”, và “vì sao **hàm mất mát** (*loss function*) giảm nhưng mô hình không học được gì”. Trả lời những câu đó cần hiểu *cấu trúc* của phép lấy đạo hàm, không cần kỹ năng đạo hàm tay.

**Học xong mục này thì làm được gì mà trước đó không làm được?** Ba việc: viết và kiểm chứng một **lượt ngược** (*backward pass*) tự cài trong mười lăm phút thay vì nửa ngày; đọc một đường cong huấn luyện và đặt tên đúng cho nguyên nhân trước khi thử ngẫu nhiên các siêu tham số; và hiểu vì sao mọi optimizer hiện đại đều là biến thể của cùng một ý tưởng, nên không cần học thuộc từng cái.

#### B. Bộ máy vi phân: khớp shape trước

##### TRƯỚC GIÁC

Quy tắc chuỗi ở dạng ma trận là một bài toán *lắp ống dẫn*. Mỗi tầng là một cái ống có đường kính vào và đường kính ra; Jacobian của tầng là tấm chuyển tiếp kích thước (đường kính ra)  $\times$  (đường kính vào). Backprop là ghép các tấm chuyển tiếp từ cuối về đầu, và cách duy nhất để ghép sai là ghép hai tấm không khớp đường kính. Vì thế: khớp shape trước, lo dấu và hệ số sau — nếu shape khớp thì công thức chỉ còn sai được ở một hằng số hoặc một dấu trừ, và cả hai lộ ra ngay trong một lần **kiểm tra gradient** (*gradient check*).

Với hàm  $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ , Jacobian  $J \in \mathbb{R}^{m \times n}$  là bản tuyến tính hoá tốt nhất tại một điểm:

$$f(x + \delta) \approx f(x) + J\delta.$$

Nói bằng lời: *trong một lân cận đủ nhỏ, mọi hàm trơn đều là một phép nhân ma trận* — và đó là toàn bộ lý do vi phân hữu ích. **Quy tắc chuỗi** (chain rule) nói Jacobian của hàm hợp là tích các Jacobian theo đúng thứ tự hợp,  $J_{g \circ f} = J_g J_f$ .

Toàn bộ lan truyền ngược [8] chỉ là một nhận xét về *thứ tự nhân*. Đầu ra cuối là một số vô hướng, tức  $m = 1$  ở tầng hàm mất mát. Khi đó nhân tích ấy từ trái sang phải rẻ hơn từ phải sang trái rất nhiều. Lý do: mỗi bước ta nhân một vector hàng với một ma trận, thay vì nhân hai ma trận. Đó là lý do nó tên là *reverse-mode automatic differentiation*, tức là đi ngược từ đầu ra vô hướng về từng tham số; cần phân biệt *lan truyền ngược* là thuật toán còn **lượt ngược** (backward pass) là một lần chạy nó. Đây cũng là lý do nó tốn bộ nhớ: muốn nhân ngược thì phải giữ lại giá trị trung gian của lượt xuôi. Đánh đổi bộ nhớ lấy tính toán ấy quay lại nguyên vẹn ở D dưới cái tên **gradient checkpointing** [9].

↪ **ràng buộc bởi** D — bộ nhớ kích hoạt do lượt xuôi để lại mới là thứ chặn batch size trên một GPU, không phải số trọng số

**Phiên bản đơn giản hoá của mẹo khớp shape.** Nếu  $Y = XW$  với  $X \in \mathbb{R}^{B \times d_{in}}$ ,  $W \in \mathbb{R}^{d_{in} \times d_{out}}$ , thì gradient của một đại lượng phải có đúng shape của chính đại lượng đó. Vậy  $\partial L / \partial W$  phải là  $d_{in} \times d_{out}$ ; hai thứ có sẵn để ghép ra kích thước đó là  $X^\top$  ( $d_{in} \times B$ ) và  $\partial L / \partial Y$  ( $B \times d_{out}$ ); chỉ có một cách ghép hợp lệ, nên

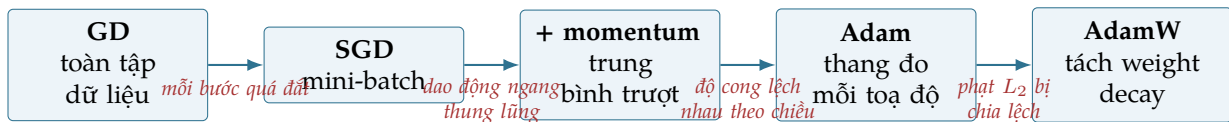
$$\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial Y}, \quad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W^\top.$$

Không cần nhớ gì, chỉ cần đếm chiều. Quy tắc này giải quyết khoảng chín phần mười trường hợp thực tế; phần còn lại — chuẩn hoá, **softmax**, phép toán chia sẻ trọng số — thì kiểm bằng `torch.autograd.gradcheck` trên đầu vào kiểu double, và nếu sai thì gần như luôn sai ở một chỗ tổng-hợp-theo-*batch* bị bỏ quên.

**Ma trận Hessian** (Hessian) gần như không bao giờ được lập tường minh, vì kích thước của nó là bình phương số tham số. Nhưng *tích Hessian với một vector* tính được với chi phí cỡ một lượt ngược, và thế là đủ để đo độ cong theo một hướng. Vai trò của Hessian ở đây không phải để làm bước Newton mà để *đọc bài toán*: trị riêng lớn nhất chặn **learning rate** ổn định, phổ trị riêng cho biết landscape dẹt hay nhọn, trị riêng âm cho biết ta đang ở **điểm yên ngựa** (saddle point) chứ không phải cực tiểu.

### C. Vì sao optimizer có hình dạng như hiện nay

Hình 1.3 đọc theo trục thời gian: mỗi bước sinh ra vì bước trước để lại một nút thắt cụ thể, không phải vì ai đó muốn thêm một siêu tham số.



**Hình 1.3.** Dòng tiến hoá của optimizer. Nhãn dưới mỗi mũi tên là *nút thắt* mà bước kế tiếp gỡ; đọc theo chiều này thì không cần học thuộc từng thuật toán, chỉ cần nhớ nó sinh ra để chữa cái gì.

## I Mạch 2 — cùng dòng tiến hoá, kèm giá phải trả

### 1 Vấn đề

hàm mất mát nhiều chiều, không có nghiệm đóng.

### Giải pháp

đi ngược hướng gradient từng bước nhỏ.

### Giả định

hàm khả vi và trơn đủ để tuyến tính hoá còn đúng trong lân cận cỡ learning rate.

### Giá phải trả

phải chọn learning rate; chọn sai thì hoặc phân kỳ hoặc bò.

- 2 Vấn đề** tính gradient trên toàn tập dữ liệu quá đắt.  
**Giải pháp** ước lượng trên một mini-batch.  
**Hệ quả** mỗi bước rẽ đi hàng nghìn lần nên ta đi được nhiều bước hơn với cùng ngân sách.  
**Vấn đề mới** gradient thành biến ngẫu nhiên, với nhiều tỉ lệ nghịch với căn của **kích thước batch** (*batch size*). Tăng batch gấp bốn chỉ giảm nhiễu một nửa. Quan hệ lợi ích giảm dần đó quyết định phần lớn cách chọn batch size [10].
- 3 Vấn đề** các hướng có độ cong rất khác nhau nên một learning rate chung không vừa cho hướng nào.  
**Giải pháp** co giãn từng tọa độ theo lịch sử độ lớn gradient, tức họ Adam [5].  
**Giả định** độ cong xấp xỉ tách rời được theo từng tọa độ.  
**Giá phải trả** mất đảm bảo hội tụ của SGD trong vài trường hợp, và sinh tương tác lạ với *weight decay* — chính chỗ đó đẻ ra AdamW [11].
- 4 Vấn đề** nhiều mini-batch làm quỹ đạo dao động ở cuối quá trình.  
**Giải pháp** giảm learning rate theo lịch trình.  
**Hệ quả** bước cuối đủ nhỏ để trung bình hoá nhiễu.  
**Vấn đề mới** lịch trình thành một siêu tham số nhạy, tương tác với cả batch size lẫn số epoch — nên chép kiến trúc mà không chép lịch trình thì không tái lập được kết quả.

**Bảng 1.2.** Bốn optimizer thường gặp trên cùng tiêu chí. Cột cuối quan trọng nhất.

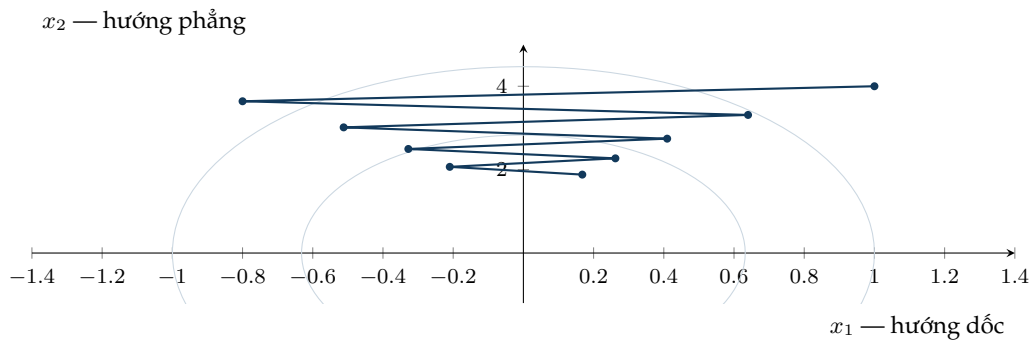
| Thuật toán     | Cơ chế                                           | Mạnh khi nào                                                                 | Hổng khi nào                                                                                           |
|----------------|--------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| SGD            | Bước theo gradient mini-batch                    | Bài toán điều hoà tốt sau chuẩn hoá; hay tổng quát hoá tốt hơn trên thị giác | Landscape lệch thang đo mạnh; cần rất nhiều công tune learning rate                                    |
| SGD + momentum | Trung bình trượt gradient                        | Thung lũng hẹp dài — triệt tiêu dao động ngang                               | Momentum quá lớn gây vọt lố ở giai đoạn cuối                                                           |
| Adam           | Chia mỗi tọa độ cho ước lượng độ lớn riêng       | Gradient thưa hoặc lệch thang đo mạnh, ví dụ <i>transformer</i>              | Đầu quá trình khi ước lượng phương sai chưa ổn (cần <i>warmup</i> ); tổng quát hoá đôi khi kém hơn SGD |
| AdamW          | Như Adam, tách weight decay khỏi bước thích nghi | Mặc định hợp lý cho hầu hết mô hình lớn hiện nay                             | Vẫn cần warmup và lịch trình; không cứu được bài toán mà tín hiệu giám sát sai                         |

#### D. Lỗi hay không lỗi, và hình dạng thật của landscape

Câu nói phổ thông “deep learning khó vì hàm mất mát không lồi” đúng về chữ nhưng dẫn tới trực giác sai. Với bài toán **lồi** (*convex*), mọi cực tiểu địa phương là cực tiểu toàn cục và ta có chứng nhận dừng. Với bài toán không lồi cả hai thứ đó mất. Nhưng thực nghiệm trên mạng lớn cho thấy thủ phạm làm chậm không phải các cực tiểu địa phương tồi: trong không gian rất nhiều chiều, một điểm dừng ngẫu nhiên gần như chắc chắn là điểm yên ngựa, vì để là cực tiểu thì hàng triệu trị riêng phải dương *cùng lúc*. Cái làm chậm là vùng phẳng quanh yên ngựa và vùng có số điều kiện tệ — tức đúng hiện tượng ở mục trước.

**Ví dụ nhỏ nhất tính tay được, và nó đủ giải thích cả hiện tượng.** Với  $L(x) = \frac{1}{2}(x_1^2 + \varepsilon x_2^2)$ , hạ gradient bước  $\eta$  nhân thành phần thứ nhất với  $(1 - \eta)$  và thành phần thứ hai với  $(1 - \eta\varepsilon)$  mỗi

vòng. Ổn định buộc  $\eta < 2$ . Lấy  $\eta = 1,8$  và  $\varepsilon = 0,05$ . Hướng thứ nhất dao động đổi dấu mỗi bước với hệ số  $-0,8$ ; hướng thứ hai mỗi vòng chỉ có 9%. Tỉ số  $1/\varepsilon = 20$  chính là số điều kiện. Hình 1.4 vẽ đúng quỹ đạo đó: chi phí không nằm ở chỗ “đi xuống”, nó nằm ở chỗ đi ngang thung lũng.



**Hình 1.4.** Hạ gradient trên hàm toàn phương có số điều kiện 20: quỹ đạo nảy qua nảy lại theo hướng dốc trong khi bò rất chậm theo hướng phẳng. Chuẩn hoá đầu vào và các optimizer thích nghi đều nhăm vào việc làm hai trục của ellipse gần bằng nhau, tức là làm hình này tròn lại.

Khác biệt thứ hai, thực dụng hơn: khi không lỗi, *cực tiểu nào* ta rơi vào là một phần của kết quả. Có bằng chứng thực nghiệm rằng nghiệm ở vùng dẹt tổng quát hoá tốt hơn nghiệm ở vùng nhọn, và batch lớn có xu hướng rơi vào vùng nhọn nếu không hiệu chỉnh gì [12]. Đây là quan sát đo được và tái lập nhiều lần, nhưng cách giải thích vẫn còn tranh cãi và định nghĩa “dẹt” phụ thuộc cách tham số hoá — hãy nhớ ở mức hiện tượng, đừng nhớ ở mức định lý.

### E. Điều kiện dừng nào có nghĩa

Trong tối ưu cổ điển ta dừng khi chuẩn gradient nhỏ. Trong huấn luyện mạng, tiêu chí đó gần như vô dụng: gradient mini-batch không bao giờ nhỏ vì nhiều lấy mẫu, và ngay cả gradient toàn phần nhỏ cũng không nói gì về chất lượng mô hình. Thứ thực sự được dùng là dừng theo chỉ số trên tập *validation* — tức dừng khi mục tiêu ta *quan tâm*, không phải mục tiêu ta *tối ưu*, ngừng cải thiện.

→ **tiền đề A/01** — câu hỏi “validation đã cải thiện chưa” là câu hỏi thống kê về sai số lấy mẫu, được trả lời ở mục xác suất kế tiếp

Chuyển từ “điều kiện dừng của thuật toán” sang “điều kiện dừng của phép ước lượng ngoài mẫu” là một dịch chuyển tư duy lớn. Nó cũng là cầu nối sang A/01/03. Câu hỏi “validation đã ngừng cải thiện chưa” là câu hỏi thống kê, không phải câu hỏi giải tích.

### F. Giới hạn và trường hợp hỏng

- **Giả định khả vi bị vi phạm thường xuyên hơn ta tưởng.** *ReLU* không khả vi tại không (vô hại, tập điểm có độ đo không), nhưng các phép toán rời rạc thật thì không cứu được: bước lọc bỏ các hộp trùng nhau và chỉ giữ hộp điểm cao nhất (*non-maximum suppression*) dùng argmax, phép lấy mẫu cứng, chỉ số nguyên trong **lượng tử hoá** (*quantization*) đều làm gradient bằng không ở mọi nơi. Cả một mảng kỹ thuật ở C và D — *surrogate loss*, *straight-through estimator*, *Gumbel-softmax* — tồn tại chỉ để vá chỗ này, và mỗi cách vá đưa vào một thiên lệch riêng.

→ **hệ quả của C** — mọi cách vá chỗ không khả vi đều sinh ra từ đúng một sự thật: gradient bằng không ở mọi nơi trên một phép toán rời rạc

- **Autograd không cứu được sai số số học.** Gradient có thể đúng công thức nhưng vẫn tràn hoặc mất chính xác ở *half precision*. Triệu chứng là loss thành NaN sau vài trăm bước. Đó là

lý do tồn tại của *loss scaling* trong huấn luyện hỗn hợp độ chính xác [13]. Ở chiều ngược lại, gradient check với float32 hay báo sai giả; phải kiểm ở double.

- **Đừng suy từ bài toán lỗi sang mạng sâu.** Trực giác “quỹ đạo trơn tru đi xuống lòng chảo” mô tả sai cái đang xảy ra: quỹ đạo thật đi qua vùng có độ cong đổi dấu, và learning rate lớn ở giai đoạn đầu đóng vai trò **chính quy hoá** (*regularization*) chứ không chỉ là “đi nhanh” — giảm nó quá sớm làm mất tác dụng đó.

#### CẠM BÃY

Loss giảm không có nghĩa mô hình đang học thứ bạn muốn. Ba trường hợp hay gặp và cách phân biệt: loss giảm vì mô hình học thiên lệch của nhãn (kiểm bằng cách so với *baseline* đoán lớp phổ biến nhất); loss train giảm mà validation đi ngang vì **rò rỉ dữ liệu** (*data leakage*) làm hai tập không độc lập (kiểm bằng cách chia theo nhóm nguồn); loss giảm đều nhưng chỉ số nghiệp vụ đứng yên vì hàm mất mát không đại diện tốt cho chỉ số đó (kiểm bằng cách vẽ tương quan giữa hai đại lượng). Luôn xem loss như một *proxy* có sai lệch; B/09 mở xẻ sai lệch ấy.

#### CẦN NHỚ

Khớp shape trước, dấu và hệ số sau. Trước khi đổ lỗi cho tính không lười, hãy kiểm tra số điều kiện. Và đừng bao giờ dừng huấn luyện theo chuẩn gradient — đừng theo chỉ số trên tập validation, vì đó mới là đại lượng bạn thật sự quan tâm. [T1]

### G. Kết nối

Mục này nhận trực tiếp từ [A/01/01-dai-so-tuyen-tinh](#) (Hessian và phổ của nó) và chuyển tiếp sang [A/01/03-xac-suat-va-thong-tin](#), nơi câu hỏi “validation ngừng cải thiện chưa” được trả lời bằng công cụ thống kê thay vì bằng mắt. Theo chiều ngang, đây là tiền đề của hai chỗ. Thứ nhất là toàn bộ chương optimizer và lịch trình learning rate ở D. Thứ hai là phần thiết kế hàm mất mát ở B/09, nơi ta thấy nhiều lựa chọn loss thật ra là lựa chọn về *hình dạng gradient* chứ không về ý nghĩa thống kê.

#### ❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chế độ ngược tồn bộ nhớ còn chế độ xuôi thì không, và điều đó dẫn tới kỹ thuật gì khi mô hình quá lớn?
2. Bạn derive được  $\partial L / \partial W$  mà không cần nhớ công thức. Lập luận đó dựa trên tính chất nào, và khi nào nó không đủ để xác định duy nhất đáp án?
3. Trong không gian nhiều triệu chiều, vì sao điểm dừng gần như chắc chắn là yên ngựa chứ không phải cực tiểu địa phương?
4. Vì sao “chuẩn gradient nhỏ” là điều kiện dừng vô nghĩa khi huấn luyện với mini-batch?
5. Tăng batch size gấp bốn thì nhiều gradient giảm bao nhiêu, và vì sao quan hệ đó khiến batch rất lớn kém hiệu quả về chi phí?
6. Loss giảm ổn định suốt 50 epoch nhưng mô hình vô dụng khi triển khai — ba nguyên nhân nào giải thích được chuyện đó, và mỗi nguyên nhân phân biệt bằng phép kiểm nào?

# Nguồn

---

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [29] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [30] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [31] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [32] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [33] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [34] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [37] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [38] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [39] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.

### 1.3 Xác suất, độ hợp lý và lý thuyết thông tin (Probability, Likelihood and Information Theory)

**PHỤ THUỘC** A/01-toan-toi-thieu/02-vi-phan-va-toi-uu (điều kiện dừng theo validation — mục này cung cấp công cụ để trả lời “validation đã thật sự cải thiện chưa”).

#### TÓM TẮT

Mục này làm hai việc: cho thấy gần như mọi **hàm mất mát** (*loss function*) là âm log **độ hợp lý** (*likelihood*) của một giả định phân phối được chọn ngầm, và cho bạn công cụ để biết một con số đo được đáng tin đến đâu. Đọc xong bạn không cần nhớ công thức loss nào nữa, vì suy ra được từ giả định. Bạn biết ngay loss đó hỏng ở đâu: đúng chỗ giả định vỡ. Và bạn tính được trong ba mươi giây xem chênh lệch một điểm phần trăm giữa hai mô hình có đáng bàn không. Nếu chỉ nhớ một điều: *mọi loss là một giả định phân phối trả hình, và mọi con số đo được là một ước lượng có sai số* — ai bỏ qua về thứ hai sẽ dành hàng tháng để đuổi theo nhiều.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **biến ngẫu nhiên, kỳ vọng, phương sai** — biến ngẫu nhiên là một đại lượng mà giá trị phụ thuộc kết quả của một phép thử; kỳ vọng là giá trị trung bình dài hạn của nó; phương sai đo mức dao động quanh kỳ vọng, và căn bậc hai của phương sai là độ lệch chuẩn.
- **phân phối (distribution)** — quy luật gán xác suất cho các giá trị mà biến ngẫu nhiên có thể nhận. Chọn một phân phối là đưa ra một giả định về cách dữ liệu được sinh ra.
- **độ hợp lý (likelihood)** — với một bộ tham số cho trước, xác suất mà mô hình gán cho *dữ liệu đã quan sát được*. Lưu ý nó là hàm của tham số, không phải hàm của dữ liệu.
- **prior** — niềm tin về tham số *trước khi* nhìn dữ liệu, viết dưới dạng một phân phối. Trong thực hành, một prior tương đương với một số hạng phạt cộng vào hàm mất mát.
- **hàm mất mát (loss function)** — một con số đo mức sai của dự đoán so với nhãn; huấn luyện là đi tìm tham số làm nó nhỏ nhất.
- **softmax** — phép biến một vector số thực bất kỳ thành một bộ số dương cộng lại bằng một, để đọc như xác suất của từng lớp. “Đọc như” không có nghĩa “là”.
- **tập test và tập validation** — validation là phần dữ liệu dùng để chọn siêu tham số và quyết định khi nào dừng; test là phần chỉ được chạm vào *một lần* ở cuối để báo cáo. Dùng test như validation làm mọi con số trở nên lạc quan giả.
- **độ chính xác (accuracy)** — tỉ lệ mẫu được dự đoán đúng. Nó là trung bình của các biến chỉ báo đúng hay sai, nên bản thân nó cũng là một ước lượng có sai số.

### A. Đặt vấn đề

Hai câu nói kết thúc rất nhiều tranh luận kỹ thuật một cách sai: “mô hình A đạt 90,2%, mô hình B đạt 89,1%, vậy A tốt hơn” và “loss của tôi là **entropy chéo** (*cross-entropy*) vì mọi người dùng cross-entropy”. Câu thứ nhất là một phát biểu thống kê không kèm sai số. Câu thứ hai là một lựa chọn mô hình xác suất được nhớ như một quy ước. Mục này chứa cả hai bằng cùng một bộ công cụ.

**Học xong mục này thì làm được gì mà trước đó không làm được?** Bốn việc: suy ra hàm mất mát từ giả định về nhiễu thay vì tra bảng; dự đoán trước loss sẽ hỏng ở đâu; tính khoảng tin cậy và quyết định một thí nghiệm có đủ mẫu không *trước khi* chạy; và đọc đúng hướng của **phân kỳ KL** (*Kullback–Leibler divergence*) trong một paper để biết mô hình sẽ trái ngược hay chọn một mode. Nền lý thuyết ở [14], [15]; phần thông tin ở [16].

### TRỰC GIÁC

Cross-entropy là hoá đơn tiền điện của việc mã hoá sự thật bằng bảng mã của mô hình. Mô hình tin chắc vào điều đúng thì sự kiện thật rơi vào một ký hiệu ngắn và hoá đơn nhỏ; mô hình tin chắc vào điều sai thì sự kiện thật rơi vào một ký hiệu dài kinh khủng, đúng bằng  $-\log q$  của một xác suất gần không. **Entropy** là hoá đơn tối thiểu không tránh được; KL là phần phụ trội phải trả vì dùng sai bảng mã. Cách nhìn này giải thích ngay vì sao mô hình bị phạt nặng bất đối xứng khi *tự tin mà sai*, chứ không phải khi chỉ đơn giản là sai.

### B. Sai số lấy mẫu: một tập test 500 mẫu đáng tin đến đâu

Độ chính xác đo trên tập test là trung bình của các biến Bernoulli, nên sai số chuẩn là  $\sqrt{p(1-p)/n}$ . Nói bằng lời: *đại lượng này đo mức dao động của con số nếu bạn rút một mẫu 500 phần tử khác từ cùng phân phối* — không hơn.

**Tính tay.** Với  $n = 500$ ,  $p = 0,90$ : sai số chuẩn  $= \sqrt{0,9 \cdot 0,1/500} = \sqrt{0,00018} \approx 0,0134$ , tức 1,34 điểm phần trăm. Khoảng tin cậy 95% xấp xỉ  $90\% \pm 2,6\%$ , tức từ 87,4% tới 92,6% — rộng hơn năm điểm.

Ở đây có một tình huống quan trọng, và bỏ qua nó thì sai theo cả hai chiều. Khi hai mô hình được đánh giá trên *cùng* một tập test, chúng sai ở phần lớn cùng những mẫu khó, nên phép so sánh đúng là so sánh *ghép cặp*: cái cần đếm là số mẫu hai mô hình bất đồng, và độ bất định của hiệu số nhỏ hơn nhiều so với độ bất định của từng con số riêng lẻ. Nếu trong 500 mẫu chỉ có 20 mẫu bất đồng và 15 nghiêng về A thì kết luận còn yếu; nếu 60 mẫu bất đồng và 45 nghiêng về A thì hiệu số một điểm có thể có ý nghĩa. Công cụ đúng là kiểm định ghép cặp (McNemar) hoặc *bootstrap* trên chính tập test, không phải hai khoảng tin cậy vẽ cạnh nhau [17].

Còn hai nguồn bất định nữa mà công thức trên *không* bắt được, và chúng thường lớn hơn:

- **Bất định do chọn tập test.** Công thức chỉ trả lời “nếu rút 500 mẫu khác từ cùng phân phối”. Dòng công trình dựng lại tập test của ImageNet theo đúng quy trình gốc cho thấy độ chính xác tụt hàng điểm [18] — tức bản thân quy trình thu thập cũng là một nguồn ngẫu nhiên.
- **Nhân sai.** Các tập chuẩn phổ biến có tỉ lệ nhân sai đủ để đảo thứ hạng ở phần đỉnh bảng [19]. Nó đặt một trần cứng lên mọi phép so sánh, và không phương pháp thống kê nào vượt qua được trần đó.

### C. Likelihood, MLE, MAP: nguồn gốc chung của mọi hàm mất mát

Giả sử ta mô hình hoá dữ liệu bằng một phân phối có tham số  $\theta$ . **Ước lượng độ hợp lý cực đại** (*maximum likelihood estimation, MLE*) chọn  $\theta$  cực đại  $\prod_i p(y_i | x_i; \theta)$ , tương đương cực tiểu  $-\sum_i \log p(y_i | x_i; \theta)$ . Nói bằng lời: *huấn luyện chính là tìm bộ tham số làm cho dữ liệu quan sát được trở nên ít bất ngờ nhất*. Chọn dạng phân phối nào thì ra loss này — và đó là toàn bộ nội dung Bảng 1.3.

**Bảng 1.3.** Hàm mất mát nào cũng là một giả định phân phối. Cột cuối là cột đáng giá nhất: giả định vỡ ở đâu thì loss hỏng ở đó.

| Loss          | Giả định phân phối                                                                | Cho ra ước lượng gì     | Hỏng khi nào                                                                             |
|---------------|-----------------------------------------------------------------------------------|-------------------------|------------------------------------------------------------------------------------------|
| MSE           | Gaussian, phương sai hằng số                                                      | Trung bình có điều kiện | Có ngoại lai; nhiễu phụ thuộc tín hiệu; mục tiêu đa phương thức thì cho ra trung bình mờ |
| L1            | Laplace                                                                           | Trung vị có điều kiện   | Cần ước lượng trung bình; gradient hằng số gây dao động gần nghiệm                       |
| Cross-entropy | Categorical                                                                       | Xác suất từng lớp       | Nhân sai hoặc mất cân bằng cực đoan; phạt rất nặng khi tự tin mà sai                     |
| Phạt $L_2$    | <b>Cực đại hậu nghiệm</b> ( <i>maximum a posteriori, MAP</i> ) với prior Gaussian | Nghiệm co về 0          | Khi thang đo các tham số khác nhau nhiều                                                 |

Bảng trên giải thích một chuyện hay gây bối rối: vì sao hồi quy độ sâu bằng MSE cho ra biên vật thể mờ. Ở biên, độ sâu thật là *đa phương thức*: hoặc thuộc vật trước, hoặc thuộc nền. MLE dưới giả định Gaussian đơn phương thức buộc mô hình trả về trung bình của hai đáp án đúng. Kết quả là một giá trị sai nằm ở giữa. Không siêu tham số nào cứu được; phải đổi giả định phân phối, và đó chính là động cơ của các mô hình sinh có điều kiện ở C.

→ **tiền đề** B/09 — mọi lựa chọn hàm mất mát ở chương thiết kế loss bắt đầu bằng việc chọn — có ý thức hay không — một giả định phân phối

MAP thêm một prior: cực đại  $p(\theta \mid \text{dữ liệu}) \propto p(\text{dữ liệu} \mid \theta) p(\theta)$ , tương đương thêm một số hạng phạt vào loss. Với nền SLAM đây là chỗ nổi quen thuộc: một prior trên pose *chính* là một số hạng phạt trong hàm chi phí, và damping của Levenberg–Marquardt hoạt động y hệt một prior Gaussian quanh điểm hiện tại — cùng một cơ chế, hai cái tên.

Cần trung thực một chỗ: *không phải mọi loss đều là âm log độ hợp lý*. **Focal loss** [20] là phép định lại trọng số theo độ khó; **làm mượt nhãn** (label smoothing) [21] cố ý làm sai lệch mục tiêu; các loss dạng biên (triplet, contrastive) tối ưu một thứ tự chứ không một mật độ. Đó là kỹ thuật, không phải hệ quả của một mô hình sinh — và chính vì thế xác suất mà mô hình xuất ra sau khi huấn luyện bằng chúng không còn được **hiệu chuẩn** (calibrated).

#### D. Entropy, cross-entropy, KL và sự bất đối xứng

Ba đại lượng, một quan hệ:

$$H(p) = - \sum_i p_i \log p_i, \quad H(p, q) = - \sum_i p_i \log q_i, \quad D_{\text{KL}}(p \parallel q) = H(p, q) - H(p).$$

Vì  $H(p)$  không phụ thuộc tham số mô hình, *cực tiểu cross-entropy chính là cực tiểu KL từ dữ liệu tới mô hình, và cũng chính là MLE* — ba cách nói của một việc.

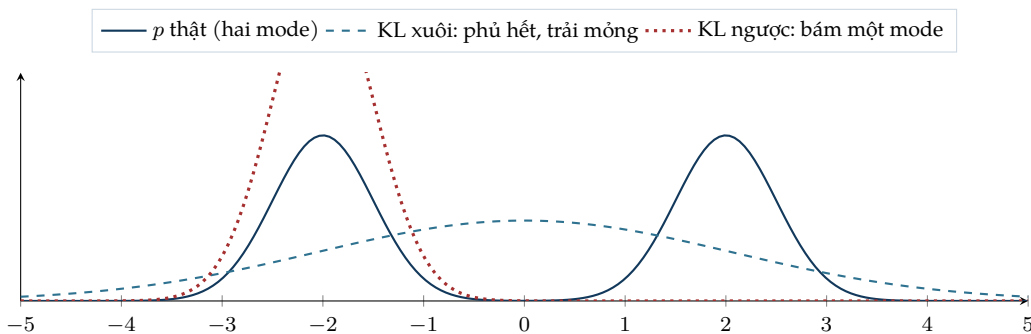
**Một phép kiểm tra rẻ tiền rơi ra từ đây.** Với bài toán 1000 lớp, mô hình khởi tạo hợp lý phải cho loss ban đầu xấp xỉ  $\ln 1000 \approx 6,91$ . Nếu là 20 thì khởi tạo hoặc tăng cuối sai; nếu là 2 thì có rò rỉ nhãn. Một dòng kiểm tra, bắt được cả một lớp bug.

KL không đối xứng, và đây không phải khiếm khuyết toán học mà là một lựa chọn về hành vi.

**Ví dụ tính tay** với hai phân phối Bernoulli  $p = (0,5; 0,5)$  và  $q = (0,9; 0,1)$ :

$$D_{\text{KL}}(p \parallel q) = 0,5 \ln \frac{0,5}{0,9} + 0,5 \ln \frac{0,5}{0,1} \approx 0,511, \quad D_{\text{KL}}(q \parallel p) = 0,9 \ln \frac{0,9}{0,5} + 0,1 \ln \frac{0,1}{0,5} \approx 0,368.$$

Hai con số khác nhau vì hai đại lượng phạt hai kiểu sai lầm khác nhau. Hình 1.5 cho thấy hệ quả rõ hơn mọi lời giải thích: cùng một phân phối thật hai đỉnh, hai hướng KL cho ra hai nghiệm hoàn toàn khác.



**Hình 1.5.** Hai hướng của KL cho hai hành vi trái ngược khi mô hình quá đơn giản để khớp phân phối thật. KL xuôi  $D_{\text{KL}}(p \parallel q)$  phạt nặng chỗ  $p$  có khối lượng mà  $q$  gần bằng không, nên nghiệm bị ép phủ cả hai mode và đặt khối lượng vào vùng trống ở giữa. KL ngược  $D_{\text{KL}}(q \parallel p)$  phạt chỗ  $q$  đặt khối lượng mà  $p$  không có, nên nghiệm chọn một mode và bỏ mode kia.

Sự khác biệt ấy giải thích ba hiện tượng sẽ gặp lại:

↔ **cùng nguyên lý với C** — chiều của KL được tối ưu quyết định mô hình sinh trái mỏng qua mọi mode hay bám lấy một mode — cùng cơ chế ở VAE, chưng cất và huấn luyện theo độ hợp lý

- **Mô hình sinh huấn luyện theo độ hợp lý sinh ra mẫu mờ.** MLE dùng KL xuôi, nên mô hình buộc phải phủ cả những vùng trống giữa các mode. Mẫu rút từ vùng ấy không giống bất kỳ mẫu thật nào.

- **Posterior collapse trong VAE** [22] — số hạng KL ngược tới prior khiến mô hình thả bỏ hẳn vài chiều tiềm ẩn còn hơn trải mỏng.
- **“Dark knowledge” trong chưng cất tri thức (knowledge distillation)** [23] — học trò khớp phân phối thầy theo chiều xuôi, nên bị buộc phải phủ cả các đuôi xác suất của những lớp *không phải* đáp án; chính các đuôi đó mang thông tin về quan hệ giữa các lớp.

## E. Giới hạn và trường hợp hỏng

- **Giả định độc lập và cùng phân phối gần như luôn sai trong thị giác.** Các khung hình liên tiếp của một video giống nhau; nhiều ảnh của cùng một bệnh nhân cũng vậy; nhiều mẫu chụp cùng buổi thì cùng ánh sáng. Một tập test 500 ảnh trích từ 20 video có kích thước mẫu *hiệu dụng* gần 20 hơn gần 500. Khoảng tin cậy thật vì thế rộng gấp nhiều lần. Chia tập theo nhóm (theo video, theo người, theo thiết bị) quan trọng hơn mọi tinh chỉnh mô hình.
- **Xác suất mà mạng xuất ra không phải xác suất thật.** Mạng hiện đại có xu hướng quá tự tin: trong nhóm dự đoán độ tin cậy 0,99, tỉ lệ đúng thực tế thấp hơn đáng kể, và điều này được đo lặp lại nhiều lần [24].

↔ **hệ quả của** E — mạng quá tự tin không phải lỗi cài đặt mà là hệ quả của việc tối ưu cross-entropy chứ không tối ưu độ hiệu chuẩn

Đừng dùng *softmax* làm ngưỡng ra quyết định khi chưa hiệu chuẩn; E bàn *temperature scaling*, *ensemble* và *conformal prediction* [25] như các cách vá với chi phí khác nhau.

- **Khung độ hợp lý có giới hạn khái niệm.** Nó tối ưu độ khớp phân phối, còn thứ ta quan tâm thường là một chỉ số nghiệp vụ bất đối xứng — bỏ sót một người đi bộ tệ hơn nhiều so với một báo động giả. Không prior nào biến MLE thành thứ tối ưu chỉ số đó; phải đặt vào hàm chi phí một cách tường minh, và đó là nội dung của B/09.

### CẠM BÃY

“Cải thiện 0,3% trên tập test” gần như luôn là nhiều — nhưng cách sửa phổ biến, chạy nhiều seed rồi báo trung bình, *vẫn* chưa đủ nếu bạn đã dùng chính tập test đó để chọn siêu tham số. Khi tập test bị nhìn nhiều lần, nó biến thành tập validation và mọi khoảng tin cậy tính trên nó đều quá lạc quan. Đây là dạng rò rỉ tinh vi nhất, vì không có dòng code nào sai cả.

### CẦN NHỚ

Trước khi hỏi “mô hình nào tốt hơn”, hãy tính  $\sqrt{p(1-p)/n}$  — ba mươi giây, và nó loại bỏ phần lớn tranh luận vô ích. Trước khi chọn loss, hãy hỏi “tôi đang giả định nhiều phân phối gì” — vì loss sẽ hỏng đúng chỗ giả định đó vỡ. [T1]

## F. Kết nối

Mục này khép lại phần “ngôn ngữ chung” của chương, và là tiền đề của ba nhánh xa nhau. B/09 về thiết kế hàm mất mát, nơi ta cố ý lệch khỏi độ hợp lý. C về mô hình sinh, nơi hướng của KL quyết định hành vi mẫu sinh ra. Và toàn bộ E về đánh giá và độ tin cậy. Theo chiều đối lập, nó bổ khuyết cho A/01/02: mục kia nói gradient descent đi tới đâu, mục này nói làm sao biết chỗ đó có thật sự tốt hơn hay không.

### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao KL không đối xứng, và điều đó thay đổi hành vi cụ thể nào của một mô hình sinh?

2. Với 500 mẫu test và độ chính xác 90%, khoảng tin cậy 95% rộng bao nhiêu? Vì sao con số đó *không* trực tiếp bác bỏ được chênh lệch một điểm giữa hai mô hình đánh giá trên cùng tập test?
3. Tập test gồm 500 khung hình trích từ 20 video. Kích thước mẫu hiệu dụng gần con số nào hơn, và hệ quả với cách chia dữ liệu là gì?
4. Vì sao hồi quy độ sâu bằng MSE cho ra biên mờ, và vì sao đổi optimizer không sửa được?
5. Loss ban đầu của bài 1000 lớp phải xấp xỉ bao nhiêu, và nếu nó bằng 2 thì bạn nghi ngờ điều gì?
6. Focal loss và label smoothing không phải âm log độ hợp lý của mô hình nào. Điều đó gây hệ quả gì cho việc đọc đầu ra softmax?

# Nguồn

---

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [29] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [30] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [31] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [32] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [33] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [34] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [37] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [38] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [39] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.

## 1.4 Hình học biến đổi và xử lý tín hiệu (Transformation Geometry and Signal Processing)

**PHỤ THUỘC** [A/01-toan-toi-thieu/01-dai-so-tuyen-tinh](#) (SVD — dùng để chiếu một ma trận nhiều về  $SO(3)$ ). Mục này là tiền đề bắt buộc của cả chương [A/02-vat-ly-cao-anh](#).

### TÓM TẮT

Mục này gộp hai chủ đề vì chúng chia nhau một vai trò: cả hai mô tả cấu trúc mà dữ liệu ảnh *đã có sẵn* trước khi mô hình nhìn thấy nó. Đọc xong bạn biết vì sao một mạng dự đoán phép quay bằng bốn số khó huấn luyện hơn bằng sáu số. Bạn biết vì sao CNN không thật sự bất biến tịnh tiến. Và bạn biết vì sao một dòng `resize` sai trong dataloader làm mất vài điểm độ chính xác, mà không sinh ra exception nào. Nếu chỉ nhớ một điều, hãy nhớ hai câu hỏi phòng vệ. *Trước mọi lần giảm độ phân giải: đã lọc thông thấp chưa? Trước mọi lần cho mạng xuất ra phép quay: biểu diễn này có liên tục không?* Hai câu đó bắt được phần lớn lỗi im lặng ở khâu hình học và tín hiệu.

### Kiến thức cần có trước

- **tư thế (pose) và  $SE(3)$**  — tư thế là vị trí cộng hướng của một vật cứng trong không gian; tập mọi tư thế tạo thành nhóm  $SE(3)$ , còn tập mọi phép quay thuần túy là  $SO(3)$ , biểu diễn bằng ma trận  $3 \times 3$  trực giao có định thức bằng một.
- **đa tạp (manifold)** — một tập hợp mà quanh mỗi điểm nó “trông giống” một không gian phẳng nhiều chiều, nhưng toàn cục thì cong. Hệ quả thực hành: không cộng trực tiếp hai phân tử được, phải cộng ở không gian tiếp tuyến.
- **convolution và kernel** — kernel là một cửa sổ nhỏ chứa các trọng số; convolution là trượt cửa sổ đó khắp ảnh và tại mỗi vị trí lấy tổng có trọng số của vùng lân cận. Đây là phép toán nền của mạng CNN.
- **CNN** — mạng nơ-ron mà phần lớn các tầng là convolution; nó được thiết kế dựa trên giả định rằng cấu trúc ảnh mang tính cục bộ và không đổi khi dịch chuyển.
- **stride và pooling** — stride là bước nhảy khi trượt kernel — stride 2 nghĩa là chỉ lấy một trên hai vị trí, tức giảm độ phân giải một nửa. Pooling cũng giảm độ phân giải bằng cách lấy giá trị lớn nhất hoặc trung bình của một vùng.
- **tần số không gian** — đo bằng “chu kỳ trên pixel”: một hoa văn 0,25 chu kỳ/pixel lặp lại sau mỗi bốn pixel. Tần số cao ứng với chi tiết mảnh, tần số thấp ứng với cấu trúc lớn.
- **biến đổi Fourier** — cách viết lại một tín hiệu thành tổng các sóng sin ở nhiều tần số khác nhau, để nhìn xem tín hiệu chứa những tần số nào và mạnh yếu ra sao.
- **nội suy (interpolation)** — khi thay đổi kích thước hoặc xoay ảnh, giá trị pixel mới phải được ước lượng từ các pixel cũ lân cận; cách ước lượng đó gọi là nội suy, và mỗi lần nội suy là một lần làm mờ thêm.

### A. Đặt vấn đề

Bỏ qua cấu trúc có sẵn của dữ liệu ảnh thì mạng phải học lại nó từ đầu, tốn nhãn và tốn tham số; hiểu sai cấu trúc ấy thì bạn tự tay phá dữ liệu ở khâu tiền xử lý và không bao giờ tìm ra nguyên nhân. Với người đã làm SLAM, nửa đầu mục này là ôn tập nhanh có đối trọng tâm: câu hỏi không còn là “ $SE(3)$  hoạt động thế nào” mà là “vì sao mạng khó xuất ra một phép quay”. Nửa sau thì hầu như luôn là chỗ hỏng thật, kể cả với người có kinh nghiệm.

**Học xong mục này thì làm được gì mà trước đó không làm được?** Chọn đúng biểu diễn đầu ra cho mọi bài toán ước lượng tư thế bằng mạng; kiểm tra một pipeline tiền xử lý và chỉ ra chỗ đang sinh hoa văn giả; và giải thích bằng ngôn ngữ tần số vì sao một mô hình huấn luyện ở  $224 \times 224$  sụt điểm khi đưa vào ảnh độ phân giải khác.

### TRỰC GIÁC

**Aliasing** không phải chuyện làm ảnh xấu đi; nó là chuyện làm ảnh *nói dối*. Nạn hoa bánh xe quay ngược trong phim là ví dụ thuần khiết: máy quay không làm mờ bánh xe, nó tạo ra một chuyển động không hề tồn tại. Khi bạn resize ảnh bằng *nearest-neighbor* rồi đưa vào mạng, mạng nhận được những hoa văn cũng không hề có trên vật thật. Nó học các hoa văn ấy một cách hoàn toàn nghiêm túc. Rồi camera triển khai có độ phân giải khác, hoa văn giả đổi kiểu, và mô hình sập — mà không ai chạm vào trọng số nào.

### B. Hình học: ba thao tác là đủ

**Toạ độ đồng nhất (homogeneous coordinates)** tồn tại vì một lý do thực dụng: nó biến phép tịnh tiến, vốn là phép cộng, thành phép nhân ma trận. Nhờ đó cả chuỗi biến đổi gộp được thành một tích duy nhất. Phép chiếu phối cảnh cũng viết được ở dạng tuyến tính, rồi chia cho thành phần cuối. Toàn bộ hình học nhiều view ở [A/02/03](#) xây trên tiện lợi đó.

Nhóm quay  $SO(3)$  và nhóm chuyển động cứng  $SE(3)$  là các **đa tạp** (*manifold*) chứ không phải không gian vector: cộng hai ma trận quay không cho ra ma trận quay. Hệ quả thực hành — và là toàn bộ phần **đại số Lie** (*Lie algebra*) bạn cần — gói trong ba thao tác:

1.  $\exp(\delta)$  đưa một vector ba chiều (trục nhân góc) thành một ma trận quay;
2.  $\log(R)$  làm ngược lại, cho ra “sai lệch quay” ở dạng vector cộng được;
3. cập nhật  $R \leftarrow R \exp(\delta)$  thay cho  $R \leftarrow R + \delta$ , tức là *cộng ở không gian tiếp tuyến rồi mới quay về đa tạp*.

Nếu bạn đã viết một solver **hiệu chỉnh chùm tia** (*bundle adjustment*) thì bạn đã cài đúng ba thao tác này; điều đáng nói là chúng không biến mất khi chuyển sang mạng nơ-ron, chỉ đổi chỗ. Nền lý thuyết ở mức kỹ sư: [26], [27].

Chỗ chúng xuất hiện lại là bài toán cho mạng *xuất ra* một phép quay, và đây là một trong ít chỗ mà lý thuyết cho lời khuyên rất rõ. Không có phép tham số hoá liên tục nào của  $SO(3)$  bằng ba hoặc bốn số mà không có điểm kỳ dị hoặc điểm gián đoạn; mà một mạng liên tục theo đầu vào thì không thể khớp một hàm mục tiêu gián đoạn, nên nó cứ xử tệ đúng quanh vùng gián đoạn. Bảng 1.4 so bốn lựa chọn.

**Bảng 1.4.** Bốn cách cho mạng xuất ra một phép quay. Cột cuối quyết định lựa chọn.

| Biểu diễn       | Số đầu ra | Cơ chế đưa về $SO(3)$      | Hỏng khi nào                                                                             |
|-----------------|-----------|----------------------------|------------------------------------------------------------------------------------------|
| Góc Euler       | 3         | Nhân ba phép quay trục     | <i>Gimbal lock</i> ; gián đoạn ở biên $\pm\pi$ ; trung bình vô nghĩa                     |
| Quaternion      | 4         | Chuẩn hoá về chuẩn 1       | Phủ hai lần ( $q$ và $-q$ cùng một phép quay) nên hàm mục tiêu gián đoạn; phải xử lý dấu |
| 6D (hai vector) | 6         | Gram–Schmidt trực giao hoá | Không có điểm gián đoạn — lựa chọn mặc định hiện nay; chỉ tốn thêm vài phép tính         |
| Ma trận 9 số    | 9         | Chiếu về $SO(3)$ bằng SVD  | Cũng liên tục, nhưng SVD trong vòng huấn luyện đắt hơn và đạo hàm phức tạp hơn           |

↔ **trường hợp riêng của C** — mọi bài ước lượng tư thế bằng mạng là một trường hợp riêng của việc chọn biểu diễn liên tục cho đầu ra trên đa tạp

Cách sáu số cho mạng xuất ra hai vector ba chiều rồi trực giao hoá. Nó được phát biểu rõ trong dòng công trình về “biểu diễn quay liên tục” của Zhou và cộng sự. Cách chín số rồi chiếu bằng SVD tương đương về tinh thần, và dùng đúng kết quả xấp xỉ hạng thấp ở A/01/01.

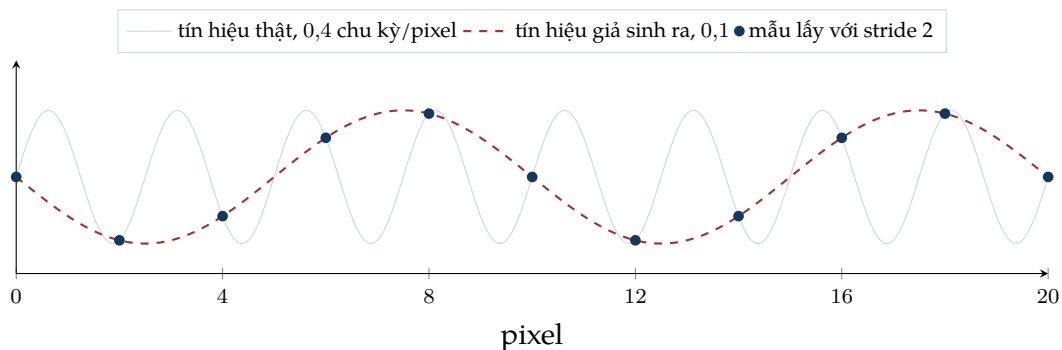
Một chi tiết dễ nhầm và tốn thời gian debug: *lấy trung bình các phép quay*. Trung bình cộng của các ma trận quay không phải ma trận quay; trung bình quaternion phụ thuộc dấu bạn chọn; trung bình góc Euler vô nghĩa qua chỗ chuyển  $\pm\pi$ . Cách đúng là trung bình trên không gian tiếp tuyến quanh một điểm neo rồi ánh xạ về, hoặc chiếu trung bình ma trận về  $SO(3)$  bằng SVD. Sai lầm này xuất hiện lại nguyên vẹn khi người ta gộp dự đoán của nhiều mô hình hay nhiều *test-time augmentation*.

### C. Tín hiệu: mọi lỗi im lặng đều là lỗi lấy mẫu

**Convolution** là phép toán duy nhất vừa tuyến tính vừa **đẳng biến tịnh tiến** (*translation equivariance*): dịch ảnh rồi lọc cũng bằng lọc rồi dịch. Đó là toàn bộ lý do CNN tồn tại. Ta *cài sẵn* giả định “một cạnh vẫn là một cạnh khi dịch sang phải mười pixel”, thay vì bắt mạng học nó từ dữ liệu. Định lý convolution nói convolution trong miền không gian là phép nhân từng điểm trong miền Fourier; nói bằng lời, *mỗi kernel là một bộ lọc tần số* — kernel làm mịn là thông thấp, kernel Sobel là thông cao. Từ đó cả kiến trúc đọc được gọn: một CNN sâu là chuỗi lọc xen kẽ các bước giảm mẫu, và mỗi bước giảm mẫu vớt bỏ một nửa dải tần.

Chính chỗ vớt bỏ ấy là nơi mọi thứ hỏng. **Định lý lấy mẫu** (*sampling theorem*) nói tần số lấy mẫu phải lớn hơn hai lần tần số cao nhất có trong tín hiệu; nếu không, thành phần tần số cao không biến mất mà *gập* xuống thành tần số thấp giả.

**Ví dụ tính tay.** Hoa văn 0,4 chu kỳ trên pixel, lấy mẫu cách hai pixel (stride 2): tần số lấy mẫu 0,5, tần số Nyquist 0,25; thành phần 0,4 vượt ngưỡng nên gập về  $|0,4 - 0,5| = 0,1$  chu kỳ trên pixel — một hoa văn thô gập bốn lần, không hề tồn tại trên vật thật. Hình 1.6 vẽ đúng phép tính đó.



**Hình 1.6.** Aliasing không làm mất thông tin một cách trung thực, nó *bịa* thông tin. Các điểm lấy mẫu cách hai pixel nằm chính xác trên một hình sin tần số thấp hơn bốn lần; mọi tầng phía sau đều không có cách nào biết hoa văn thô ấy là giả.

→ **tiền đề** A/02 — khử khảm và giảm mẫu màu 4:2:0 trong chuỗi ISP là đúng bài toán lấy mẫu này, chỉ đổi tên

Từ đó rút ra quy tắc bất di bất dịch của mọi phép giảm mẫu: **lọc thông thấp trước, rồi mới lấy mẫu thưa**. Hệ quả ở hai chỗ:

- **Trong dataloader** — thu nhỏ ảnh phải dùng nội suy vùng (INTER\_AREA) hoặc làm mờ Gaussian trước; *nearest-neighbor* gần như luôn sai khi thu nhỏ. Đây là một dòng code, và nó đáng vài điểm độ chính xác.
- **Trong kiến trúc** — tầng stride-2 và *max-pooling* lấy mẫu thưa mà không lọc trước, nên CNN *không* bất biến dịch chuyển như người ta hay nói: dịch ảnh một pixel có thể đổi hẳn dự đoán. Dòng công trình “làm cho CNN bất biến dịch trở lại” của Zhang chèn một bộ lọc mờ ngay trước mỗi bước giảm mẫu. Cách đó khôi phục phần lớn tính bất biến. Kết quả đã được đo và tái lập, dù mức lợi ích tùy bài toán.

Góc nhìn Fourier còn giải thích hai hiện tượng nữa. Thứ nhất là **spectral bias**. Mạng khớp thành phần tần số thấp trước, tần số cao sau, nên chi tiết mảnh xuất hiện muộn trong huấn luyện. Khi mục tiêu vốn nhiều tần số cao — chẳng hạn hàm mật độ của một cảnh 3D trong *NeRF* — người ta phải mã hoá đầu vào bằng đặc trưng Fourier để mạng học nổi [28].

→ **tiền đề** C — đẳng biến tịnh tiến là thiên lệch quy nạp mà cả dòng CNN dựa vào, và việc nó không đúng hoàn toàn là một trong các động cơ dẫn tới *attention*

Thứ hai là chuyện thang đo: một mô hình huấn luyện ở  $224 \times 224$  thấy các cạnh ở một dải tần nhất định; đưa vào ảnh 4K rồi crop, cùng vật thể xuất hiện ở dải tần khác hẳn, và mô hình sục điểm dù “vẫn là ảnh của cùng cái xe”.

### I Mạch 3 — tiền xử lý ảnh, và cái giá của mỗi bước

- 1 Vấn đề** ảnh vào có kích thước bất kỳ, mạng cần kích thước cố định.

**Giải pháp** resize.

**Giả định** nội suy giữ nguyên nội dung ngữ nghĩa.

**Giá phải trả** đổi phổ tần số của ảnh; thu nhỏ mà không lọc trước thì sinh hoa văn giả.
- 2 Vấn đề** resize làm méo tỉ lệ khung hình.

**Giải pháp** *letterbox* — giữ tỉ lệ, đệm phần thiếu.

**Giá phải trả** đưa vào vùng đệm nhân tạo mà mạng có thể học như tín hiệu, và mọi tọa độ hộp phải được biến đổi ngược đúng cách — một chỗ sinh bug rất phổ biến.
- 3 Vấn đề** muốn dữ liệu phong phú hơn nên xoay, lật, cắt.

**Giả định** phép biến đổi không đổi nhãn.

**Vấn đề mới** giả định đó sai cho biển báo có chữ, cho ảnh y tế có tính trái-phải, và cho mọi bài toán phải dự đoán hướng; ngoài ra mỗi lần xoay là một lần nội suy, tức lại đổi phổ.
- 4 Vấn đề** muốn augmentation mạnh hơn nên chồng nhiều phép liên tiếp.

**Giả định** các phép nội suy chồng nhau không tích lũy hại.

**Giá phải trả** sai — mỗi lần nội suy làm mờ thêm.

**Giải pháp** gộp các phép biến đổi thành một ma trận rồi lấy mẫu *một lần* duy nhất, đúng như cách bạn gộp chuỗi  $SE(3)$  thành một phép biến đổi trước khi warp.

### D. Giới hạn và trường hợp hỏng

- **Bức tranh Fourier giả định tuyến tính và bất biến dịch**, mà mạng thật có phi tuyến sau mỗi tầng. “Mỗi tầng là một bộ lọc tần số” là trực giác hướng dẫn tốt, không phải mô tả chính xác — đừng suy diễn định lượng từ nó.
- **Hình học ở trên giả định phép biến đổi cứng và camera lý tưởng**. *Rolling shutter* phá giả định “một ảnh ứng với một pose” vì mỗi hàng pixel chụp ở một thời điểm khác; ống kính có méo làm phép biến đổi giữa hai ảnh không còn là homography ngay cả với mặt phẳng; vật thể không cứng thì  $SE(3)$  không mô tả nổi. Mỗi giả định vỡ ở một chỗ và đòi một cách vá khác nhau — chủ đề của [A/02/03](#).

#### CẦN NHỚ

Hai câu hỏi phòng vệ, hỏi mỗi lần: *trước khi giảm độ phân giải — đã lọc thông thấp chưa?* và *trước khi cho mạng xuất ra phép quay — biểu diễn này có liên tục không?* Chúng rẻ như một dòng code và bất được phần lớn lỗi im lặng ở khâu hình học và tín hiệu. [T1]

### E. Kết nối

Mục này là nền trực tiếp của cả chương [A/02-vat-ly-tao-anh](#): phần lấy mẫu giải thích vì sao **khử khảm** (*demosaiicing*) là một bài toán khôi phục chứ không phải nội suy tầm thường, và phần hình học là toàn bộ ngôn ngữ của [A/02/03](#). Theo chiều ngang, tính bất biến dịch chuyển là giả

định thiết kế của dòng CNN ở C, và việc nó *không* đúng hoàn toàn là một trong các động cơ dẫn tới *attention*; còn phần biểu diễn quay quay lại ở mọi bài toán ước lượng tư thế bằng mạng.

#### ■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao không tồn tại cách cho mạng xuất ra phép quay bằng bốn số mà không có điểm gián đoạn, và cách sáu số sửa được gì?
2. Hoa văn 0,45 chu kỳ/pixel bị lấy mẫu với stride 2 sẽ hiện ra ở tần số nào? Tính tay.
3. Vì sao CNN không thật sự bất biến dịch chuyển, dù convolution thì có? Chỗ phá vỡ nằm ở tầng nào?
4. Bạn chồng năm phép augmentation hình học liên tiếp, mỗi phép một lần nội suy. Hại gì xảy ra và sửa thế nào?
5. Vì sao lấy trung bình cộng các ma trận quay là sai, và cách đúng dựa trên kết quả nào ở mục đại số tuyến tính?
6. Một mô hình huấn luyện ở  $224 \times 224$  sụl điểm khi chạy trên ảnh 4K đã crop về  $224 \times 224$  — giải thích bằng ngôn ngữ tần số thì chuyện gì đã xảy ra?

# Nguồn

---

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [29] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [30] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [31] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [32] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [33] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [34] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [37] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [38] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [39] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.

## 1.5 Thực hành

Phần lý thuyết ở trên chỉ trở thành kiến thức dùng được sau khi bạn tự tay chạm vào một lần. Khối dưới đây được thiết kế để tốn ít thời gian nhất mà chạm được vào nhiều điểm nhất; hãy làm mini project ngay cả khi thấy đã hiểu, vì câu hỏi kết thúc nó là câu hỏi mà lý thuyết một mình không trả lời được.

**Repo và tài nguyên.** Cho phần lý thuyết, *Mathematics for Machine Learning* [2] có PDF miễn phí và đúng liều lượng cho mục tiêu ở đây. Khi cần đổi chiều một đạo hàm ma trận, [matrixcalculus.org](https://matrixcalculus.org) trả lời nhanh hơn tra sách; cần chắc chắn hơn nữa thì [sympy](https://sympy.org/) cho phép derive tay có kiểm chứng ký hiệu. Đạo hàm tự viết phải kiểm bằng `torch.autograd.gradcheck` trên đầu vào kiểu double — kiểm bằng float32 hay báo sai giả. Ghi chú lan truyền ngược của CS231n vẫn là bản giải thích đồ thị tính toán tốt nhất ở mức nhập môn, dù danh mục kiến trúc trong cùng khoá học thì nên đọc như lịch sử tư duy. Nhóm tài nguyên này ít bị bán rã — sách toán và công cụ ký hiệu bền hơn nhiều so với tên mô hình dẫn đầu bảng, vốn sẽ cũ trong 6–12 tháng kể từ mốc chốt tháng 9/2026.

**Câu hỏi kiểm tra hiểu.** Ba câu, mỗi câu chạm một mục khác nhau:

- Vì sao KL divergence không đối xứng, và sự bất đối xứng đó khiến một mô hình sinh trái mông hay chọn một mode?
- Số điều kiện lớn của một ma trận trọng số nói gì về độ ổn định huấn luyện — và nó *không* nói gì?
- Với tập test 500 mẫu và độ chính xác 90%, khoảng tin cậy 95% rộng bao nhiêu, và chênh lệch một điểm phần trăm giữa hai mô hình có nghĩa không? Câu trả lời khác nhau ra sao giữa hai phép đo độc lập và một phép so sánh ghép cặp trên cùng tập test?

**Bài tập phụ.** Cài PCA và *whitening* bằng NumPy thuần trên CIFAR-10 — tự tính ma trận hiệp phương sai, tự phân rã, hiển thị 16 vector riêng đầu tiên dưới dạng ảnh — rồi trả lời: vì sao whitening gần như không được dùng trong pipeline thị giác hiện đại? Ba lý do đáng tự tìm trước khi đọc gợi ý: chuẩn hoá theo tầng bên trong mạng đã làm một việc tương đương nhưng thích nghi hơn; whitening khuếch đại đúng những hướng phương sai nhỏ vốn chủ yếu là nhiễu cảm biến; và thống kê whitening tính trên tập huấn luyện trở thành một cái bẫy dịch chuyển miền khi triển khai trên camera khác. Nửa ngày, và nó xa SLAM — bài chính của chương nằm ở khối mini project bên dưới.

**Active recall** (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                                   | Trả lời                                                                                                                                                                                                                    |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ••• Phép chiếu vuông góc                              | Phép hạ một vector xuống không gian con gần nó nhất, sao cho phần dư vuông góc với không gian đó. Mọi bài bình phương tối thiểu <i>là</i> một phép chiếu; phương trình chuẩn tắc chỉ là cách viết lại điều kiện vuông góc. |
| •• SVD                                                | Phân rã giá trị kỳ dị: cách viết một ma trận bất kỳ thành xoay, kéo dãn theo vài trục, rồi xoay lần nữa. Dùng để đọc hạng, để xấp xỉ hạng thấp, và để ép một ma trận nhiễu về $SO(3)$ .                                    |
| ••• Số điều kiện                                      | Tỉ số giữa trị kỳ dị lớn nhất và nhỏ nhất. Nó là hệ số khuếch đại sai số của <i>bài toán</i> , nên đối solver hay đối optimizer không cứu được. Nó vô nghĩa khi so giữa hai cách chọn đơn vị đo khác nhau.                 |
| •• Xấp xỉ hạng thấp                                   | Giữ lại $k$ trị kỳ dị lớn nhất để có ma trận hạng $k$ gần bản gốc nhất. Đứng sau PCA, sau phép chiếu về $SO(3)$ , và sau LoRA.                                                                                             |
| • Hạng hiệu dụng                                      | Entropy của phổ trị kỳ dị đã chuẩn hoá. Cần nó vì với số dấu phẩy động hầu như không trị kỳ dị nào bằng đúng không, nên đếm số trị khác không là vô nghĩa.                                                                 |
| •• Vì sao lập $A^T A$ làm mất chữ số có nghĩa?        | Vì trị kỳ dị bị bình phương, nên số điều kiện cũng bình phương. Bạn mất khoảng một nửa số chữ số; QR hoặc SVD giải thẳng trên $A$ thì không.                                                                               |
| ••• Lan truyền ngược                                  | Thuật toán tính gradient bằng cách đi ngược đồ thị phép tính đúng một lượt. Nó rẻ vì mỗi lượt trả về đạo hàm theo <i>mọi</i> tham số, nhưng phải giữ lại giá trị trung gian của lượt xuôi nên tốn bộ nhớ.                  |
| ••• Lan truyền ngược so với lượt ngược                | Lan truyền ngược là <i>thuật toán</i> ; lượt ngược là <i>một lần chạy</i> thuật toán đó trên một batch. Hai từ này hay bị dùng lẫn.                                                                                        |
| ••• Quy tắc khớp shape                                | Gradient của một đại lượng phải có đúng shape của chính nó, nên thường chỉ còn một cách ghép hợp lệ. Khớp shape trước, dấu và hệ số sau; phần còn lại kiểm bằng gradcheck ở kiểu double.                                   |
| •• Điểm yên ngựa                                      | Điểm có gradient bằng không nhưng độ cong âm theo ít nhất một hướng. Trong không gian nhiều triệu chiều, gần như mọi điểm dừng đều là yên ngựa chứ không phải cực tiểu địa phương.                                         |
| •• Vì sao chuẩn hoá và Adam làm huấn luyện nhanh hơn? | Cả hai kéo các hướng về cùng thang đo, tức hạ số điều kiện một cách gián tiếp. Khi các trục gần bằng nhau, một learning rate chung mới vừa cho mọi hướng.                                                                  |

| Thuật ngữ / câu hỏi                                   | Trả lời (tiếp)                                                                                                                                                                                |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●● Loss thành NaN sau vài trăm bước — nguyên nhân?    | Learning rate quá lớn, hoặc tràn số ở độ chính xác 16 bit. Phân biệt: hạ learning rate mười lần mà vẫn NaN thì là tràn số, hãy bật loss scaling hoặc đổi sang bf16.                           |
| ●●● Độ hợp lý                                         | Xác suất mà mô hình gán cho dữ liệu đã quan sát, xem như hàm của tham số. Cực đại nó tương đương cực tiểu âm log của nó — và đó chính là gần hết các hàm mất mát đang dùng.                   |
| ●●● Sai số chuẩn                                      | Với độ chính xác đo trên $n$ mẫu, nó bằng $\sqrt{p(1-p)/n}$ . Với $n = 500$ và $p = 0,9$ thì bằng 1,34 điểm, nên khoảng tin cậy 95% rộng hơn năm điểm. Nó phình to khi các mẫu không độc lập. |
| ●● Phân kỳ KL                                         | Chi phí phụ trội khi mã hoá dữ liệu của phân phối này bằng bảng mã của phân phối kia. Không đối xứng: chiều xuôi ép mô hình phủ mọi mode, chiều ngược ép nó bám lấy một mode.                 |
| ●● Vì sao hồi quy độ sâu bằng MSE cho ra biên mờ?     | Vì ở biên, độ sâu thật là đa phương thức, còn MSE giả định Gaussian đơn phương thức. Mô hình buộc phải trả về trung bình của hai đáp án đúng, tức một giá trị sai ở giữa.                     |
| ● Loss ban đầu của bài 1000 lớp bằng 2 — nghĩa là gì? | Nghi có rò rỉ nhân. Khởi tạo hợp lý phải cho khoảng $\ln 1000 \approx 6,9$ ; nếu ngược lại nó bằng 20 thì lỗi nằm ở khởi tạo hoặc ở tầng cuối.                                                |
| ●●● Aliasing                                          | Hiện tượng thành phần tần số cao <i>gấp</i> xuống thành tần số thấp giả khi lấy mẫu quá thưa. Nó không làm mất thông tin một cách trung thực mà bịa thêm cấu trúc không có thật.              |
| ●● Đẳng biến tịnh tiến                                | Tính chất dịch ảnh rồi lọc cũng bằng lọc rồi dịch. Convolution có tính chất này, nhưng bước stride và pooling phá nó, nên CNN không thật sự bất biến với dịch chuyển.                         |
| ●● Vì sao không cho mạng xuất ra quaternion?          | Vì $q$ và $-q$ là cùng một phép quay, nên hàm mục tiêu gián đoạn, mà một mạng liên tục thì không khớp nổi hàm gián đoạn. Cách sáu số rồi trực giao hoá thì liên tục.                          |

●●● phải thuộc ●● nên nắm ● biết là có

**I Kiến thức này đang sống ở đâu** (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

| Công trình / hệ thống                             | Dùng ý tưởng nào của chương                                                                                                                                                        | Ghi chú                                                                                                          |
|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| PyTorch, JAX                                      | Toàn bộ <i>autograd</i> là hiện thực của vi phân chế độ ngược trên đồ thị tính toán — đúng mạch ở mục 2                                                                            | Hạ tầng, ổn định từ 2017; ít bán rã nhất trong bảng này                                                          |
| Ceres, g2o, GTSAM                                 | Bình phương tối thiểu phi tuyến với phân rã thừa và damping — phép chiếu cộng điều hoà số học của mục 1                                                                            | Hạ tầng SLAM; chính là chỗ nổi với nền sẵn có của bạn                                                            |
| LoRA và họ tinh chỉnh hiệu quả tham số<br>AdamW   | Giả thiết ma trận cập nhật trọng số có hạng thấp, tức định lý xấp xỉ hạng thấp<br>Tách weight decay khỏi bước thích nghi — hệ quả trực tiếp của việc Adam đổi thang đo từng toạ độ | Cách tinh chỉnh mặc định cho mô hình lớn tính tới 9/2026<br>Optimizer mặc định của phần lớn mô hình lớn hiện nay |
| Đặc trưng Fourier trong NeRF và họ biểu diễn ngầm | Mã hoá đầu vào bằng nhiều tần số, vì mạng vốn khớp tần số thấp trước                                                                                                               | Ý tưởng còn sống trong các biến thể dựng cảnh gần đây                                                            |

MINI PROJECT SLAM · Đo số điều kiện của bài toán tư thế trên hai chuỗi EuRoC · vài giờ tới một ngày

**Mục tiêu** biến ba khái niệm trừu tượng nhất của chương — phép chiếu, SVD, số điều kiện — thành ba con số đọc được trên chính hệ SLAM bạn đang chạy. Câu cần trả lời: chuỗi nào khó hơn, và khó vì *hình học chuyển động* chứ không vì thuật toán.

|          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dữ liệu  | hai chuỗi EuRoC có đặc tính chuyển động khác hẳn nhau — một chuỗi tịnh tiến rộng và một chuỗi có nhiều đoạn xoay gần tại chỗ — cùng chân trị tư thế đi kèm tập dữ liệu. Không cần dữ liệu tự quay.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Các bước | (1) với mỗi <i>keyframe</i> , lấy chính ma trận Jacobian mà solver của bạn đã dựng cho bài tối ưu tư thế, hoặc dựng lại bằng cách xếp các hàng ràng buộc sai số tái chiếu của những điểm đang bám được. (2) Cân lại thang đo hai khối biến trước khi so sánh — tịnh tiến tính bằng mét, quay tính bằng radian — vì nếu không thì số điều kiện chỉ phản ánh lựa chọn đơn vị. (3) Tính phổ trị kỳ dị bằng SVD và ghi $\kappa = \sigma_{\max}/\sigma_{\min}$ theo thời gian cho cả hai chuỗi. (4) Ở mỗi chỗ phải chuẩn hoá một ma trận quay ước lượng, hãy làm bằng SVD: đặt cả ba trị kỳ dị bằng một. Ghi lại độ lệch trục giao <i>trước</i> khi chiếu, như một chỉ báo sức khoẻ số học. (5) Tính <i>RPE</i> bằng <i>evo</i> trên cửa sổ trượt để có một đường sai số cùng trục thời gian. |
| Xong khi | bạn có một hình duy nhất, vẽ hai đường $\kappa$ theo thời gian trên cùng trục. Trên đó bạn chỉ ra các đoạn $\kappa$ vọt lên, và đối chiếu chúng với đoạn chuyển động gần thuần quay hoặc điểm gần đồng phẳng. Cuối cùng bạn nêu một ngưỡng $\kappa$ mà vượt qua thì RPE của sổ trượt tăng rõ trên cả hai chuỗi.                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Nối với  | <a href="#">A/01/01</a> cho SVD và số điều kiện; <a href="#">A/01/04</a> cho phép chiếu về $SO(3)$ ; <a href="#">A/02/03</a> cho lý do hình học khiến cấu hình gần thuần quay là suy biến. Đường $\kappa$ theo thời gian sinh ra ở đây được dùng lại làm biến giải thích trong mini project của chương 2.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

#### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì chi phí nằm ở *thứ tự* nhân các Jacobian, không ở quy tắc chuỗi. Nhân ngược từ phía đầu ra vô hướng cho mọi đạo hàm trong một lượt; nhân theo chiều xuôi thì tốn một lượt cho mỗi tham số.
- A2. Vì số điều kiện đo tỉ số giữa trục dài nhất và trục ngắn nhất của bài toán *sau khi đã chọn đơn vị*; đổi đơn vị chỉ co giãn các trục chứ không đổi lượng thông tin mà dữ liệu ràng buộc. Nên chỉ so sánh được khi cách tham số hoá giữ nguyên.
- A3. Vì đặt hai khoảng tin cậy độc lập cạnh nhau là phép so sánh sai. Trên cùng một tập test, hai mô hình sai ở phần lớn cùng những mẫu khó, nên chỉ các mẫu *bất đồng* mới mang thông tin — và kiểm định ghép cặp trên chúng có độ bất định nhỏ hơn nhiều.
- A4. Vì không tồn tại phép tham số hoá liên tục của  $SO(3)$  bằng ba hay bốn số: quaternion phủ hai lần nên hàm mục tiêu gián đoạn, mà một mạng liên tục theo đầu vào thì không khớp nổi hàm gián đoạn. Sáu số rồi trục giao hoá thì liên tục.

#### NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn phải tìm chỗ thấp nhất của một thung lũng, giữa sương mù dày, và chỉ nhìn được mặt đất ngay dưới chân. Cách duy nhất là sờ xem phía nào dốc xuống rồi bước một bước về phía đó. Chương này là ba câu hỏi nảy ra từ đúng tình huống ấy.

Thung lũng này hình gì? Nhiều thung lũng thật ra là một khe rất hẹp và rất dài: đi ngang thì dốc đứng, đi dọc thì gần như bằng phẳng. Không biết điều đó thì bạn cứ nảy qua nảy lại giữa hai vách mà chẳng tiến xuống bao nhiêu. Có một phép tính cho biết khe hẹp gấp bao nhiêu lần chiều dài của nó, và con số ấy dự báo trước bạn sẽ phải chờ lâu cỡ nào.

Bước bao nhiêu là vừa? Bước dài thì văng qua vách bên kia, bước ngắn thì cả đời không tới đáy. Làm sao biết mình đã xuống thật? Cái đồng hồ đo độ cao trong tay có sai số. Nếu nó sai chừng hai mét mà bạn khoe vừa xuống thêm một mét, thì bạn chưa khoe được gì.

Cái giá: mọi mẹo trong chương chỉ làm khe rộng ra một chút cho dễ đi, chứ không ai vén được sương mù.

**Chỗ không rút gọn được:** vì sao dừng ở một cái đáy rộng lại cho kết quả bền hơn dừng ở đáy nhọn thì tới nay chưa có lời giải thích vừa gọn vừa đúng — và ngay cả chữ “rộng” cũng đổi nghĩa khi ta đổi cách viết các con số mô tả mô hình.

**Một câu:** Toán ở chương này chỉ để trả lời ba câu: thung lũng hẹp theo hướng nào, bước bao nhiêu thì an toàn, và khi nào cái đồng hồ đo mới thật sự nói rằng bạn đã xuống thấp hơn.

## 1.6 Kết nối

Chương này là tiền đề kỹ thuật của cả cây, nhưng nó nổi ra ngoài theo ba hướng rất khác nhau. Hướng thứ nhất đi sang [A/02-vat-ly-tao-anh](#): phần lấy mẫu và phần hình học của mục 4 được dùng gần như liên tục ở đó, nên đây là nhánh nên đọc tiếp ngay. Hướng thứ hai đi sang Phần B, nơi các đại lượng ở đây đổi vai. Số điều kiện và bộ nhớ kích hoạt không còn là tính chất toán học; chúng trở thành *ràng buộc thiết kế*, đứng cạnh ràng buộc về nhân và về độ trễ. Hướng thứ ba đi sang E: toàn bộ mục 3 là bộ công cụ mà chương đánh giá dùng hằng ngày, và nếu bỏ qua nó thì mọi bảng kết quả về sau đều đọc sai.

### Tổng kết chương

Chương này đã dựng bốn công cụ và, quan trọng hơn, đã gắn mỗi công cụ vào một câu hỏi cụ thể sẽ quay lại. Không gian con và phép chiếu cho ta cách đọc mọi bài bình phương tối thiểu; phổ và số điều kiện cho ta ngôn ngữ chung giữa “bài toán suy biến” trong ước lượng trạng thái và “landscape khó tối ưu” trong huấn luyện. Bộ máy vi phân cho ta quy tắc khớp shape trước rồi mới lo dấu, cùng hiểu biết vì sao lan truyền ngược đặt về bộ nhớ chứ không về phép tính. Xác suất và lý thuyết thông tin cho thấy hàm mất mát không phải phát minh rời rạc. Nó là âm log độ hợp lý của một giả định phân phối. Kèm theo đó là thói quen tính sai số trước khi tin một con số. Hình học và tín hiệu cho ta cấu trúc có sẵn của dữ liệu ảnh cùng hai câu hỏi phòng vệ: đã lọc trước khi giảm mẫu chưa, và biểu diễn quay này có liên tục không.

Còn bỏ ngỏ khá nhiều, và bỏ ngỏ có chủ ý. Chương này không đụng tới lý thuyết học thống kê — vì sao học được từ mẫu hữu hạn — đó là nội dung của Phần B. Nó không đụng tới tối ưu hoá lồi ở mức chứng minh, không đụng tới giải tích ngẫu nhiên hay lý thuyết ma trận ngẫu nhiên, và cố ý chỉ chạm Lie group ở đúng ba thao tác dùng được. Nếu sau này bạn thấy cần sâu hơn ở một nhánh, đó là dấu hiệu tốt: nhu cầu thật xuất hiện trước, việc học đến sau. Nhánh nên đọc tiếp ngay là [A/02-vat-ly-tao-anh](#), nơi phần lấy mẫu và phần hình học của mục 4 được dùng liên tục.

### Câu hỏi tự kiểm (trả lời không nhìn bài)

- Mục 1 nói số điều kiện là tính chất của *bài toán* nên đối solver không cứu được, còn mục 2 nói chuẩn hoá và Adam làm huấn luyện nhanh hơn nhờ hạ số điều kiện. Hai phát biểu này mâu thuẫn ở chỗ nào, và giả định nào khác nhau khiến cả hai cùng đúng?
- Bài bình phương tối thiểu của mục 1 và âm log độ hợp lý Gauss của mục 3 là cùng một thứ viết hai cách. Nếu đổi giả định nhiều sang một phân phối đuôi nặng, công cụ nào của mục 1 — phương trình chuẩn tắc, phép chiếu, số điều kiện — còn dùng được và cái nào mất?
- Bản đồ độ sâu của bạn mờ dọc theo mọi biên vật thể. Mục 3 và mục 4 cho hai nguyên nhân khác hẳn nhau cho cùng triệu chứng đó; hai nguyên nhân là gì, và phép thử nào phân biệt chúng mà không cần huấn luyện lại?
- Bạn cho mạng xuất sáu số rồi trực giao hoá bằng SVD để có một phép quay. Thao tác đó nằm trong đồ thị tính toán, nên mục 2 đòi hỏi thêm điều gì mà mục 4 chưa nói tới, và chuyện gì xảy ra khi hai trị kỳ dị gần bằng nhau?

5. Ghép điều kiện dừng của mục 2 với sai số chuẩn của mục 3: vì sao “val loss không giảm thêm sau nhiều epoch” là quy tắc dừng có nghĩa hơn “chuẩn gradient nhỏ”, và quy tắc đó vẫn hỏng trong trường hợp nào?
6. Whitening bằng PCA và chuẩn hoá bên trong mạng đều nhằm kéo các hướng về cùng thang đo. Vì sao chỉ cái sau sống sót trong pipeline hiện đại, và lý do đó liên quan thế nào tới việc phổ trị riêng thay đổi trong lúc huấn luyện?
7. Đường  $\kappa$  theo thời gian trong mini project chỉ đọc được sau khi bạn cân lại thang đo giữa khối tịnh tiến và khối quay. Nếu bỏ bước cân đó, đường  $\kappa$  có còn vọt lên ở các đoạn gần thuần quay không, và khi đó bạn đang đo cái gì?

# Nguồn

---

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [29] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [30] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [31] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [32] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [33] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [34] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [37] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [38] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [39] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.

## Vật lý tạo ảnh và mô hình camera (Image Formation Physics and Camera Models)

**PHỤ THUỘC** A/01-toan-toi-thieu/04-hinh-hoc-va-tin-hieu. Cần ba thứ từ đó: tọa độ đồng nhất,  $SE(3)$ , và quy tắc lấy mẫu. Đây là chương duy nhất của Phần A có phụ thuộc bắt buộc.

### TÓM TẮT

Chương này mở hộp đen giữa thế giới thật và tensor đầu vào: photon và nhiễu, chuỗi xử lý trong máy ảnh, mô hình camera cùng hình học nhiều view, và các cảm biến ngoài RGB. Đọc xong bạn thiết kế được augmentation có cơ sở vật lý thay vì theo cảm tính. Bạn cập nhật đúng ma trận nội tại sau mọi phép biến đổi ảnh. Và khi hiệu năng tụt lúc triển khai, bạn đi ngược đường đi của tín hiệu để tìm nguyên nhân, thay vì thử đổi mô hình. Nếu chỉ nhớ một điều: *phần lớn khoảng cách giữa kết quả thí nghiệm và kết quả triển khai được quyết định trước khi mô hình nhìn thấy dữ liệu*. Đây là phần bị bỏ sót nhiều nhất, và là thứ phân biệt người làm thị giác máy tính với người dùng CNN.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **ảnh số** — một mảng số hai chiều (ảnh xám) hoặc ba chiều (ảnh màu, chiều thứ ba là kênh). Giá trị mỗi ô gọi là pixel, và trong ảnh 8 bit thông thường nó nằm trong khoảng 0–255.
- **photon và phép đếm** — ánh sáng tới cảm biến dưới dạng các hạt rời rạc; giá trị pixel về bản chất là một phép *đếm* số hạt bắt được, nên nó mang sẵn một mức dao động thống kê không loại bỏ được.
- **ISP (image signal processor)** — chuỗi xử lý bên trong máy ảnh biến số đo thô của cảm biến thành ảnh xem được: nội suy màu, cân bằng trắng, nén dải sáng, khử nhiễu, nén file. Nó được thiết kế cho mắt người chứ không cho mô hình.
- **RAW so với sRGB** — RAW là số đo gần như thô, tuyến tính theo lượng ánh sáng; sRGB là ảnh đã qua ISP, phi tuyến và phụ thuộc nội dung. Mọi tính toán mang ý nghĩa vật lý chỉ đúng ở miền tuyến tính.
- **ma trận nội tại  $K$**  — ma trận  $3 \times 3$  chứa tiêu cự tính bằng pixel và tọa độ tâm ảnh; nó biến một tia trong hệ tọa độ camera thành tọa độ pixel. Nó thuộc về camera, còn tư thế thì thuộc về lần chụp.
- **tăng cường dữ liệu (data augmentation)** — sinh thêm mẫu huấn luyện bằng cách biến đổi mẫu có sẵn — lật, xoay, đổi màu, thêm nhiễu — để dạy mô hình rằng những biến đổi đó không làm đổi nhãn.
- **domain gap (dịch chuyển miền)** — hiện tượng phân phối dữ liệu lúc triển khai khác phân phối lúc huấn luyện, khiến mô hình tụt hiệu năng dù bản thân mô hình không đổi.

### CÂU HỎI MỞ ĐẦU

- Q1. Vì sao nhân đôi giá trị pixel trên ảnh sRGB *không* tương đương phơi sáng gấp đôi — và sai lệch đó lớn cỡ nào?
- Q2. Cùng một cảm biến và cùng một cảnh: vì sao tăng ISO làm ảnh sáng lên mà không làm nó đáng tin hơn chút nào?

- Q3.** Bạn resize ảnh còn một nửa trước khi đưa vào mạng, không có exception nào được báo và loss vẫn giảm — vậy cái gì vừa hỏng?
- Q4.** Một mô hình đạt 95% khi thử nghiệm và 80% khi triển khai, dù vẫn là ảnh của cùng loại vật thể: phần lớn mười lăm điểm ấy mất ở đâu trên đường đi của tín hiệu?

Đây là phần bị bỏ sót nhiều nhất trong mọi lộ trình học thị giác, và là thứ phân biệt người làm computer vision với người dùng CNN. Lý do bỏ sót thì dễ hiểu: các tập dữ liệu chuẩn phát hành dưới dạng JPEG đã xử lý xong, nên toàn bộ chuỗi vật lý phía trước trở nên vô hình. Nhưng nó không biến mất. Nó trở thành một tập giả định ngầm mà bạn thừa hưởng, không biết mình đang giữ. Và nó xuất hoá đơn đúng lúc bạn mang mô hình ra khỏi phòng thí nghiệm.

Bốn mục của chương xếp theo *đường đi của tín hiệu*, từ photon tới quyết định, và mỗi mục trả lời một câu hỏi mà mục trước để lại:

- **Mục 1 — ánh sáng và nhiễu:** tín hiệu bắt đầu từ đâu và nó mang sẵn bao nhiêu bất định. Trả lời “vì sao ảnh thiếu sáng làm mô hình sập, ngoài chuyện tối hơn”.
- **Mục 2 — ISP và không gian màu:** tín hiệu ấy bị biến đổi thế nào trước khi thành ảnh. Trả lời “vì sao augmentation trên sRGB không tương đương thao tác vật lý thật”.
- **Mục 3 — camera và hình học nhiều view:** ánh xạ giữa pixel và thế giới ba chiều. Trả lời “thông tin nào bị xoá lúc chụp và mua lại bằng cách nào”.
- **Mục 4 — cảm biến khác và domain gap:** cùng khung phân tích áp cho depth, thermal, event, LiDAR, radar, rồi tổng kết thành luận điểm chung của cả chương.

Thứ tự này nên giữ. Mục 2 dùng mô hình nhiễu của mục 1, còn mục 4 tổng quát hoá cả ba mục trước. Riêng mục 3, với người đã làm SLAM, phần lớn là ôn tập. Ở đó chỉ có hai chỗ đáng đọc kỹ: quy tắc cập nhật  $K$  sau resize và crop, và đoạn nói mô hình học được che giấu các cấu hình suy biến.

## Bản đồ chương — Vật lý tạo ảnh và mô hình camera

### TÓM TẮT

Bản đồ này nói chương 2 gồm bốn mục nào, và vì sao chúng xếp theo đường đi của tín hiệu từ photon tới quyết định. Nó cũng nói mục nào phụ thuộc mục nào, và phần nào là cốt lõi so với phần nào chỉ cần biết là có. Đọc xong bạn quyết định được trong một phút nên vào mục nào trước cho nhu cầu hiện tại. Nếu chỉ nhớ một điều: thứ tự bốn mục là thứ tự nhân quả của tín hiệu thật, nên đọc lệch thứ tự thì sẽ phải quay lại. Luận điểm gộp của cả chương là domain gap sinh ra dọc chuỗi này, trước khi kiến trúc kịp có ý kiến.

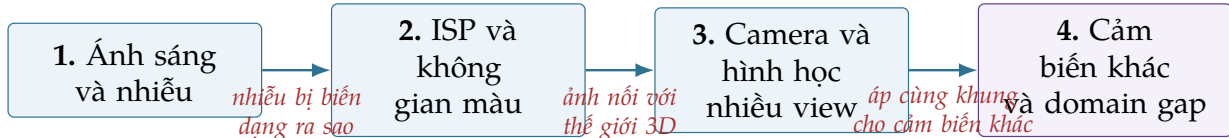
### A. Chương này có gì

- **01-anh-sang-va-nhiều** — phơi sáng, dải động, nhiễu shot so với nhiễu đọc, mô hình phương sai affine, vì sao ảnh thiếu sáng làm mô hình sập.
- **02-isp-va-khong-gian-mau** — khử nhiễu, cân bằng trắng, gamma, ánh xạ tone, nén JPEG; RAW so với sRGB; các không gian màu RGB, HSV, YUV, Lab và augmentation nào có nghĩa vật lý.
- **03-camera-va-hinh-hoc-nhiều-view** — pinhole, ma trận nội tại, méo và hiệu chuẩn; ràng buộc epipolar, homography, tam giác hoá, PnP.

- **04-cam-bien-khac-va-domain-gap** — depth, thermal, event, LiDAR, radar và inductive bias riêng của mỗi loại; nguồn gốc domain gap.

### B. Vì sao chia và xếp thứ tự như vậy

Bốn mục được xếp theo *đường đi vật lý của tín hiệu*, không theo độ khó hay theo lịch sử. Photon tới cảm biến ở mục 1. Chuỗi xử lý trong máy ảnh biến đổi tín hiệu ấy ở mục 2. Mục 3 nối ảnh kết quả lại với hình học ba chiều. Mục 4 áp cùng khung phân tích cho các cảm biến khác, rồi tổng kết thành luận điểm về domain gap. Cách xếp này có một lợi ích cụ thể: mỗi mục trả lời đúng câu hỏi mà mục trước để lại, nên mạch đọc không bị đứt. Hình 2.1 tóm tắt quan hệ đó.



**Hình 2.1.** Bốn mục của chương 2 xếp theo đường đi của tín hiệu. Mục 4 (tô khác màu) không phải một chủ đề mới mà là phần tổng quát hoá: nó áp lại khung phân tích của ba mục đầu cho các cảm biến khác, rồi rút ra luận điểm chung về domain gap.

### C. Phụ thuộc

Cả chương phụ thuộc [A/01/04-hinh-hoc-va-tin-hieu](#). Phần toạ độ đồng nhất và  $SE(3)$  là ngôn ngữ của mục 3. Phần lấy mẫu và aliasing giải thích vì sao khử khảm là bài toán khôi phục, và vì sao undistort làm mờ ảnh. Bên trong chương, mục 2 dùng mô hình nhiễu của mục 1, và mục 4 tổng quát hoá cả ba mục trước; mục 3 thì tương đối độc lập với mục 1 và 2, nên có thể đọc trước nếu nhu cầu hiện tại là hình học.

### D. Cốt lõi hay tham khảo

- **Cốt lõi — phải nắm chắc:** quan hệ  $SNR = \sqrt{N}$  và mô hình  $Var = aI + b$  (mục 1); tính phi tuyến của gamma cùng hệ quả với augmentation (mục 2); quy tắc cập nhật ma trận nội tại sau resize và crop, cùng công thức sai số độ sâu  $Z^2/(fB)$  (mục 3); luận điểm domain gap và mạch bốn bước thu hẹp nó (mục 4).
- **Tham khảo — biết là có, tra khi cần:** chi tiết các thuật toán khử khảm, công thức chuyển đổi giữa các không gian màu, mô hình méo bậc cao, và thông số kỹ thuật cụ thể của từng dòng cảm biến. Nhóm cuối cùng này bán rã nhanh nhất trong cả chương.

### E. Riêng với người đọc đã làm SLAM

Mục 3 phần lớn là ôn tập, nhưng có ba chỗ mới đáng dừng: quy tắc cập nhật  $K$  khi ảnh bị resize hoặc crop trong dataloader (bug im lặng phổ biến nhất khi ghép mạng vào pipeline hình học); sự khác biệt giữa hình học cổ điển “hồng ồn ào” và mô hình học được “hồng im lặng”; và lý do các mô hình độ sâu đơn ảnh dự đoán độ sâu tương đối. Ngược lại, mục 1 và mục 2 gần như hoàn toàn mới với người xuất thân từ ước lượng trạng thái. Chúng giải thích một trải nghiệm quen thuộc mà kinh nghiệm SLAM không giải thích nổi: vì sao hệ thống mất track ngay khi trời tối.

## I 2.1 Ánh sáng, cảm biến và mô hình nhiễu (Light, Sensors and Noise Models)

**PHỤ THUỘC** A/01-toan-toi-thieu/03-xac-suat-va-thong-tin (kỳ vọng, phương sai — ở đây dùng cho một biến đếm Poisson); A/01/04-hinh-hoc-va-tin-hieu nếu muốn hiểu vì sao khử nhiễu và giảm mẫu tương tác với nhau.

### TÓM TẮT

Mục này truy nguyên chuỗi nhân quả từ số photon rơi vào cảm biến tới lúc mô hình mất độ chính xác, và cho bạn một mô hình nhiễu định lượng dùng được: phương sai affine theo cường độ. Đọc xong bạn giải thích được vì sao ảnh thiếu sáng phá mô hình theo cách mà nhiễu Gauss thêm vào ảnh sRGB không mô phỏng nổi. Bạn cũng biết khi nào tăng ISO là vô ích, và vùng cháy sáng khác vùng chìm tối ở chỗ nào. Nếu chỉ nhớ một điều: *nhiều ảnh phụ thuộc tín hiệu*. Tỉ số tín hiệu trên nhiễu chỉ tăng theo  $\sqrt{N}$ , với  $N$  là số photon. Mọi kỹ thuật “làm sáng ảnh” vì thế chỉ khuếch đại; chỉ thu thêm ánh sáng thật mới thêm thông tin.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **photon và electron** — photon là hạt ánh sáng; cảm biến ảnh biến mỗi photon bắt được thành một electron tích trong một “giếng” ứng với một pixel, rồi đếm số electron đó. Vì vậy giá trị pixel về bản chất là một *phép đếm*.
- **phơi sáng (exposure)** — tổng lượng ánh sáng mà cảm biến thu được trong một lần chụp; nó do thời gian mở cửa trập và độ mở của ống kính quyết định.
- **ISO** — mức khuếch đại áp lên tín hiệu *sau khi* đã đo. Tăng ISO làm ảnh sáng lên nhưng không làm cảm biến thu thêm ánh sáng.
- **tỉ số tín hiệu trên nhiễu (SNR)** — tỉ số giữa độ lớn của tín hiệu thật và độ lớn trung bình của nhiễu. SNR càng nhỏ thì càng khó phân biệt chi tiết thật với nhiễu.
- **phân phối Poisson** — phân phối của một phép đếm các sự kiện độc lập xảy ra với tốc độ trung bình cố định. Tính chất định nghĩa của nó là phương sai bằng kỳ vọng — đếm được trung bình  $N$  thì dao động cỡ  $\sqrt{N}$ .
- **ISP (image signal processor)** — chuỗi xử lý bên trong máy ảnh biến số đo thô của cảm biến thành ảnh xem được: nội suy màu, cân bằng trắng, nén dải sáng, khử nhiễu, nén file.
- **RAW so với sRGB** — RAW là số đo gần như thô của cảm biến, còn tuyến tính theo ánh sáng; sRGB là ảnh đã qua ISP, phi tuyến, đã bị chỉnh cho hợp mắt người. Mọi tính toán vật lý chỉ đúng trên RAW.
- **track trong SLAM** — trạng thái hệ thống còn bám được vào bản đồ và ước lượng được tư thế camera. “Mất track” nghĩa là không tìm đủ điểm khớp giữa ảnh hiện tại và bản đồ.

### A. Đặt vấn đề

Có một câu hỏi mà hầu hết người làm deep learning cho thị giác không trả lời được, dù nó quyết định phần lớn thất bại thực tế: *ảnh thiếu sáng khác ảnh đủ sáng ở chỗ nào, ngoài chuyện tối hơn?* Câu trả lời phổ biến — “nó nhiễu hơn” — đúng nhưng nông tới mức vô dụng, vì nó gợi ý một cách sửa sai: thêm nhiễu Gauss vào tập huấn luyện. Cách sửa đó gần như không giúp gì, và lý do nằm ở vật lý của phép đếm photon.

Với người đã cài SLAM, đây là mảnh còn thiếu của một trải nghiệm quen thuộc. Bạn đã thấy hệ thống chạy hoàn hảo ban ngày và mất **track** ngay khi trời tối, và đã quy nó cho “ít feature”. Đúng, nhưng chưa đủ: ít feature chỉ là triệu chứng cuối của một chuỗi bắt đầu từ số photon rơi vào mỗi giếng điện tích trong khoảng phơi sáng. Mục này đi ngược chuỗi đó tới tận gốc, vì gốc mới là chỗ can thiệp được.

**Học xong mục này thì làm được gì mà trước đó không làm được?** Đo và fit mô hình nhiễu của một camera cụ thể bằng vài chục tấm ảnh; quyết định đúng giữa “tăng thời gian phơi sáng”

và “tăng ISO” cho một robot đang chạy; và thiết kế **tăng cường dữ liệu** (*data augmentation*) mô phỏng điều kiện thiếu sáng theo cách có cơ sở vật lý thay vì theo cảm tính.

#### TRỰC GIÁC

Cảm biến là một lưới xô hứng mưa, mở nắp trong đúng một khoảng thời gian. Số giọt rơi vào một cái xô là biến ngẫu nhiên: cùng cơn mưa, cùng khoảng thời gian, hai cái xô cạnh nhau vẫn đếm ra hai số khác nhau — và độ dao động ấy là tính chất của mưa, không phải khuyết tật của xô. Đó là *shot noise*, và không cảm biến nào loại bỏ được. Tăng ISO không phải hứng thêm mưa; nó là đổi sang thước đo có vạch to hơn, để cái xô gần cạn cũng đọc ra một con số. Con số ấy vẫn dựa trên đúng vài giọt đã hứng, nên không đáng tin hơn chút nào.

### B. Ba núm điều khiển ánh sáng, và chỉ hai núm thêm thông tin

Lượng ánh sáng tới cảm biến được điều khiển bởi ba núm, nhưng chúng *không* tương đương nhau — đây là điểm bị hiểu sai nhiều nhất.

**Bảng 2.1.** Ba núm điều khiển phơi sáng. Cột “thêm photon?” là cột phân biệt bản chất; cột cuối quyết định lựa chọn trên robot.

| Núm                 | Thêm photon? | Cơ chế                                                          | Hỏng khi nào                                                                                               |
|---------------------|--------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Thời gian phơi sáng | Có           | Mở cửa trập lâu hơn, thu nhiều photon hơn theo tỉ lệ tuyến tính | Vật chuyển động hoặc camera rung $\Rightarrow$ nhoè; với robot di động đây thường là cái giá đắt hơn nhiều |
| Khẩu độ             | Có           | Mở rộng lỗ thu sáng                                             | Giảm độ sâu trường ảnh nên vật ngoài mặt phẳng lấy nét bị mờ; nhiều camera nhúng có khẩu cố định           |
| ISO (khuếch đại)    | Không        | Nhân tín hiệu đã đo lên trước hoặc trong khi số hoá             | Không thêm thông tin — ảnh sáng lên và nhiễu sáng lên cùng hệ số; chỉ có ích khi read noise đang chi phối  |

$\rightarrow$  cùng nguyên lý với B/03 — độ khó không biến mất mà bị đẩy đi chỗ khác: giảm nhiễu bằng phơi sáng lâu thì trả giá bằng ảnh mờ do chuyển động

Chuỗi đánh đổi này giải thích vì sao ảnh ban đêm từ một robot di động luôn tẻ theo *hai* chiều cùng lúc: vừa nhiễu vừa nhoè. Nó cũng giải thích vì sao mô phỏng chỉ một trong hai chiều là không đủ để mô hình chịu được ban đêm thật.

### C. Hai nguồn nhiễu, và chế độ nào đang chi phối

Số photon thu được trong một khoảng thời gian tuân theo phân phối Poisson, mà tính chất định nghĩa của Poisson là phương sai bằng kỳ vọng. Nếu trung bình một điểm ảnh thu  $N$  photon thì độ lệch chuẩn là  $\sqrt{N}$ , nên

$$\text{SNR} = \frac{N}{\sqrt{N}} = \sqrt{N}.$$

Nói bằng lời: *muốn ảnh sạch gấp đôi thì phải thu gấp bốn lần ánh sáng* — một quan hệ lợi ích giảm dần đặt trần cứng lên mọi thứ phía sau. Ví dụ tính tay: điểm thu 10 000 photon có SNR bằng 100; điểm thu 100 photon có SNR bằng 10. Chênh một trăm lần về ánh sáng chỉ đổi lấy mười lần về SNR. Đó là lý do vùng bóng đổ luôn là chỗ mô hình sai trước tiên, ngay trong một tấm ảnh mà mắt người thấy là “đủ sáng”.

Nguồn thứ hai, *read noise*, đến từ mạch đọc và bộ chuyển đổi tương tự sang số. Nó gần như độc lập với mức tín hiệu, nên mô hình hoá bằng một hằng số cộng vào phương sai. Gộp lại, phương sai đo được ở miền tuyến tính có dạng affine:

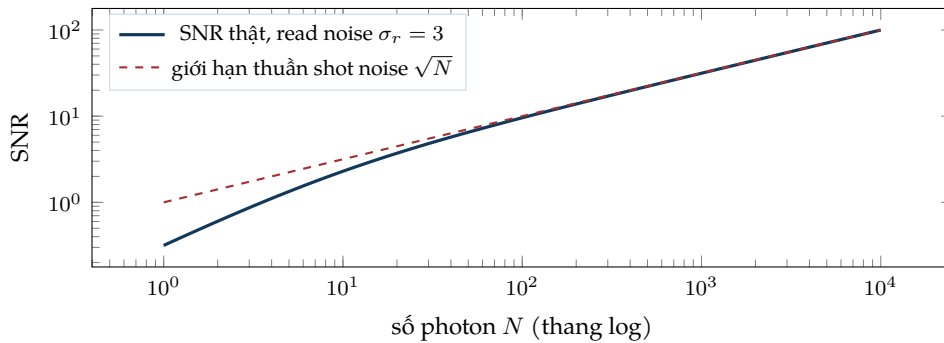
$$\text{Var}(I) = aI + b.$$

Nói bằng lời: *phần tỉ lệ với cường độ là nhiễu photon, phần hằng số là nhiễu của mạch điện*.

↪ **trường hợp riêng của  $A/01$**  — mô hình nhiễu này là một giả định phân phối, nên chọn hàm mất mát cho bài khử nhiễu chính là chọn mô hình nhiễu

Đây chính là mô hình mà mini project cuối chương yêu cầu bạn fit. Nó khớp rất tốt trên dữ liệu thật: đo được, không phải phỏng đoán.

Hình 2.2 cho thấy hai chế độ và chỗ giao nhau của chúng.



**Hình 2.2.** Hai chế độ nhiễu. Ở mức sáng (bên phải) hai đường trùng nhau: shot noise chi phối, và không cải tiến mạch điện nào giúp được. Ở mức rất tối (bên trái) đường thật tụt hẳn xuống dưới: read noise chi phối, và đây là vùng duy nhất mà cảm biến tốt hơn tạo khác biệt thật — một khác biệt phần cứng mà không kỹ thuật huấn luyện nào bù được.

Kiểm chứng bằng hai phép tính tay với  $\sigma_r = 3$  electron. Ở mức 100 photon, tổng độ lệch chuẩn là  $\sqrt{100 + 9} \approx 10,4$  — gần như thuần shot noise, tăng ISO chẳng ích gì. Ở mức 4 photon, tổng là  $\sqrt{4 + 9} \approx 3,6$  so với tín hiệu 4, tức SNR khoảng 1,1: tín hiệu và nhiễu ngang nhau, ảnh chỉ còn là hạt.

#### D. Dải động: thông tin bị xoá vĩnh viễn theo hai kiểu khác nhau

**Dải động** (*dynamic range*) của cảm biến là tỉ số giữa mức bão hoà của giếng điện tích và sà nhiễu đọc, thường quy ra “stop” (mỗi stop là một lần gấp đôi). Cảnh thật thì có dải động lớn hơn nhiều: một hành lang có cửa sổ ban ngày dễ dàng vượt khả năng ghi của cảm biến điện thoại trong một lần phơi sáng. Hệ quả là hai chế độ mất mát *không đối xứng*:

- **Vùng cháy sáng** — bão hoà ở giá trị lớn nhất, mọi khác biệt bên trong bị xoá hoàn toàn. Không hậu xử lý nào, không mạng nào khôi phục được, vì thông tin không còn ở đó.
- **Vùng chìm tối** — không bị xoá hẳn nhưng nằm dưới sà nhiễu; kéo lên được, nhưng kéo lên chỉ khuếch đại nhiễu.

Sự bất đối xứng ấy có một hệ quả thực hành ít người tận dụng: khi phải chọn, *phơi sáng thiếu một chút an toàn hơn phơi sáng thừa*. Nó cũng giải thích vì sao mọi hệ thống thị giác ngoài trời cần

điều khiển phơi sáng thông minh hơn quy tắc “giữ trung bình ảnh ở giữa”. Và vì sao HDR ghép nhiều mức phơi sáng gần như bắt buộc cho xe tự lái. Cái giá của HDR là bóng ma chuyển động khi ghép — lại một đánh đổi mới.

### I Mạch 1 — ảnh thiếu sáng phá mô hình theo đúng thứ tự này

- 1 Vấn đề** ít photon.

**Hệ quả** SNR tụt theo  $\sqrt{N}$ , nên vùng tối gần như chỉ còn nhiễu.

**Giải pháp** cảm biến hoạt động đúng — đây là giới hạn vật lý, không phải lỗi kỹ thuật.
- 2 Vấn đề** ảnh quá tối để nhìn.

**Giải pháp** khuếch đại bằng ISO và bằng đường cong tone của ISP.

**Giá phải trả** nhiễu được khuếch đại cùng hệ số, và vì đường cong *gamma* dốc ở vùng tối, nhiễu ở bóng đổ bị kéo giãn *mạnh hơn* nhiễu ở vùng sáng.

**Vấn đề mới** nhiễu vốn đã phụ thuộc tín hiệu, sau bước này còn phụ thuộc theo một cách phi tuyến.
- 3 Vấn đề** ảnh nhiễu trông xấu.

**Giải pháp** ISP bật khử nhiễu mạnh.

**Giá phải trả** chi tiết tần số cao bị xoá cùng với nhiễu, nên góc và cạnh — đúng thứ mà cả ORB lẫn tầng đầu của CNN dựa vào — biến mất.

**Vấn đề mới** mô hình đối mặt với một phân phối ảnh chưa từng thấy: mịn nhân tạo ở vùng phẳng, nhiễu có tương quan không gian ở vùng còn lại.
- 4 Vấn đề** muốn mô hình chịu được ban đêm nên thêm nhiễu Gauss vào ảnh sRGB khi huấn luyện.

**Giải định** nhiễu là cộng, độc lập từng pixel, không phụ thuộc cường độ.

**Hệ quả** cả ba giả định đều sai sau khi qua ISP, nên augmentation kiểu này mô phỏng một thứ không tồn tại.

**Giải pháp** gỡ xử lý ảnh sRGB ngược về miền tuyến tính, thêm nhiễu shot cộng read đúng mô hình affine, rồi cho chạy lại qua ISP mô phỏng — ý tưởng của dòng *Unprocessing Images for Learned Raw Denoising* [29].

### E. Giới hạn và trường hợp hỏng

- **Mô hình affine bỏ qua vài hiệu ứng thật.** Dòng tối phụ thuộc nhiệt độ, nên cùng một camera cho tham số khác nhau giữa sáng sớm và giữa trưa trên xe. Có nhiễu cố định theo vị trí (*fixed-pattern noise*) không ngẫu nhiên và không giảm khi lấy trung bình nhiều khung. Điểm ảnh chết và điểm nóng bị ISP nội suy đè lên, tạo cấu trúc giả.
- **Mô hình chỉ đúng ở miền tuyến tính.** Sau khử khảm và gamma thì nhiễu không còn độc lập giữa các pixel lẫn giữa các kênh — chủ đề của mục tiếp theo.
- **Nhiều thật khó mô phỏng đủ tốt để thay dữ liệu thật.** Các tập dữ liệu chụp cặp ảnh nhiễu-sạch trên điện thoại thật, chẳng hạn SIDD [30], tồn tại chính vì mô phỏng chưa đủ; một bộ khử nhiễu huấn luyện thuần trên nhiễu tổng hợp thường tụt rõ khi đem sang ảnh thật.

#### CẠM BẦY

“Tăng độ sáng” bằng cách nhân giá trị pixel không phải là mô phỏng phơi sáng nhiều hơn — sai theo cả hai nghĩa. Sai về phi tuyến (mục sau tính ra con số cụ thể), và sai về thống kê: phơi sáng thật gấp đôi làm SNR tăng  $\sqrt{2}$  lần, còn nhân giá trị lên gấp đôi giữ nguyên

SNR. Một mô hình học từ augmentation kiểu này sẽ tin rằng ảnh sáng và ảnh tối có cùng mức tin cậy — đúng cái niềm tin làm nó sập lúc trời tối.

#### CẦN NHỚ

Nhiều ảnh phụ thuộc tín hiệu:  $\text{Var} = aI + b$ , phần thứ nhất là photon, phần thứ hai là mạch điện. Hệ quả duy nhất cần mang theo: *chỉ có thu thêm ánh sáng thật mới thêm thông tin*; mọi thứ khác — ISO, kéo tone, khử nhiễu, augmentation độ sáng — đều chỉ phân phối lại cái đã có, và thường xoá bớt. [T1]

## F. Kết nối

Mục này là gốc của cả chương. Nó chuyển tiếp trực tiếp sang [A/02/02-isp-va-khong-gian-mau](#), nơi ta theo dấu tín hiệu qua các tầng xử lý và thấy nhiễu bị biến dạng ra sao; và là tiền đề của [A/02/04](#), vì “mỗi cảm biến có đặc tính nhiễu riêng” chỉ có nghĩa khi đã hiểu đặc tính nhiễu của cảm biến RGB. Theo chiều ngang nó nối với [A/01/03](#): mô hình nhiễu ở đây là một giả định phân phối, nên chọn loss cho bài khử nhiễu chính là chọn mô hình nhiễu.

↔ **hệ quả của E** — nhiều dạng nhiễu trong các benchmark robustness là mô phỏng thô của đúng chuỗi nhân quả này, nên điểm số trên chúng phải đọc thận trọng

Và nó nối với E: nhiều dạng **corruption** trong các benchmark robustness thực chất là mô phỏng thô của chuỗi nhân quả mô tả ở đây, nên điểm số trên chúng nên đọc thận trọng.

#### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nhiễu shot phụ thuộc tín hiệu. Điều đó phá vỡ giả định nào của mô hình nhiễu Gauss cộng, và hệ quả với hàm mất mát là gì?
2. Vì sao tăng ISO không cải thiện SNR trong khi kéo dài thời gian phơi sáng thì có? Cái giá của lựa chọn thứ hai với một robot đang di chuyển là gì?
3. Vùng cháy sáng và vùng chìm tối hỏng theo hai kiểu khác nhau. Kiểu nào cứu được, và điều đó gợi ý gì về chiến lược phơi sáng?
4. Ở mức tín hiệu rất thấp, nguồn nhiễu nào chi phối, và làm sao nhận ra chế độ đó từ dữ liệu đo được?
5. Vì sao thêm nhiễu Gauss vào ảnh sRGB là cách mô phỏng ban đêm tồi, và quy trình đúng gồm những bước nào?
6. Bạn đo  $\text{Var} = aI + b$  trên cùng một camera ở ISO 100 và ISO 6400. Hai tham số thay đổi thế nào, và vì sao?

# Nguồn

---

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [29] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [30] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [31] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [32] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [33] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [34] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [37] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [38] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [39] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.

## 2.2 ISP, RAW so với sRGB và không gian màu (Image Signal Processing and Color Spaces)

**PHỤ THUỘC** [A/02-vat-ly-tao-anh/01-anh-sang-va-nhieu](#) (mô hình nhiễu affine — mục này theo dấu nhiễu ấy qua từng tầng xử lý); [A/01/04-hinh-hoc-va-tin-hieu](#) (lấy mẫu — khử nhiễu là một bài toán lấy mẫu).

### TÓM TẮT

Mục này mở hộp đen giữa cảm biến và tensor đầu vào: một tấm ảnh sRGB đã đi qua khử nhiễu, cân bằng trắng, gamma, ánh xạ tone và nén JPEG trước khi mô hình nhìn thấy nó, và mỗi tầng phá một giả định khác nhau. Đọc xong bạn biết vì sao “tăng độ sáng” trên sRGB không tương đương tăng phơi sáng (tính ra được con số: gấp đôi phơi sáng chỉ làm giá trị sRGB tăng khoảng 37%), vì sao giả thiết nhiễu độc lập từng pixel sai sau khử nhiễu, và augmentation màu nào có nghĩa vật lý. Nếu chỉ nhớ một điều: *ISP là một phép biến đổi phi tuyến, phụ thuộc nội dung, và không khả nghịch* — nên mọi phép mô phỏng vật lý phải làm ở miền tuyến tính rồi mới dựng ảnh lại, chứ không làm thẳng trên sRGB.

### Kiến thức cần có trước

- **ISP (image signal processor)** — chuỗi xử lý bên trong máy ảnh biến số đo thô của cảm biến thành ảnh xem được. Nó được thiết kế cho mắt người chứ không cho mô hình.
- **RAW so với sRGB** — RAW là số đo gần như thô, còn *tuyến tính* theo lượng ánh sáng — gấp đôi ánh sáng thì gấp đôi giá trị. sRGB là ảnh đã qua ISP: phi tuyến, phụ thuộc nội dung, và không khôi phục lại RAW được một cách chính xác.
- **ma trận lọc màu Bayer và khử khảm** — mỗi pixel của cảm biến chỉ đo được *một* trong ba màu, sắp theo một hoa văn cố định gọi là Bayer; khử khảm là bước nội suy hai màu còn thiếu tại mỗi pixel.
- **cân bằng trắng (white balance)** — nhân mỗi kênh màu với một hệ số để vật màu trắng trông trắng dưới nguồn sáng hiện tại. Hệ số này được ước lượng từ chính cảnh, nên nó thay đổi theo nội dung ảnh.
- **gamma** — phép nén dải sáng bằng hàm mũ khoảng  $1/2,2$ , để các mã số được phân bố đều theo cảm nhận của mắt thay vì đều theo năng lượng ánh sáng. Đây là chỗ ảnh mất tính tuyến tính.
- **ánh xạ tone (tone mapping)** — điều chỉnh độ sáng và tương phản, thường *cục bộ* — cùng một mức sáng ở hai vùng khác nhau của ảnh có thể bị xử lý khác nhau.
- **JPEG và DCT** — JPEG chia ảnh thành các khối  $8 \times 8$ , biến mỗi khối thành các hệ số tần số bằng phép biến đổi cosin rời rạc (DCT), rồi làm tròn thô các hệ số tần số cao. Đây là nén mất mát: thông tin bị bỏ đi không lấy lại được.
- **tăng cường dữ liệu (data augmentation)** — sinh thêm mẫu huấn luyện bằng cách biến đổi mẫu có sẵn — lật, xoay, đổi màu, thêm nhiễu — nhằm dạy mô hình rằng những biến đổi đó không làm đổi nhãn.

### A. Đặt vấn đề

Người mới coi ảnh đầu vào là “dữ liệu thô”. Nó không phải. Giữa photon và mảng số mà `cv2.imread` trả về có một chuỗi xử lý dài, phần lớn do nhà sản xuất thiết kế cho *mắt người*, không cho mô hình. Chuỗi ấy phi tuyến, phụ thuộc nội dung ảnh, thay đổi theo từng hãng máy, và phần lớn là không khả nghịch. Mọi giả định ngầm mà ta mang từ vật lý — nhiễu độc lập từng pixel, giá trị pixel tỉ lệ với ánh sáng, cùng vật thể cho cùng màu — đều bị một tầng nào đó trong chuỗi phá vỡ.

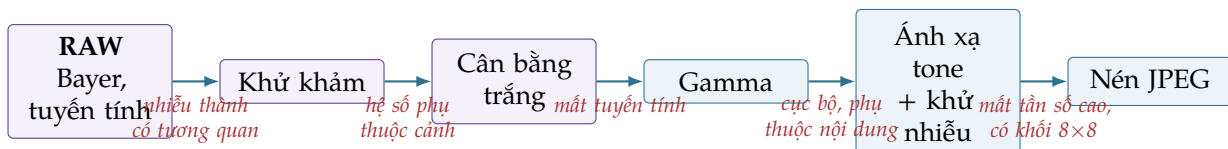
**Học xong mục này thì làm được gì mà trước đó không làm được?** Ba việc: thiết kế augmentation có cơ sở vật lý thay vì chọn theo thói quen; dự đoán trước cái gì hỏng khi huấn luyện trên JPEG rồi triển khai trên luồng RAW hoặc trên một hãng camera khác; và biết khi nào phải bỏ công lấy RAW thay vì tiếp tục làm việc trên sRGB.

### TRỰC GIÁC

ISP giống một phiên dịch viên có phong cách riêng. Bạn đưa cho anh ta một biên bản đo đặc khô khan (ảnh RAW) và anh ta trả lại một bản dịch *để nghe*: cân đối lại màu cho hợp mắt, nhấn chỗ này, làm dịu chỗ kia, cắt bớt chi tiết mà anh ta cho là không ai để ý. Bản dịch đẹp hơn bản gốc và hầu hết người đọc thích nó hơn. Nhưng nếu công việc của bạn là *đo đạc* thì rắc rối bắt đầu. Đổi hãng máy, đổi firmware, đổi chế độ chụp đều là đổi phiên dịch viên. Số liệu của bạn đổi theo, dù hiện thực ngoài kia không đổi chút nào.

### B. Chuỗi ISP: mỗi tầng phá một giả định

Hình 2.3 vẽ chuỗi điển hình và đánh dấu chỗ mỗi giả định vỡ.



**Hình 2.3.** Chuỗi ISP điển hình. Ba khối đầu (tô tím) còn ở miền tuyến tính — đây là nơi duy nhất mô hình nhiễu  $\text{Var} = aI + b$  còn đúng. Từ gamma trở đi, mọi phép mô phỏng vật lý làm trực tiếp trên giá trị pixel đều sai. Nhân dưới mũi tên là giả định bị phá ở bước đó.

→ cùng nguyên lý với A/01 — giảm mẫu màu 4:2:0 trong JPEG là aliasing có chủ ý — cùng cơ chế với bước giảm mẫu trong kiến trúc mạng

Đi qua từng tầng, mỗi tầng để lại một hệ quả cụ thể cho việc huấn luyện:

- **Khử nhiễu** — cảm biến chỉ đo một kênh màu tại mỗi pixel qua ma trận lọc màu Bayer, nên hai phần ba giá trị màu là *nội suy*. Hai hệ quả. Thứ nhất, chi tiết màu tần số cao sinh hoa văn giả: *moiré* và “răng cửa màu”. Thứ hai, nhiễu trở nên có tương quan cả về không gian lẫn giữa các kênh. Giả định “nhiều độc lập từng pixel” chết ngay ở tầng đầu tiên.
- **Cân bằng trắng** — nhân mỗi kênh với một hệ số ước lượng từ chính cảnh. Hệ quả: cùng một vật thể cho ra màu khác nhau tùy cảnh xung quanh; mọi đặc trưng dựa trên màu tuyệt đối đều không đáng tin, và *hệ số này thay đổi giữa hai khung hình liên tiếp* khi cảnh đổi.
- **Gamma** — nén dải sáng bằng hàm mũ khoảng  $1/2,2$  để phân bố mã đều theo cảm nhận của mắt. Đây là chỗ mất tuyến tính, và là chỗ mọi tính toán vật lý phải dừng lại.
- **Ánh xạ tone và khử nhiễu** — *cực bộ* và phụ thuộc nội dung: cùng một mức sáng ở hai vùng khác nhau của ảnh có thể bị xử lý khác nhau.

→ ràng buộc bởi A/02/03 — hình học trực tiếp và ghép ảnh ở mục camera dựa trên giả định này, nên chính sách ISP đặt một trần lên độ chính xác của chúng

Điều này phá vỡ giả định **độ sáng không đổi** (*brightness constancy*), mà **dòng quang** (*optical flow*) và SLAM trực tiếp lại dựa vào. Đó chính là lý do các hệ thống VO trực tiếp phải hiệu chuẩn quang trắc — một chỗ nổi rất thật với nền của bạn.

- **Nén JPEG** — chuyển sang YCbCr, giảm mẫu kênh màu (thường 4:2:0, tức độ phân giải màu chỉ còn một nửa mỗi chiều), rồi lượng tử hoá hệ số DCT trên từng khối  $8 \times 8$ . Hệ quả: mất tần số cao, xuất hiện cấu trúc khối, và vật nhỏ có màu là dấu hiệu chính — đèn giao thông xa chẳng hạn — mất đúng cái dấu hiệu ấy.

### C. Vì sao augmentation trên sRGB không tương đương thao tác vật lý

Đây là hệ quả quan trọng nhất của mục này, và nó tính ra được bằng số.

**Ví dụ tính tay.** Xấp xỉ đường cong sRGB bằng  $V = L^{1/2,2}$  với  $L$  là ánh sáng tuyến tính. Tăng gấp đôi thời gian phơi sáng nghĩa là  $L \rightarrow 2L$ , nên

$$V' = (2L)^{1/2,2} = 2^{0,4545} L^{1/2,2} \approx 1,37 V.$$

Nói bằng lời: *gấp đôi ánh sáng thật chỉ làm giá trị pixel sRGB tăng khoảng 37%, không phải 100%*. Nhân giá trị sRGB lên gấp đôi tương ứng với tăng ánh sáng thật  $2^{2,2} \approx 4,6$  lần — một phép augmentation mạnh hơn nhiều so với ý định.

Sai lệch về hệ số còn là phần nhẹ. Phần nặng là *hành vi của nhiễu*: phơi sáng thật gấp đôi làm SNR tăng  $\sqrt{2}$  lần, còn nhân giá trị pixel thì nhân cả tín hiệu lẫn nhiễu nên SNR không đổi. Mô hình học từ augmentation kiểu này rút ra một quy luật sai — rằng ánh sáng và ảnh tối đáng tin như nhau.

### I Mạch 2 — từ “augmentation trông có vẻ hợp lý” tới quy trình đúng

- 1 Vấn đề** muốn mô hình chịu được thay đổi ánh sáng.

**Giải pháp** nhân giá trị pixel sRGB với một hệ số ngẫu nhiên.

**Giả định** giá trị pixel tỉ lệ với ánh sáng.

**Giá phải trả** giả định sai vì gamma, nên biên độ thực tế lệch vài lần và hành vi nhiễu hoàn toàn sai.
- 2 Vấn đề** vậy làm ở miền tuyến tính.

**Giải pháp** gỡ gamma ( $L = V^{2,2}$ ), nhân hệ số, áp gamma lại.

**Hệ quả** bây giờ hệ số đúng nghĩa vật lý.

**Vấn đề mới** nhiễu vẫn không đúng: ánh sáng lên nhưng vẫn mang nhiễu của mức phơi sáng cũ.
- 3 Vấn đề** cần cả nhiễu đúng.

**Giải pháp** gỡ toàn bộ ISP về RAW xấp xỉ: gỡ gamma, gỡ cân bằng trắng, gỡ ma trận màu. Mô phỏng phơi sáng và nhiễu shot cộng read ở đó, rồi dựng lại ảnh qua một ISP mô phỏng [29].

**Giá phải trả** phải biết hoặc đoán tham số ISP của camera; kết quả là xấp xỉ, không phải RAW thật.
- 4 Vấn đề** khi nào thì bỏ hẳn sRGB và lấy RAW thật?

**Giải pháp** khi bài toán là bài toán tín hiệu mức thấp — khử nhiễu, HDR, chụp thiếu sáng, đo đặc quang học — hoặc khi phải triển khai qua nhiều hãng camera khác nhau.

**Giá phải trả** dữ liệu nặng gấp nhiều lần, hầu như không có mô hình tiền huấn luyện nào nhận RAW, và pipeline phải tự viết.

#### D. Không gian màu: chọn cái nào cho việc gì

**Bảng 2.2.** Bốn không gian màu thường gặp. Cột cuối là cột quyết định.

| Không gian  | Cơ chế                                        | Mạnh khi nào                                       | Hỏng khi nào                                                                                           |
|-------------|-----------------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| RGB         | Ba kênh gắn với cảm biến và màn hình          | Mặc định; mọi mô hình tiền huấn luyện dùng nó      | Phụ thuộc thiết bị; khoảng cách Euclid giữa hai màu không tương ứng cảm nhận                           |
| HSV         | Tách sắc độ, độ bão hoà, độ sáng              | Ngưỡng màu thủ công, augmentation sắc độ rẻ tiền   | Sắc độ vô định khi bão hoà thấp (vùng xám, vùng tối) nên nhiễu sắc độ nổ tung; không đều theo cảm nhận |
| YUV / YCbCr | Tách độ chói khỏi hai kênh màu                | Nén và truyền video; xử lý riêng độ chói           | Đã có giảm mẫu màu sẵn trong hầu hết luồng thật, nên chi tiết màu mất trước khi bạn chạm vào           |
| Lab         | Xấp xỉ đều theo cảm nhận quanh một điểm trắng | Đo khác biệt màu, tô màu ảnh, chuẩn hoá phong cách | Phụ thuộc điểm trắng giả định; chuyển đổi phi tuyến nên nhiễu bị biến dạng khó lường                   |

→ **tiền đề** D — phân biệt augmentation mô phỏng miền với augmentation chính quy hoá ở chương huấn luyện bắt nguồn từ đúng tiêu chí này

Từ bảng trên rút ra tiêu chí thực dụng cho augmentation màu: một phép biến đổi *có nghĩa vật lý* khi nó tương ứng với một thứ có thể thật sự xảy ra ở khâu tạo ảnh.

- **Có nghĩa vật lý:** thay đổi hệ số cân bằng trắng (nhân từng kênh ở miền tuyến tính — tương ứng đổi nguồn sáng); thay đổi phơi sáng ở miền tuyến tính kèm điều chỉnh nhiễu; thêm nhiễu shot cộng read ở miền tuyến tính; thay đổi chất lượng nén JPEG; làm mờ chuyển động dọc theo một hướng hợp lý.
- **Ít hoặc không có nghĩa vật lý:** xoay sắc độ biên độ lớn (không nguồn sáng nào làm vậy); nhiễu Gauss cường độ cố định trên sRGB; đảo kênh màu; thay đổi độ tương phản độc lập từng kênh. Chúng vẫn có ích như một dạng chỉnh quy hoá. Nhưng đừng nhầm chúng với mô phỏng điều kiện triển khai. Khi báo cáo, hãy nói rõ đó là chỉnh quy hoá, không phải mô phỏng miền.

### E. Giới hạn và trường hợp hỏng

- **Gỡ xử lý chỉ là xấp xỉ.** ISP thật có các bước không công bố, phụ thuộc firmware, và một số bước cục bộ thì về nguyên tắc không khả nghịch. Ảnh “RAW tổng hợp” luôn khác RAW thật, và độ lệch ấy đủ lớn để mô hình khử nhiễu tụt điểm — lý do các tập dữ liệu chụp thật vẫn cần thiết [30].
- **Bỏ ISP không phải lúc nào cũng tốt hơn.** ISP là kết quả của hàng chục năm kỹ thuật. Với các bài toán ngữ nghĩa mức cao, ảnh sRGB đã xử lý thường cho kết quả tốt hơn RAW thô. Lý do đơn giản: mô hình tiền huấn luyện đã quen với thống kê của ảnh sRGB. RAW đáng công khi bài toán là bài toán tín hiệu mức thấp.
- **Không gian màu không giải quyết được vấn đề gốc.** Chuyển sang Lab hay HSV không lấy lại được thông tin đã bị lượng tử hoá hay giảm mẫu; nó chỉ sắp xếp lại thông tin còn sót. Nếu chi tiết màu đã mất ở bước 4:2:0 thì mọi phép đổi toạ độ đều vô ích.

#### CẠM BẤY

Huấn luyện trên JPEG chất lượng 90 rồi triển khai trên luồng ít nén hoặc RAW là dịch chuyển miền theo chiều *ít người ngờ*: mô hình đã học rằng vùng phẳng thì thật sự phẳng (vì lượng tử hoá DCT xoá nhiễu ở đó) và biên có cấu trúc khối. Ảnh ít nén mang lại nhiễu tần số cao mà mô hình chưa từng thấy, và nó thường hỏng trước ở các bài toán mức thấp — phân đoạn biên, đếm vật nhỏ — chứ không phải ở phân loại toàn ảnh. Cách phòng vệ rẻ nhất là đưa mức nén thành một tham số augmentation ngay từ đầu.

#### CẦN NHỚ

Ảnh sRGB không phải phép đo, nó là một *bản dịch phi tuyến phụ thuộc nội dung* của phép đo. Mọi thao tác mang ý nghĩa vật lý — phơi sáng, nhiễu, cân bằng trắng — phải làm ở miền tuyến tính rồi mới dựng lại ảnh; làm thẳng trên sRGB thì bạn đang mô phỏng một hiện tượng không tồn tại. [T1]

### F. Kết nối

Mục này tiếp nối trực tiếp A/02/01: mô hình nhiễu affine chỉ đúng ở ba khối đầu của Hình 2.3, và toàn bộ phần còn lại của chuỗi là lý do vì sao nó không quan sát được trên ảnh sRGB. Nó là tiền đề của A/02/04, nơi ta thấy mỗi loại cảm biến có một “ISP” riêng với các tầng méo riêng. Theo chiều ngang, nó nối với ba chỗ. Với A/01/04, vì giảm mẫu màu 4:2:0 chính là aliasing có chủ ý. Với B về thiết kế pipeline dữ liệu. Và với D về augmentation, nơi ta phân biệt rạch ròi augmentation *mô phỏng miền* với augmentation *chỉnh quy hoá* — một phân biệt bắt nguồn từ đúng mục này.

**Câu hỏi tự kiểm** (trả lời không nhìn bài)

1. Vì sao augmentation “tăng độ sáng” trên ảnh sRGB không tương đương tăng phơi sáng thật? Nêu cả sai lệch về hệ số lẫn sai lệch về nhiễu.
2. Nếu train trên JPEG chất lượng 90 và triển khai trên luồng RAW, hỏng ở đâu trước và vì sao?
3. Sau khử nhiễu, giả định nào của mô hình nhiễu ở mục trước không còn đúng? Hệ quả với một mô hình khử nhiễu là gì?
4. Vì sao ánh xạ tone cục bộ phá vỡ giả định độ sáng không đổi, và các hệ VO trực tiếp xử lý chuyện đó bằng cách nào?
5. Khi nào thì đáng bỏ công lấy RAW thay vì tiếp tục dùng sRGB, và cái giá cụ thể là gì?
6. Kể hai phép augmentation màu có nghĩa vật lý và hai phép không có. Với hai phép sau, chúng vẫn có thể có ích — ích ở khía cạnh nào?

## Nguồn

---

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [29] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [30] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [31] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [32] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [33] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [34] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [37] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [38] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [39] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.

## 2.3 Mô hình camera và hình học nhiều view (Camera Models and Multi-View Geometry)

**PHỤ THUỘC** [A/01-toan-toi-thieu/04-hinh-hoc-va-tin-hieu](#) (toạ độ đồng nhất,  $SE(3)$ , nội suy và lấy mẫu — undistort và warp đều là phép lấy mẫu lại).

### TÓM TẮT

Mục này nối pixel với thế giới thực: mô hình pinhole, ma trận nội tại, méo ống kính và hiệu chuẩn, rồi ràng buộc epipolar, homography, tam giác hoá và PnP. Đọc xong bạn biết ma trận nội tại đổi thế nào khi resize hay crop ảnh. Quên chỗ đó là bug im lặng số một khi ghép mạng vào pipeline hình học. Bạn cũng tính được sai số độ sâu ứng với một pixel sai lệch thị sai, và gọi tên được các cấu hình suy biến làm tam giác hoá vô nghĩa. Nếu chỉ nhớ một điều: *camera không đo điểm, nó đo tia*. Độ sâu bị xoá ngay lúc chụp. Mọi phép khôi phục 3D vì thế phải mua lại thông tin đó, bằng một điểm nhìn thứ hai, một chuyển động, hoặc một prior học từ dữ liệu.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **pixel và toạ độ ảnh** — ảnh là một lưới ô vuông; toạ độ  $(u, v)$  đếm cột và hàng, gốc quy ước ở góc trên trái. Đơn vị của mọi đại lượng “trong ảnh” — tiêu cự, thị sai — đều là pixel, không phải mét.
- **toạ độ đồng nhất** — thêm một thành phần vào vector toạ độ để phép tịnh tiến và phép chiếu viết được dưới dạng nhân ma trận: điểm  $(x, y)$  viết thành  $(x, y, 1)$ , và hai vector tỉ lệ với nhau chỉ cùng một điểm.
- **ma trận nội tại  $K$  (intrinsics)** — ma trận  $3 \times 3$  chứa tiêu cự tính bằng pixel  $(f_x, f_y)$  và toạ độ tâm ảnh  $(c_x, c_y)$ ; nó biến một tia trong hệ toạ độ camera thành toạ độ pixel. Nó thuộc về *camera*, không thuộc về cảnh.
- **ma trận ngoại tại (extrinsics)** — phép quay và tịnh tiến đưa toạ độ thế giới về hệ toạ độ camera — tức là tư thế của camera. Nó thuộc về *lần chụp*, không thuộc về camera.
- **méo ống kính (distortion)** — sai lệch làm đường thẳng ngoài đời cong trên ảnh, mạnh nhất ở rìa; mô hình phổ biến dùng vài hệ số đa thức theo bán kính. “Undistort” là lấy mẫu lại ảnh để bù sai lệch đó.
- **hiệu chuẩn (calibration)** — quá trình ước lượng  $K$  và các hệ số méo bằng cách chụp một vật đã biết hình dạng, thường là bàn cờ, ở nhiều tư thế.
- **thị sai (disparity) và đường cơ sở (baseline trong stereo)** — với hai camera đặt cách nhau một khoảng  $B$  (đường cơ sở), cùng một điểm 3D rơi vào hai vị trí lệch nhau  $d$  pixel trên hai ảnh;  $d$  gọi là thị sai và độ sâu là  $Z = fB/d$ .
- **RANSAC** — thủ tục ước lượng chịu được dữ liệu rác: rút ngẫu nhiên tập nhỏ điểm, dựng giả thuyết, đếm số điểm khớp, lặp nhiều lần và giữ giả thuyết được ủng hộ nhất. Cần nó vì ghép đặc trưng luôn có một tỉ lệ ghép sai.

### A. Đặt vấn đề

Với người đã làm SLAM, phần lớn nội dung dưới đây là ôn tập. Trọng tâm vì thế được đặt ở chỗ khác: *cái gì thay đổi khi một mạng nơ-ron được ghép vào pipeline hình học*. Ba câu hỏi cụ thể dẫn dắt cả mục. Thứ nhất, mô hình học được có được phép “quên” hình học không — và nếu không thì hình học vào mô hình bằng đường nào? Thứ hai, những phép biến đổi ảnh mà ta dùng vô tư khi huấn luyện (resize, crop, warp) làm gì với các đại lượng hình học đi kèm? Thứ ba, khi hình học suy biến, mô hình học được có sập theo không, hay nó “đoán” và cho ra một con số trông hợp lý nhưng sai?

**Học xong mục này thì làm được gì mà trước đó không làm được?** Cập nhật đúng ma trận nội tại sau mọi phép biến đổi ảnh; đọc một paper 3D và biết ngay nó giả định biết  $K$  hay không, cần bao nhiêu view, và suy biến ở cấu hình nào; và ước lượng bằng con số xem một hệ stereo cho sai số độ sâu bao nhiêu ở tầm mà bạn quan tâm.

### TRỰC GIÁC

Camera là một cái máy đóng dấu: nó lấy tất cả những gì nằm trên một tia xuất phát từ tâm quang học và đóng chúng chồng lên nhau vào đúng một pixel. Phép đóng dấu ấy không mất mát về *hướng* — một pixel xác định chính xác một tia — nhưng xoá sạch thông tin về *khoảng cách dọc theo tia*. Vì vậy mọi kỹ thuật 3D đều là một cách mua lại đúng chiều thông tin đã bị xoá. Cách thứ nhất là đóng dấu lần thứ hai từ chỗ khác: stereo hoặc chuyển động. Cách thứ hai là dùng hiểu biết sẵn có về thế giới, chẳng hạn kích thước vật quen thuộc hay mặt đất phẳng. Một mạng ước lượng độ sâu đơn ảnh đã học đúng loại hiểu biết thứ hai; nó không phá vỡ định luật nào cả.

## B. Pinhole, nội tại, và chỗ mọi người làm sai

Mô hình pinhole nói rằng một điểm 3D trong hệ tọa độ camera chiếu xuống ảnh theo

$$u = f_x \frac{X}{Z} + c_x, \quad v = f_y \frac{Y}{Z} + c_y.$$

Nói bằng lời: *toạ độ pixel chỉ phụ thuộc tỉ số giữa các toạ độ ngang và độ sâu* — nhân cả cảnh lên gấp đôi và đẩy ra xa gấp đôi thì ảnh không đổi một pixel nào. Đó là gốc rễ của tính **nhập nhằng thang đo** (*scale ambiguity*) trong mọi hệ đơn ảnh và đơn camera. Nó cũng là lý do các mô hình độ sâu đơn ảnh hiện đại dự đoán độ sâu *tương đối* chứ không phải mét [31].

→ **hệ quả của C** — mô hình độ sâu đơn ảnh dự đoán độ sâu *tương đối* chính vì phép chiếu xoá thang đo tuyệt đối

**Ví dụ tính tay.** Với  $f = 500$  pixel, một vật cao 1 m ở khoảng cách 10 m cao  $500 \times 1/10 = 50$  pixel trên ảnh. Cùng vật đó ở 50 m chỉ còn 10 pixel — và đây là lý do vật ở xa không phải “khó nhận dạng hơn một chút” mà là *gần như không còn thông tin*: mọi kiến trúc, mọi augmentation đều không tạo ra được các pixel không tồn tại.

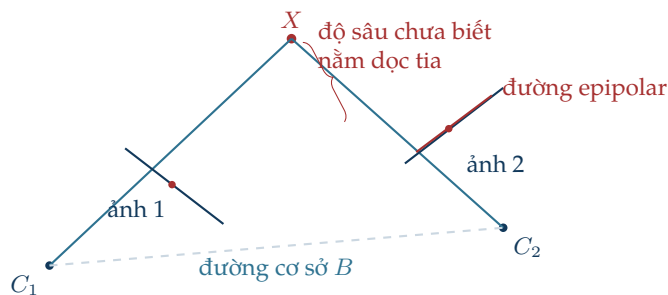
Chỗ mọi người làm sai, và làm sai rất im lặng, là quên rằng  $K$  gắn với *hệ tọa độ pixel* nên nó phải đổi theo mọi phép biến đổi ảnh:

- **Thu nhỏ ảnh với hệ số  $s$**  — mọi phần tử liên quan pixel nhân với  $s$ :  $f'_x = sf_x$ ,  $f'_y = sf_y$ ,  $c'_x = sc_x$ ,  $c'_y = sc_y$  (với quy ước tâm pixel, còn có một lượng hiệu chỉnh nửa pixel; sai ở đó ít khi chết người nhưng sai ở hệ số  $s$  thì chết ngay).
- **Cắt ảnh từ điểm  $(x_0, y_0)$**  — tiêu cự *không đổi*, chỉ tâm ảnh dịch:  $c'_x = c_x - x_0$ ,  $c'_y = c_y - y_0$ . Đây là chỗ hay bị bỏ sót nhất vì crop trông “vô hại”.
- **Letterbox** — vừa co vừa đệm, nên phải áp cả hai luật trên theo đúng thứ tự.
- **Lật ngang** —  $c'_x = W - 1 - c_x$ , và nếu bài toán có tính trái-phải thì nhãn cũng phải lật theo.

Méo ống kính thì được ước lượng cùng  $K$  trong bước hiệu chuẩn, thường bằng phương pháp bàn cờ của Zhang [32]. Chỉ số sức khoẻ của một lần hiệu chuẩn là *sai số tái chiếu trung bình*; theo kinh nghiệm thực hành, dưới nửa pixel là bình thường với camera phổ thông, miễn bàn cờ được chụp ở đủ nhiều tư thế. Vượt quá một pixel là dấu hiệu có gì đó sai: ảnh mờ, bàn cờ không phẳng, hoặc mô hình méo không đủ bậc. Lưu ý một điều mà lý thuyết ít nói: hiệu chuẩn *trôi*. Nhiệt độ, rung động, và nhất là ống kính có lấy nét tự động đều làm  $K$  thay đổi theo thời gian, nên một bộ tham số hiệu chuẩn từ năm ngoái không đáng tin bằng bạn nghĩ.

## C. Nhiều view: bốn công cụ và điều kiện dùng của chúng

Khi có từ hai ảnh trở lên, thông tin về độ sâu quay lại — nhưng chỉ dưới những điều kiện rất cụ thể. Hình 2.4 minh hoạ ràng buộc cơ bản nhất.



**Hình 2.4.** Ràng buộc epipolar. Một pixel ở ảnh 1 xác định một tia; ảnh của cả tia đó trên ảnh 2 là một *đường thẳng*. Giá trị thực dụng: bài toán tìm điểm tương ứng có từ tìm kiếm hai chiều xuống một chiều — và nếu biết  $K$  của cả hai camera thì ràng buộc này còn cho ra tư thế tương đối giữa hai lần chụp.

Bốn công cụ hay dùng, đặt cạnh nhau trong Bảng 2.3; cột cuối là cột phải thuộc.

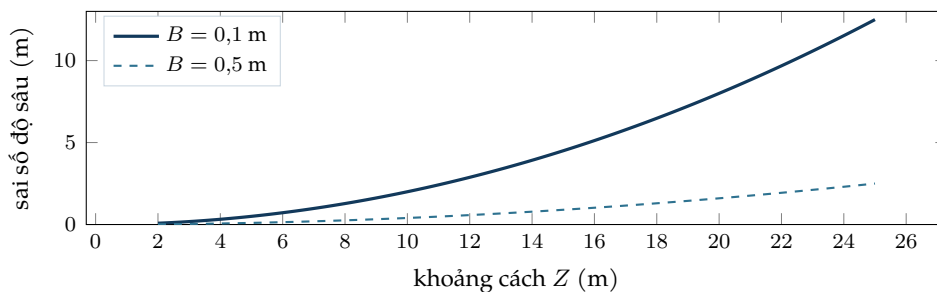
**Bảng 2.3.** Bốn công cụ hình học nhiều view. Biết “hồng khi nào” quan trọng hơn biết công thức, vì các cấu hình suy biến xuất hiện thường xuyên trong dữ liệu thật.

| Công cụ                   | Cơ chế                                              | Giả định                                                   | Hồng khi nào                                                                                                                                        |
|---------------------------|-----------------------------------------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Homography                | Ánh xạ $3 \times 3$ giữa hai ảnh của cùng mặt phẳng | Cảnh là một mặt phẳng, <i>hoặc</i> camera chỉ xoay tại chỗ | Cảnh có độ sâu thật và camera có tịnh tiến — lúc đó không tồn tại homography đúng, và ước lượng cho ra một phép biến đổi “trung bình” sai ở mọi nơi |
| Ma trận cơ bản / bản chất | Ràng buộc epipolar giữa hai view                    | Cảnh cứng; ma trận bản chất còn cần biết $K$               | Camera chỉ xoay không tịnh tiến (tư thế tương đối không xác định được thang đo lẫn hướng tịnh tiến); cảnh phẳng làm bài toán suy biến               |
| Tam giác hoá              | Giao hai tia từ hai tâm quang học                   | Đã biết tư thế tương đối và ghép điểm đúng                 | Đường cơ sở ngắn so với khoảng cách: hai tia gần song song nên giao điểm cực nhạy với nhiễu                                                         |
| PnP                       | Tư thế từ các cặp điểm 3D–2D                        | Đã biết $K$ và có ít nhất ba cặp không thẳng hàng          | Các điểm 3D gần đồng phẳng hoặc gần thẳng hàng; ghép sai điểm (phải bọc trong RANSAC)                                                               |

Cấu hình suy biến của tam giác hoá đáng được định lượng, vì nó quyết định thiết kế phần cứng. Từ  $Z = fB/d$ , đạo hàm cho

$$\left| \frac{\partial Z}{\partial d} \right| = \frac{Z^2}{fB},$$

nghĩa là *sai số độ sâu tăng theo bình phương khoảng cách*. Lấy  $f = 500$  pixel và  $B = 0,1$  m. Một pixel sai lệch thị sai gây sai số 0,5 m ở khoảng cách 5 m. Cũng một pixel ấy gây sai số 2 m ở 10 m, và 8 m ở 20 m. Hình 2.5 vẽ đường cong đó. Muốn đo xa hơn thì phải tăng đường cơ sở hoặc tiêu cự, và cả hai đều kéo theo cái giá riêng: đường cơ sở dài làm vùng nhìn chung hẹp lại và làm bài toán ghép điểm khó hơn, tiêu cự dài làm trường nhìn hẹp lại.



**Hình 2.5.** Sai số độ sâu ứng với một pixel sai lệch thị sai, với  $f = 500$  pixel. Quan hệ bình phương nghĩa là mọi hệ stereo đều có một “tầm dùng được” rất rõ; ngoài tầm đó, con số độ sâu trả về vẫn có nhưng không còn nghĩa. Đây là lý do một hệ stereo đường cơ sở ngắn không dùng được cho vật ở xa dù thuật toán khớp điểm hoàn hảo.

### | Mạch 3 – khôi phục 3D: mỗi bước gỡ một nút thắt và tạo một nút mới

- 1 Vấn đề** một ảnh không cho biết độ sâu.

**Giải pháp** chụp thêm ảnh thứ hai từ vị trí khác và dùng ràng buộc epipolar.

**Giả định** cảnh cứng và có tính tiến giữa hai lần chụp.

**Giá phải trả** mất thang đo tuyệt đối nếu chỉ có một camera; phải ghép điểm được, tức cảnh phải có kết cấu.
- 2 Vấn đề** ghép điểm luôn có một tỉ lệ ghép sai.

**Giải pháp** bọc mọi ước lượng trong RANSAC.

**Giá phải trả** chi phí tăng, và có ngưỡng phải chọn; ngưỡng quá chặt thì loại cả điểm đúng, quá lỏng thì mô hình sai vẫn được chấp nhận.
- 3 Vấn đề** hai view cho kết quả nhiễu và không nhất quán toàn cục.

**Giải pháp** gộp nhiều view lại và tinh chỉnh đồng thời — đúng là **hiệu chỉnh chùm tia** (*bundle adjustment*), và COLMAP [33] là bản cài đặt tham chiếu.

**Giá phải trả** bài toán tối ưu lớn, cần khởi tạo tốt, và vẫn nhập nhằng thang đo nếu không có đường cơ sở thật hay cảm biến khác.
- 4 Vấn đề** cảnh không có kết cấu, hoặc chỉ có một ảnh, hoặc không biết  $K$ .

**Giải pháp** thay ràng buộc hình học bằng prior học từ dữ liệu — độ sâu đơn ảnh [31], hoặc dòng hồi quy trực tiếp bản đồ điểm từ ảnh không cần nội tại như DUST3R [34].

**Giá phải trả** kết quả chỉ đáng tin trong phạm vi phân phối dữ liệu huấn luyện, và mô hình *không* báo cho bạn biết khi nó ra ngoài phạm vi đó — nó vẫn trả về một con số trông hợp lý.

#### D. Giới hạn và trường hợp hỏng

- **Nhập nhằng thang đo là định luật, không phải khuyết điểm.** Một camera đơn không bao giờ tự cho ra mét. Mọi hệ thống trông như làm được điều đó đều đang mượn thông tin từ nơi khác: IMU, bánh xe, chiều cao camera đã biết, hay kích thước điển hình của vật thể học từ dữ liệu.
- **Rolling shutter phá giả định “một ảnh ứng với một tư thế”** vì mỗi hàng pixel chụp ở một thời điểm khác nhau; hiệu ứng này lớn hẳn lên khi camera quay nhanh, và làm cả hiệu chuẩn lẫn tam giác hoá lệch có hệ thống.
- **Undistort và warp là phép lấy mẫu lại**, nên chúng chịu đúng bài toán aliasing ở A/01/04: warp ảnh nhiều lần liên tiếp làm mờ tích lũy; nên gộp các phép biến đổi thành một rồi lấy mẫu một lần.
- **Mô hình học được che giấu cấu trúc suy biến.** Hình học cổ điển thất bại một cách *ôn ào* — ma trận suy biến, RANSAC không tìm đủ inlier. Mô hình học được thất bại một cách *im lặng*: nó vẫn trả về độ sâu cho một bức tường trắng trơn. Với hệ thống an toàn thì đây là khác biệt quan trọng nhất giữa hai cách tiếp cận, và là lý do người ta vẫn giữ một tầng kiểm tra hình học phía sau mô hình.

↔ **đối lập với** E — ước lượng bất định và *conformal prediction* tồn tại để mô hình học được cũng biết “hông ồn ào” như hình học cổ điển

#### CẠM BÃY

Resize ảnh mà quên cập nhật  $K$  là bug im lặng phổ biến nhất khi ghép mạng vào pipeline hình học. Nó không gây exception và không làm loss tăng vọt. Nó chỉ làm mọi phép chiếu lệch một hệ số không đổi. Triệu chứng hiện ra ở tận cuối chuỗi, dưới dạng “quỹ đạo bị co lại” hay “độ sâu sai một hệ số”. Phép kiểm rẻ nhất: chiếu vài điểm 3D đã biết xuống ảnh

đã biến đổi và nhìn bằng mắt xem chúng có rơi đúng chỗ không, trước khi tin bất kỳ con số nào phía sau.

#### CẦN NHỚ

Camera đo tia, không đo điểm. Độ sâu bị xoá ngay lúc chụp, và chỉ mua lại được bằng một điểm nhìn thứ hai hoặc bằng prior. Và  $K$  gắn với hệ toạ độ pixel — mọi resize, crop, letterbox, lật đều phải cập nhật nó. [T1]

### E. Kết nối

Mục này dùng trực tiếp toạ độ đồng nhất và  $SE(3)$  từ A/01/04, và phần lấy mẫu của mục đó giải thích vì sao undistort làm mờ ảnh. Nó là tiền đề của A/02/04, nơi các cảm biến khác được so sánh chính bằng “chúng đo trực tiếp cái gì mà camera phải suy ra”. Theo chiều ngang, đây là nền của toàn bộ nhánh 3D ở C: ước lượng độ sâu, dựng lại cảnh, biểu diễn trường bức xạ [28]. Tất cả các phương pháp đó đều bắt đầu bằng một bước ước lượng tư thế bằng hình học cổ điển. Sách nền đầy đủ: [35], và bản trình bày dễ vào hơn ở [36].

#### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Bạn resize ảnh còn một nửa rồi crop 64 pixel từ mép trái — ma trận nội tại mới bằng bao nhiêu, và bước nào không làm đổi tiêu cự?
2. Vì sao homography chỉ đúng cho cảnh phẳng hoặc camera xoay tại chỗ, và nếu áp nó cho cảnh có độ sâu thì sai lệch hiện ra ở đâu?
3. Với  $f = 500$  pixel và  $B = 0,1$  m, sai số độ sâu ứng với một pixel thì sai ở khoảng cách 20 m là bao nhiêu, và điều đó đặt giới hạn gì lên thiết kế hệ stereo?
4. Camera chỉ xoay tại chỗ giữa hai lần chụp — tam giác hoá hỏng ở chỗ nào, và vì sao đây lại là cấu hình rất hay gặp trong dữ liệu thật?
5. Một mạng ước lượng độ sâu đơn ảnh không vi phạm định luật vật lý nào — vậy thông tin về độ sâu mà nó dùng đến từ đâu, và điều đó dự báo nó hỏng ở loại cảnh nào?
6. Vì sao hình học cổ điển “hỏng ồn ào” còn mô hình học được “hỏng im lặng”, và hệ quả với thiết kế một hệ thống an toàn là gì?

# Nguồn

---

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [29] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [30] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [31] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [32] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [33] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [34] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [37] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [38] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [39] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.

## 2.4 Cảm biến ngoài RGB và nguồn gốc domain gap (Beyond RGB: Sensors and the Origin of Domain Gap)

**PHỤ THUỘC** [A/02-vat-ly-tao-anh/01-anh-sang-va-nhieu](#) (mô hình nhiễu) và [A/02/02-isp-va-khong-gian-mau](#) (chuỗi xử lý) — mục này áp cùng cách phân tích đó cho các cảm biến khác, rồi rút ra kết luận chung về domain gap.

### TÓM TẮT

Mục này làm hai việc. Thứ nhất, so sánh năm loại cảm biến — depth, thermal, event, LiDAR, radar — trên bốn tiêu chí: đo trực tiếp cái gì, giả định gì, mạnh khi nào, hỏng khi nào. Thứ hai, trả lời câu hỏi lớn nhất của cả chương: khoảng cách giữa hiệu năng khi huấn luyện và khi triển khai sinh ra ở đâu. Đọc xong bạn chọn được cảm biến theo ràng buộc thay vì theo thói quen. Bạn biết mỗi loại đòi hỏi thay đổi gì trong kiến trúc và hàm mất mát. Và bạn biết chỗ nào trong pipeline đáng đầu tư công sức để thu hẹp domain gap. Nếu chỉ nhớ một điều: *phần lớn domain gap trong thị giác sinh ra ở khâu tạo ảnh, không ở kiến trúc*. Đổi backbone vì thế hiếm khi chữa được thứ mà một thay đổi ở khâu thu thập hoặc tiền xử lý chữa được ngay.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **domain gap (dịch chuyển miền)** — hiện tượng phân phối dữ liệu lúc triển khai khác phân phối lúc huấn luyện, khiến mô hình tụt hiệu năng dù không có gì trong mô hình thay đổi. “Miền” ở đây nghĩa là một điều kiện thu thập: một hãng camera, một thời điểm trong ngày, một bệnh viện.
- **inductive bias (thiên lệch quy nạp)** — những giả định được cài sẵn vào mô hình hoặc vào dữ liệu, cho phép nó tổng quát hoá từ số mẫu hữu hạn. Ví dụ: *convolution* cài sẵn giả định cấu trúc ảnh mang tính cục bộ và không đổi khi dịch chuyển.
- **camera độ sâu (depth camera)** — cảm biến trả về khoảng cách tới bề mặt tại mỗi pixel, bằng một trong ba cơ chế: chiếu hoa văn hồng ngoại rồi đo biến dạng (structured light), đo thời gian ánh sáng đi và về (time-of-flight), hoặc dùng hai camera và thị sai (stereo).
- **camera nhiệt (thermal)** — cảm biến đo bức xạ hồng ngoại mà vật tự phát ra, phụ thuộc nhiệt độ và độ phát xạ của vật liệu; nó không cần nguồn sáng ngoài nên hoạt động trong bóng tối hoàn toàn.
- **event camera** — cảm biến mà mỗi pixel báo cáo độc lập và bất đồng bộ mỗi khi độ sáng log tại pixel đó đổi quá một ngưỡng; đầu ra là một dòng sự kiện thưa theo thời gian, không phải khung hình.
- **LiDAR** — cảm biến phát tia laser và đo thời gian phản hồi để lấy khoảng cách trực tiếp; đầu ra là một đám mây điểm 3D thưa, không phụ thuộc ánh sáng môi trường.
- **radar** — cảm biến dùng sóng vô tuyến; nó đo được khoảng cách và, nhờ hiệu ứng Doppler, đo trực tiếp cả vận tốc xuyên tâm của vật thể trong một lần đo.
- **độ phân giải góc** — khả năng phân biệt hai vật nằm sát nhau theo hướng nhìn. Cảm biến có độ phân giải góc thấp thì hai vật gần nhau ở xa sẽ trộn thành một.

### A. Đặt vấn đề

Hai câu hỏi tưởng như tách rời nhưng có chung một câu trả lời. Câu thứ nhất: khi bài toán không giải được bằng camera RGB — trời tối, sương mù, vật thể trong suốt, cần vận tốc tức thời — thì đổi sang cảm biến nào, và đổi kiến trúc thế nào cho khớp? Câu thứ hai: vì sao một mô hình đạt 95% khi thử nghiệm lại chỉ còn 80% khi triển khai, dù dữ liệu “vẫn là ảnh của cùng loại vật thể”?

Câu trả lời chung là: *mỗi cảm biến kèm theo một tập giả định riêng về thế giới, và hiệu năng sập đúng lúc một giả định bị vi phạm*. Với câu hỏi thứ nhất, việc chọn cảm biến chính là việc chọn tập giả định nào chấp nhận được. Với câu hỏi thứ hai, khoảng cách train–production chính là khoảng cách giữa hai tập giả định: tập lúc thu thập dữ liệu và tập lúc triển khai. Cả hai tập ấy được ẩn định ở khâu tạo ảnh, trước khi kiến trúc kịp ý kiến.

**Học xong mục này thì làm được gì mà trước đó không làm được?** Chọn cảm biến dựa trên ràng buộc và chế độ hồng thay vì theo thói quen; sửa lại pipeline dữ liệu khi phải xử lý depth, thermal hay event thay vì nhồi chúng vào một mạng RGB không đổi; và khi gặp một cú tụt hiệu năng lúc triển khai, biết tìm nguyên nhân ở đúng chỗ thay vì thử đổi mô hình.

### TRỰC GIÁC

Mỗi cảm biến là một loại nhân chứng với một kiểu mù riêng. Camera RGB là nhân chứng nhìn rất tinh nhưng chỉ làm việc khi có đèn, và kể lại màu sắc theo cách đã được biên tập. LiDAR là nhân chứng đo khoảng cách chính xác nhưng nhìn thế giới qua vài chục đường kẻ ngang và không phân biệt nổi chữ trên biển báo. Radar nhìn xuyên sương và nói ngay được vật đang lao tới nhanh cỡ nào, nhưng không phân biệt được cái vỏ lon với cái nắp cống. Ghép nhiều nhân chứng lại không cho ra một nhân chứng hoàn hảo. Nó cho ra một bài toán mới: tin ai khi hai lời khai mâu thuẫn. Đó mới là phần khó thật của hợp nhất cảm biến.

## B. Năm cảm biến, cùng một khung phân tích

**Bảng 2.4.** Năm loại cảm biến ngoài RGB. Cột “đo trực tiếp cái gì” quyết định inductive bias mà cảm biến mang lại; cột cuối quyết định bạn có dùng được nó không.

| Cảm biến                                | Đo trực tiếp cái gì                              | Mạnh khi nào                                                                    | Hông khi nào                                                                                                                                                                                                   |
|-----------------------------------------|--------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Depth (ToF / structured light / stereo) | Khoảng cách tới bề mặt tại mỗi pixel             | Trong nhà, tầm gần, cần hình học tuyệt đối mà không cần tam giác hoá            | Bề mặt bóng, trong suốt, hoặc rất tối (không phản xạ đủ); ánh sáng lắt lắt nguồn hồng ngoại; ToF bị nhiễu đa đường ở góc tường. Pixel thiếu dữ liệu <i>không</i> phân bố ngẫu nhiên mà tương quan với vật liệu |
| Thermal                                 | Bức xạ hồng ngoại vật tự phát                    | Bóng tối hoàn toàn, khói, phát hiện người và động cơ                            | Không có kết cấu bề mặt nên nhận dạng chi tiết rất khó; “thời điểm cân bằng nhiệt” lúc bình minh và hoàng hôn làm tương phản biến mất; bù đồng đều cảm biến làm ảnh đứng hình chớp nhoáng theo chu kỳ          |
| Event                                   | Thay đổi độ sáng log tại từng pixel, bất đồng bộ | Chuyển động rất nhanh, dải động cực rộng, độ trễ micro giây, tiêu thụ điện thấp | Cảnh tĩnh gần như không sinh dữ liệu; không có cường độ tuyệt đối; nhiễu nền tăng vọt ở mức sáng thấp; mọi kiến trúc ảnh thông thường phải đổi vì đầu vào là dòng sự kiện thưa                                 |
| LiDAR                                   | Khoảng cách theo tia laser                       | Hình học 3D chính xác, không phụ thuộc ánh sáng, tầm trung và xa                | Mưa, sương, bụi làm tán xạ; bề mặt phản xạ gương; mật độ điểm giảm theo bình phương khoảng cách nên vật ở xa chỉ còn vài điểm; đất và cỏ bộ phận chuyển động                                                   |
| Radar                                   | Khoảng cách và vận tốc xuyên tâm                 | Thời tiết xấu, tầm xa, cần vận tốc ngay lập tức                                 | Độ phân giải góc rất thấp nên hai vật gần nhau trộn làm một; phản xạ nhiều đường tạo mục tiêu ma; vật kim loại nhỏ có thể sáng hơn cả một chiếc xe                                                             |

Hai hệ quả cho phía mô hình, và cả hai đều bị bỏ qua thường xuyên:

- **Dữ liệu thiếu không phải nhiễu ngẫu nhiên.** Với camera độ sâu, các pixel không có giá trị tập trung ở kính, ở bề mặt bóng, ở mép vật thể — tức là tương quan mạnh với chính thứ ta muốn dự đoán. Cho mạng nhận thêm một kênh mặt nạ “chỗ nào thiếu” nghe có vẻ vô hại, nhưng nó *rò rỉ nhãn*: mô hình có thể học “chỗ nào thiếu độ sâu thì chỗ đó là cửa kính” và đạt điểm cao mà không hiểu gì về cửa kính. Đây là biến thể của cùng một cạm bẫy sẽ gặp lại ở B dưới tên *shortcut learning*.

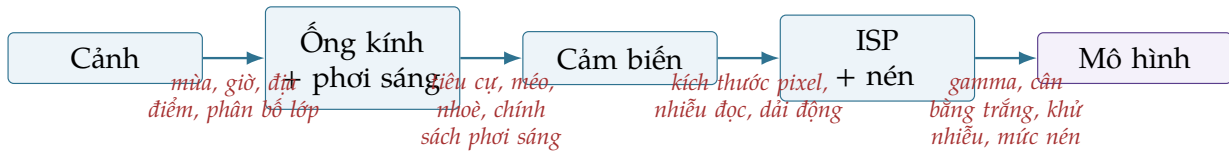
↔ **trường hợp riêng của B** — mô hình bám vào một dấu hiệu tương quan với nhãn thay vì cơ chế thật — ở đây dấu hiệu đó là chính cái mặt nạ dữ liệu thiếu

- **Cấu trúc dữ liệu quyết định kiến trúc.** Đám mây điểm LiDAR không có lưới đều nên convolution hai chiều không áp thẳng được; hoặc phải chiếu về một lưới (mất thông tin, sinh aliasing — đúng bài toán ở [A/01/04](#)), hoặc phải dùng kiến trúc làm việc trực tiếp trên tập điểm. Dòng sự kiện của event camera còn xa lưới hơn nữa. Việc “chỉ đổi cảm biến, giữ nguyên mạng” hầu như không bao giờ là lựa chọn đúng.

↔ **ràng buộc bởi C** — cấu trúc dữ liệu quyết định kiến trúc dùng được: lưới đều cho convolution, tập điểm cho kiến trúc trên tập, dòng sự kiện cho biểu diễn thời gian

### C. Domain gap sinh ra ở đâu

Đây là luận điểm trung tâm của cả chương A/02. Hình 2.6 đặt các nguồn dịch chuyển vào đúng chỗ chúng sinh ra dọc theo đường đi của tín hiệu.



**Hình 2.6.** Bốn nhóm nguồn dịch chuyển, và cả bốn đều nằm *trước* mô hình. Đối kiến trúc chỉ tác động vào khối cuối cùng, nên nó là công cụ yếu để chữa một khoảng cách sinh ra ở ba khối đầu. Ngược lại, ghi lại và kiểm soát bốn nhóm này lúc thu thập dữ liệu là công cụ mạnh — và rẻ hơn nhiều.

Luận điểm “phần lớn domain gap sinh ở khâu tạo ảnh” có bằng chứng ủng hộ, nhưng cần phát biểu cho đúng mức. Bằng chứng gồm: các bộ benchmark đo độ bền trước các biến dạng ảnh cho thấy độ chính xác tụt rất mạnh trước những biến dạng vốn là mô phỏng thô của hiệu ứng cảm biến và nén [37]; việc dựng lại một tập test theo đúng quy trình gốc vẫn làm mọi mô hình tụt hàng điểm [18]; và các tập dữ liệu được thiết kế riêng để đo dịch chuyển miền cho thấy khoảng cách giữa các miền lớn hơn nhiều so với khoảng cách giữa các kiến trúc [38]. Điều *chưa* được chứng minh là một phát biểu định lượng phổ quát, kiểu “ $x$  phần trăm khoảng cách đến từ khâu tạo ảnh”. Con số đó phụ thuộc bài toán. Ai nêu một tỉ lệ cụ thể cho mọi trường hợp là đang nói quá. Hãy giữ nó ở dạng *chiều hướng đã được kiểm chứng nhiều lần*, không phải một hằng số.

### I MẠCH 4 — THU HẸP DOMAIN GAP, THEO THỨ TỰ CHI PHÍ TĂNG DẦN

- 1 Vấn đề** mô hình tụt điểm khi triển khai, chưa biết vì sao.

**Giải pháp** trước hết *đo* chứ đừng sửa: giữ một tập nhỏ dữ liệu thu ở chính điều kiện triển khai và tính độ chính xác trên đó, tách theo từng điều kiện (thiết bị, giờ trong ngày, địa điểm).

**Hệ quả** bạn biết khoảng cách nằm ở nhóm điều kiện nào trước khi tiêu một giờ nào cho mô hình.
- 2 Vấn đề** khoảng cách đến từ khác biệt về cảm biến và ISP.

**Giải pháp** làm cho tiền xử lý *giống nhau* ở hai đầu — cùng cách resize, cùng mức nén, cùng chính sách phơi sáng — và mô phỏng phần còn lại bằng augmentation có cơ sở vật lý ở miền tuyến tính.

**Giá phải trả** tốn công dựng pipeline gỡ xử lý; đổi lại là cách rẻ nhất trong các cách thật sự hiệu quả.
- 3 Vấn đề** vẫn còn khoảng cách vì cảnh khác hẳn.

**Giải pháp** thu thêm dữ liệu ở đúng miền đích, dù chỉ vài trăm mẫu có nhãn để tinh chỉnh.

**Giả định** bạn tiếp cận được miền đích — không phải lúc nào cũng đúng.

**Hệ quả** theo kinh nghiệm thực hành, một lượng nhỏ dữ liệu đúng miền thường có giá trị hơn một lượng lớn dữ liệu sai miền.
- 4 Vấn đề** không lấy được nhãn ở miền đích.

**Giải pháp** các phương pháp thích nghi miền không giám sát, hoặc huấn luyện trên nhiều miền để mô hình học phần bất biến.

**Giá phải trả** đây là nhóm phương pháp có kết quả *không ổn định* nhất: nhiều so sánh có kiểm soát cho thấy chúng khó vượt được một *baseline* được tune từ tế [39]. Hãy coi đây là lựa chọn cuối, không phải lựa chọn đầu.

## D. Giới hạn và trường hợp hỏng

- **Hợp nhất cảm biến không phải phép cộng.** Ghép LiDAR với camera đòi hỏi hiệu chỉnh ngoại chính xác và đồng bộ thời gian chính xác; sai lệch một phần trăm giây ở tốc độ xe làm điểm 3D lệch cả mét. Khi hai cảm biến mâu thuẫn, hệ thống phải có quy tắc quyết định — và quy tắc đó là một thiết kế, không phải một thứ mạng tự học được từ dữ liệu bình thường.
- **Một cảm biến “tốt hơn” vẫn có thể làm hệ thống tệ đi** nếu chế độ hỏng của nó tương quan với chế độ hỏng của cảm biến kia. Camera và LiDAR cùng hỏng trong sương mù dày; thêm LiDAR không mua được sự độc lập mà bạn tưởng.
- **Không phải mọi khoảng cách đều thu hẹp được.** Nếu thông tin đã bị xoá ở khâu thu thập — vùng chói sáng, chi tiết màu bị giảm mẫu, vật thể chỉ còn ba pixel — thì không phương pháp thích nghi miền nào lấy lại được. Phân biệt “mô hình chưa học được” với “thông tin không còn ở đó” là bước chẩn đoán đầu tiên và hay bị bỏ qua nhất.

### CẠM BÃY

Đừng chữa một vấn đề của khâu tạo ảnh bằng một thay đổi ở kiến trúc. Đây là phản xạ tự nhiên vì kiến trúc là thứ ta kiểm soát trực tiếp và thay đổi nhanh, còn đi thu thập lại dữ liệu thì chậm và chán. Nhưng giả sử tập huấn luyện chụp bằng một hãng máy còn triển khai bằng hãng khác. Khi đó mọi giờ dành cho việc đổi backbone đều tiêu vào sai chỗ. Tệ hơn, các thí nghiệm ấy vẫn cho những khác biệt nhỏ ngẫu nhiên, đủ để bạn tin rằng mình đang tiến bộ.

### CẦN NHỚ

Mỗi cảm biến là một tập giả định về thế giới. Hệ thống sập đúng lúc một giả định bị vi phạm. Vì vậy hãy chọn cảm biến theo *chế độ hỏng*, không theo thông số đẹp. Và khi hiệu năng tụt lúc triển khai, hãy đi ngược đường đi của tín hiệu từ cảnh tới mô hình trước khi chạm vào mô hình. [T1]

## E. Kết nối

Mục này khép lại chương A/02 bằng cách tổng quát hoá hai mục đầu: mô hình nhiễu ở A/02/01 và chuỗi xử lý ở A/02/02 là trường hợp riêng cho cảm biến RGB của cùng một khung phân tích.

↔ **tiền đề** E — đánh giá theo nhóm điều kiện — thiết bị, giờ trong ngày, địa điểm — thay cho một con số trung bình duy nhất, là hệ quả trực tiếp của luận điểm này

Nó nổi sang ba nhánh. Sang B, nơi domain gap được nhìn như một dạng ràng buộc thiết kế và *shortcut learning* được mổ xẻ. Sang D ở phần augmentation, nơi ta phân biệt augmentation mô phỏng miền với augmentation chính quy hoá. Và sang E, nơi đánh giá tách theo nhóm điều kiện trở thành kỷ luật bắt buộc, thay cho một con số trung bình duy nhất.

### Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao các pixel thiếu dữ liệu của một camera độ sâu không được coi là nhiễu ngẫu nhiên, và điều đó tạo ra dạng rò rỉ nào khi bạn thêm một kênh mặt nạ?
2. Event camera mạnh ở dải động và độ trễ — vậy nó mất gì so với camera thường, và loại bài toán nào nó gần như vô dụng?
3. Vì sao thêm LiDAR vào một hệ camera không nhất thiết làm hệ bền hơn trong sương mù?
4. Luận điểm “phần lớn domain gap sinh ở khâu tạo ảnh” được ủng hộ bởi bằng chứng nào, và phát biểu nào của nó thì *chưa* được chứng minh?

5. Bạn phát hiện mô hình tụt 15 điểm khi triển khai — bước đầu tiên nên làm là gì, và vì sao nó không phải là đổi kiến trúc?
6. Làm sao phân biệt “mô hình chưa học được” với “thông tin không còn trong dữ liệu”, và vì sao phân biệt đó quyết định toàn bộ hướng xử lý tiếp theo?

## Nguồn

---

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [29] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [30] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [31] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [32] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [33] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [34] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [37] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [38] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [39] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.

## 2.5 Thực hành

Chương này khác các chương khác ở chỗ phần lớn nội dung chỉ thật sự vào đầu khi bạn cầm một cái camera thật. Hai bài dưới đây được thiết kế để mỗi bài cho ra một *con số* mà bạn tự đo được, chứ không phải một cảm giác.

**Repo và SOTA.** [opencv](#) cho hiệu chuẩn và undistort; [kornia](#) cho các phép thị giác khả vi (hữu ích khi cần đưa một phép biến đổi hình học vào trong đồ thị tính toán); [rawpy](#) cùng LibRaw để đọc ảnh RAW; [colour-science](#) cho chuyển đổi không gian màu đúng chuẩn; [COLMAP](#) [33] làm bản dựng lại 3D tham chiếu. Về việc gỡ ảnh sRGB ngược về RAW để mô phỏng nhiễu thật, xem dòng *Unprocessing Images for Learned Raw Denoising* [29] và các bản cài đặt lại; về dữ liệu nhiễu thật có cặp ảnh sạch, xem SIDD [30]. Sách nền: Szeliski [36] bản hai miễn phí, và [35] cho phần hình học nhiều view. *Cảnh báo bán rã*: danh sách này chốt tại tháng 9/2026 và tên các mô hình dẫn đầu bảng sẽ cũ trong 6–12 tháng, trong khi các repo hạ tầng như OpenCV và COLMAP thì bền hơn nhiều.

**Câu hỏi kiểm tra hiểu.** Ba câu, mỗi câu chạm một mục:

- Vì sao augmentation “tăng độ sáng” trên ảnh sRGB không tương đương tăng phơi sáng thật — sai ở hệ số bao nhiêu, và sai thế nào về hành vi của nhiễu?
- Nếu train trên JPEG chất lượng 90 và triển khai trên luồng RAW, hỏng ở đâu trước và vì sao?
- Nhiễu shot phụ thuộc tín hiệu — điều đó phá vỡ giả định nào của mô hình nhiễu Gauss, và hệ quả với việc chọn hàm mất mát cho bài khử nhiễu là gì?

**Active recall** (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                                            | Trả lời                                                                                                                                                                                                       |
|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ••• Shot noise                                                 | Dao động không tránh được của một phép đếm photon: phương sai bằng chính kỳ vọng. Hệ quả là tỉ số tín hiệu trên nhiễu chỉ tăng theo $\sqrt{N}$ , nên vùng tối luôn là chỗ mô hình sai trước.                  |
| ••• Mô hình nhiễu affine                                       | $\text{Var}(I) = aI + b$ : phần tỉ lệ với cường độ là nhiễu photon, phần hằng số là nhiễu mạch đọc. Fit được từ vài chục ảnh cảnh tĩnh, nhưng chỉ đúng ở miền tuyến tính.                                     |
| •• Dải động                                                    | Tỉ số giữa mức bão hoà của giếng điện tích và sàn nhiễu đọc. Vượt trần thì cháy sáng và mất hẳn thông tin; dưới sàn thì chìm tối nhưng còn cứu được phần nào.                                                 |
| •• Vì sao tăng ISO không thêm thông tin?                       | Vì nó khuếch đại tín hiệu <i>đã đo</i> chứ không thu thêm photon, nên nhân cả tín hiệu lẫn nhiễu. Chỉ ở mức rất tối, nơi nhiễu mạch đọc chi phối, nó mới cải thiện được phần nào.                             |
| ••• ISP                                                        | Chuỗi xử lý trong máy ảnh biến số đo thô của cảm biến thành ảnh xem được: khử khảm, cân bằng trắng, gamma, ánh xạ tone, nén. Nó phi tuyến, phụ thuộc nội dung, và không khôi phục ngược chính xác được.       |
| ••• Gamma                                                      | Phép nén dải sáng bằng hàm mũ khoảng $1/2,2$ , để mã số phân bố đều theo cảm nhận của mắt. Đây là chỗ ảnh mất tính tuyến tính, nên mọi tính toán vật lý phải dừng lại trước bước này.                         |
| •• Khử khảm                                                    | Bước nội suy hai kênh màu còn thiếu tại mỗi pixel, vì cảm biến chỉ đo được một màu mỗi ô. Nó làm nhiễu có tương quan cả về không gian lẫn giữa các kênh, phá giả định nhiễu độc lập từng pixel.               |
| ••• Vì sao augmentation độ sáng phải làm ở miền tuyến tính?    | Vì gamma bẻ cong thang sáng: gấp đôi phơi sáng chỉ làm giá trị sRGB tăng khoảng 1,37 lần. Nhân thẳng giá trị pixel còn sai cả về nhiễu, vì SNR không đổi trong khi lẽ ra phải tăng $\sqrt{2}$ .               |
| •• Train trên JPEG, chạy trên luồng ít nén — hỏng ở đâu trước? | Ở các bài toán mức thấp như phân đoạn biên hay đếm vật nhỏ, vì mô hình đã quen với vùng phẳng bị lượng tử hoá làm sạch. Phòng vệ rẻ nhất là đưa mức nén thành một tham số augmentation.                       |
| ••• Mô hình pinhole                                            | Toạ độ pixel chỉ phụ thuộc tỉ số giữa toạ độ ngang và độ sâu. Vì thế camera đo tia chứ không đo điểm: độ sâu và thang đo tuyệt đối bị xoá ngay lúc chụp.                                                      |
| ••• Ma trận nội tại                                            | Ma trận $K$ cỡ $3 \times 3$ , chứa tiêu cự tính bằng pixel và toạ độ tâm ảnh; nó biến một tia thành toạ độ pixel. Nó gắn với hệ toạ độ ảnh, nên resize nhân mọi phần tử với hệ số, còn crop chỉ dịch tâm ảnh. |
| •• Ràng buộc epipolar                                          | Một pixel ở ảnh này ứng với cả một đường thẳng ở ảnh kia. Giá trị thực dụng: bài tìm điểm tương ứng co từ tìm kiếm hai chiều xuống một chiều.                                                                 |
| •• Homography                                                  | Ánh xạ $3 \times 3$ giữa hai ảnh của cùng một mặt phẳng. Nó chỉ đúng khi cảnh phẳng hoặc camera xoay tại chỗ; áp cho cảnh có độ sâu thật thì cho ra một phép biến đổi sai ở mọi nơi.                          |
| •• Quỹ đạo bị co lại một hệ số không đổi — nguyên nhân?        | Gần như luôn là quên cập nhật $K$ sau khi resize hoặc crop ảnh. Phân biệt với lỗi thuật toán: chiếu vài điểm 3D đã biết xuống ảnh đã biến đổi và nhìn xem chúng có rơi đúng chỗ không.                        |

| Thuật ngữ / câu hỏi                                            | Trả lời (tiếp)                                                                                                                                                                       |
|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| •• Vì sao một hệ stereo có tầm dùng được rất rõ?               | Vì sai số độ sâu bằng $Z^2/(fB)$ trên mỗi pixel sai lệch thị sai, tức tăng theo bình phương khoảng cách. Với $f = 500$ và $B = 0,1$ m, một pixel là 0,5 m ở 5 m nhưng 8 m ở 20 m.    |
| •• Event camera                                                | Cảm biến mà mỗi pixel báo độc lập mỗi khi độ sáng log đổi quá một ngưỡng. Mạnh ở dải động và độ trễ micro giây, nhưng không cho cường độ tuyệt đối và gần như cảm trước cảnh tĩnh.   |
| •• Pixel độ sâu bị thiếu tập trung ở kính và mép vật — hệ quả? | Dữ liệu thiếu không ngẫu nhiên mà tương quan với chính thứ cần dự đoán. Cho mạng thêm kênh mặt nạ thiếu là rò rỉ nhãn: nó học “chỗ nào thiếu thì đó là kính”.                        |
| ••• Domain gap                                                 | Khoảng cách giữa phân phối dữ liệu lúc huấn luyện và lúc triển khai. Phần lớn nó sinh ra ở khâu tạo ảnh — cảnh, quang học, cảm biến, ISP — tức trước khi kiến trúc kịp có ý kiến.    |
| •• Vì sao đổi backbone hiếm khi chứa được domain gap?          | Vì kiến trúc là khối cuối cùng của chuỗi, còn nguyên nhân nằm ở bốn khối trước nó. Các thí nghiệm đổi kiến trúc vẫn cho khác biệt nhỏ ngẫu nhiên, đủ để bạn tưởng mình đang tiến bộ. |
| • Vì sao thêm LiDAR chưa chắc làm hệ bền hơn?                  | Vì chế độ hồng có thể tương quan: sương mù dày làm hồng cả camera lẫn LiDAR. Hợp nhất cảm biến chỉ mua được sự độc lập khi hai cảm biến hồng vì hai lý do khác nhau.                 |

••• phải thuộc •• nên nắm • biết là có

**I Kiến thức này đang sống ở đâu** (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

| Công trình / hệ thống                      | Dùng ý tưởng nào của chương                                                                        | Ghi chú                                                       |
|--------------------------------------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| OpenCV, COLMAP                             | Mô hình pinhole, hiệu chuẩn bàn cờ và hình học nhiều view gần như nguyên vẹn như trình bày ở mục 3 | Hạ tầng; bền hơn nhiều so với tên mô hình dẫn đầu bảng        |
| Dòng xử nhiều RAW kiểu <i>Unprocessing</i> | Gỡ ISP về miền tuyến tính rồi mô phỏng nhiều shot và read, đúng mạch của mục 1 và 2                | Kỹ thuật chuẩn để sinh dữ liệu huấn luyện xử nhiều            |
| Pipeline HDR trên điện thoại               | Ghép nhiều mức phơi sáng để vượt dải động cảm biến — kèm cái giá là bóng ma chuyển động            | Sản phẩm thật, nhiều hãng; chi tiết cài đặt thì không công bố |
| MiDaS và họ độ sâu đơn ảnh                 | Dự đoán độ sâu tương đối chứ không tuyệt đối, vì phép chiếu đã xóa thang đo                        | Thường được dùng làm prior cho các pipeline dựng cảnh         |
| Benchmark robustness kiểu ImageNet-C       | Mô phỏng thô các biến dạng sinh ra ở khâu tạo ảnh, để đo domain gap                                | Chỉ là xấp xỉ của hiệu ứng thật; đọc điểm số thận trọng       |

**MINI PROJECT SLAM** • Từ mô hình nhiều của một camera thật tới ngưỡng gây tracking • một tới hai ngày

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mục tiêu</b> | nối thẳng vật lý cảm biến với hiệu năng SLAM: đo mô hình nhiều của một camera thật, áp chính mô hình đó lên một chuỗi EuRoC, rồi tìm mức nhiễu mà tracking bắt đầu gây — thay vì chỉ nói “ảnh tối thì khó hơn”.                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Dữ liệu</b>  | (i) khoảng ba mươi ảnh chụp cùng một cảnh tĩnh trên chân máy ở ISO 100, 800 và 6400, chụp RAW nếu máy cho phép; (ii) hai chuỗi EuRoC có chân trị ở hai mức khó, ví dụ một chuỗi dễ và một chuỗi có đoạn ảnh mờ.                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Các bước</b> | (1) Chọn vài chục vùng phẳng ở các mức sáng khác nhau, đo phương sai theo cường độ và fit $\text{Var} = aI + b$ cho từng mức ISO; bạn sẽ thấy cả $a$ lẫn $b$ tăng theo ISO, và thấy đường fit trên ảnh JPEG không thẳng như trên RAW. (2) Hiệu chuẩn camera bằng bàn cờ, báo cáo sai số tái chiếu, rồi thử bỏ bớt tư thế bàn cờ để thấy hiệu chuẩn cần <i>đa dạng tư thế</i> chứ không cần nhiều ảnh. (3) Áp mô hình nhiễu đã fit lên ảnh EuRoC ở bốn mức tương đương ISO. Làm ở miền tuyến tính: gỡ gamma xấp xỉ trước, áp lại sau. Ghi rõ trong báo cáo rằng đây là <i>xấp xỉ</i> , vì ảnh EuRoC đã qua xử lý nên không phải RAW thật. |

- (4) Chạy hệ SLAM ba lần cho mỗi mức, ghi số đặc trưng bám được trên mỗi ảnh và ATE bằng *evo*. (5) Vẽ cả hai đại lượng theo mức nhiễu.
- Xong khi** bạn có một hình duy nhất, trục hoành là mức nhiễu (hoặc ISO tương đương), hai trục tung là số đặc trưng bám được và ATE; trên đó bạn chỉ ra mức nhiễu mà số đặc trưng tụt xuống dưới ngưỡng buộc hệ khởi tạo lại, và ATE nhảy bậc ngay tại đó. Vẽ kèm độ lệch chuẩn của ba lần chạy, để chứng minh bậc nhảy lớn hơn nhiễu của chính harness.
- Nối với** *A/02/01* cho mô hình nhiễu affine và quan hệ  $\text{SNR} = \sqrt{N}$ ; *A/02/02* cho lý do bắt buộc phải làm ở miền tuyến tính; *A/01/03* để trả lời ba lần chạy đã đủ chưa. Đường số điều kiện theo thời gian từ mini project chương 1 dùng được ngay làm biến giải thích cho các đoạn gãy.

### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1.** Vì gamma. Ánh sáng thật  $L \rightarrow 2L$  chỉ làm giá trị sRGB tăng  $2^{1/2.2} \approx 1.37$  lần chứ không phải hai lần. Nặng hơn là hành vi nhiễu: phơi sáng thật gấp đôi làm SNR tăng  $\sqrt{2}$ , còn nhân giá trị pixel thì SNR không đổi.
- A2.** Vì ISO khuếch đại tín hiệu *đã đo* chứ không thu thêm photon, nên nó nhân cả tín hiệu lẫn nhiễu với cùng một hệ số. Chỉ ở vùng cực tối, nơi read noise chi phối, nó mới cải thiện được phần nào.
- A3.** Ma trận nội tại. Tiêu cự và tâm ảnh đều tính bằng pixel nên phải nhân theo hệ số resize; quên thì mọi phép chiếu lệch một hệ số không đổi, và triệu chứng chỉ hiện ra ở cuối chuỗi dưới dạng “quỹ đạo co lại” hay “độ sâu sai một hệ số”.
- A4.** Phần lớn mất ở khâu tạo ảnh — cảnh, quang học và chính sách phơi sáng, cảm biến, ISP và nén — tức là ở bốn trạm nằm trước mô hình. Bằng chứng ủng hộ chiều hướng này khá vững, dù không có một tỉ lệ phổ quát cho mọi bài toán.

### NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ tới cái nhiệt kế điện tử treo trong phòng bạn. Bạn muốn biết phòng nóng bao nhiêu, nhưng thứ đọc được là con số trên màn hình — và giữa hai thứ đó có cả một chuỗi: cảm biến nằm ở chỗ nào, vỏ máy tự nóng lên bao nhiêu, rồi một chương trình nhỏ làm mượt số đo và làm tròn trước khi hiện lên.

Máy ảnh y hệt vậy, chỉ dài hơn. Nó đếm hạt sáng rơi vào từng ô nhỏ; chỗ tối đếm được ít nên con số nhảy nhót, còn chỗ quá sáng thì tràn và mọi khác biệt bên trong bị xoá sạch, không cách nào lấy lại. Rồi máy tự kéo sáng cho dễ nhìn, bôi mờ những chi tiết nó đoán là không ai để ý, và nén lại cho nhẹ. Cái đến tay bạn không phải một phép đo, mà là một bản đã được biên tập.

Mẹo của chương: muốn làm bất cứ phép tính nào mang ý nghĩa vật lý — giả lập trời tối hơn, thêm hạt nhiễu — thì phải gỡ ngược bản biên tập về gần số đo gốc, làm ở đó, rồi dựng lại.

Cái giá: gỡ ngược chỉ là gần đúng, phần đã bị xoá thì mất hẳn, và mỗi hãng máy biên tập một kiểu.

**Chỗ không rút gọn được:** câu “phần lớn khoảng cách giữa phòng thí nghiệm và thực tế sinh ra ở khâu chụp ảnh” nghe gọn nhưng không quy được thành một tỉ lệ chung: nó đổi theo từng bài toán, và ai đưa ra một con số phần trăm cho mọi trường hợp là đang nói quá.

**Một câu:** Ảnh không phải một phép đo mà là một bản đã biên tập, nên phần lớn khoảng cách giữa thí nghiệm và thực tế đã được quyết định xong trước khi mô hình kịp nhìn thấy gì.

## 2.6 Kết nối

Chương này khép lại Phần A và mở ra ba hướng. Hướng gần nhất là Phần B. Ở đó, các ràng buộc vật lý vừa nêu được nhìn lại như *ràng buộc thiết kế*: dải động của cảm biến, độ trễ do thời gian phơi sáng, và khác biệt giữa các hãng camera. Chúng đứng cạnh ràng buộc về nhân và về tính toán. Và chính chúng loại bỏ phần lớn phương án, trước khi độ chính xác kịp lên tiếng. Hướng

thứ hai là C, nơi hình học nhiều view của mục 3 trở thành bước khởi tạo bắt buộc cho gần như mọi phương pháp dựng cảnh học được. Hướng thứ ba là E: luận điểm domain gap ở mục 4 biến thành một kỷ luật đo đạc cụ thể — tách kết quả theo nhóm điều kiện thay vì báo một con số trung bình.

## Tổng kết chương

Chương này đã đi hết đường đi của tín hiệu và, ở mỗi trạm, chỉ ra một giả định bị phá vỡ. Ở cảm biến, phép đếm photon làm nhiều phụ thuộc tín hiệu. Tỉ số tín hiệu trên nhiễu vì thế chỉ tăng theo căn của lượng ánh sáng. Mọi kỹ thuật “làm sáng ảnh” chỉ khuếch đại, không thêm thông tin. Ở ISP, mỗi tầng phá một giả định khác nhau. Gamma phá tính tuyến tính. Khử nhiễu phá tính độc lập của nhiễu giữa các pixel. Ánh xạ tone cục bộ phá giả định độ sáng không đổi, còn nén JPEG xoá tần số cao. Vì vậy mọi phép mô phỏng vật lý phải làm ở miền tuyến tính, rồi mới dựng ảnh lại. Ở camera, phép chiếu xoá độ sâu, và  $K$  gắn với hệ toạ độ pixel. Mỗi phép biến đổi ảnh vì thế đòi một phép cập nhật  $K$  tương ứng. Quên thì không có lỗi nào được báo. Ở các cảm biến khác, mỗi loại mang một tập giả định riêng và một kiểu mù riêng, nên hợp nhất cảm biến là bài toán quyết định tin ai chứ không phải phép cộng.

Điểm gộp của cả bốn mục là luận điểm về domain gap: khoảng cách giữa phòng thí nghiệm và thực tế được ẩn định phần lớn ở những trạm nói trên, tức là *trước* khi kiến trúc kịp có ý kiến. Luận điểm này có bằng chứng ủng hộ vững về chiều hướng, dù không có con số phổ quát; và hệ quả thực hành của nó rất cụ thể — khi hiệu năng tụt lúc triển khai, hãy đi ngược đường đi của tín hiệu trước khi chạm vào mô hình.

Còn bỏ ngỏ ba nhánh, và bỏ ngỏ có chủ ý. Thứ nhất là quang học ở mức thiết kế ống kính. Thứ hai là xử lý ảnh cổ điển ở mức thuật toán chi tiết. Thứ ba là toàn bộ phần thuật toán 3D hiện đại — dựng lại cảnh, biểu diễn ngầm, hợp nhất đa cảm biến — vốn thuộc C. Nhánh nên đọc tiếp là Phần B, nơi các ràng buộc vật lý ở đây được nhìn lại như một dạng ràng buộc thiết kế, cạnh các ràng buộc về dữ liệu, tính toán và độ trễ.

## ❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Mục 1 nói kéo dài phơi sáng là cách duy nhất thật sự thêm thông tin, còn mục 3 cần đặc trưng sắc nét để tam giác hoá. Với một robot đang di chuyển, hai đòi hỏi đó xung đột ở đâu, và bạn phân xử bằng đại lượng nào?
2. Bạn muốn sinh dữ liệu “chạy đêm” từ một tập ảnh sRGB chụp ban ngày. Chuỗi thao tác đúng đi qua những trạm nào của mục 1 và mục 2, và trạm nào trong chuỗi đó không đảo ngược được, khiến phép mô phỏng chỉ còn là xấp xỉ?
3. Hãy kể ba lỗi im lặng — mỗi mục 2, 3 và 4 một lỗi — tức lỗi không sinh exception và không làm loss huấn luyện xấu đi. Với mỗi lỗi, phép kiểm nào bắt được nó trước khi triển khai?
4. Vùng chói sáng ở mục 1 và pixel thiếu dữ liệu của camera độ sâu ở mục 4 đều là mất dữ liệu *không* ngẫu nhiên. Điểm chung về mặt thống kê là gì, và vì sao cùng một cách xử lý lại không dùng được cho cả hai?
5. Random resize crop gần như vô hại với bài phân loại nhưng làm hỏng nhãn của một mạng dự đoán độ sâu metric. Cơ chế nào gây ra chuyện đó, và sửa thể nào mà vẫn giữ được phép augmentation?
6. Luận điểm gộp của chương là phần lớn domain gap sinh ra trước khi mô hình nhìn thấy dữ liệu. Bạn thiết kế thí nghiệm nào có thể *bác bỏ* luận điểm đó, và kết quả nào sẽ đủ để bác bỏ?

# Nguồn

---

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [29] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [30] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [31] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [32] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [33] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [34] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [37] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [38] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [39] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.

## Nguồn

---

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [29] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [30] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.
- [31] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [32] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [33] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [34] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [37] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [38] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [39] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.